

Z3 from First Principles

A Beginner's Guide for Leant and Djex

Logic, SMT-LIB, models, counterexamples, robust solver boundaries,
and the behavioral-synthesis roadmap

Leant snapshot	494259dbd85ae2e6749246b08377f44b2f9936ac
Djex used by Leant	7d65eba6f753727ae7b3d734fb4a7e00f1f44aa7 (the lib/Djex submodule)
Djex repository head	0cf82f2aa22debb1de2c7aaa13a2abbe190550d8
Z3 snapshot	bc4585e0ba5b1041f301b0c16ba652645729e796
Prepared	August 16, 2026 (America/Los_Angeles)
Audience	Readers who know ordinary programming but may know no formal logic, SAT, SMT, theorem proving, or Z3.

This document is a conceptual prerequisite and source-guided companion to the existing Leant manual, codebase walkthrough, rewrite analysis, and Z3 behavioral-synthesis proposal.

Abstract

Z3 is often introduced by showing three lines of Python that declare variables, add constraints, and call a solver. That is enough to produce a result, but it is not enough to understand a system such as Leant/Djex. In this codebase Z3 is not a magical oracle, not a replacement for Lean’s elaborator or kernel, and not the component that invents arbitrary programs. It is a bounded mathematical search engine behind a deliberately narrow protocol and a strict evidence boundary.

This document starts earlier than the usual Z3 tutorial. It explains what a logical formula denotes, what it means for a formula to be satisfiable, why a satisfying model can be read as a counterexample, why proving a statement is commonly reduced to asking whether its negation is unsatisfiable, what “modulo theories” adds to SAT, and why Z3 may report `sat`, `unsat`, or `unknown`. It then develops SMT-LIB from its parenthesized syntax through declarations, assertions, models, incremental scopes, assumptions, unsatisfiable cores, optimization, algebraic datatypes, bit-vectors, sequences, recursive definitions, and constrained Horn clauses.

The second half follows the actual Leant/Djex design. Djex’s current finite-list-spine Length domain seals a typed candidate together with an exact inventory, contract, provider assumptions, interpretation policy, and fingerprints; translates the counterexample condition to canonical QF_LIA; runs a capability-probed Z3 subprocess under bounded execution and parsing policies; and accepts decoded values only after independent concrete replay reproduces the violation. Leant first asks Lean to elaborate every candidate. It can then rank candidates by behavioral evidence or, under an explicit request-scoped `filter` mode, reject only candidates backed by adapter-authorized replayed counterexamples. Raw solver statuses—including `unsat`—remain observations rather than proof authority.

Finally, the document explains the planned progression from this narrow post-verification use to counterexample-guided enumeration, typed sketches with finite selector variables, semantic-signature partitioning, sound prefix pruning, richer observation domains, optimization, unsatisfiable-core diagnostics, constrained Horn clauses for recursive relational semantics, and Lean-checked proof artifacts. The goal is not to teach every Z3 feature. It is to make the reader able to inspect the present implementation, understand its safety properties and limitations, and evaluate the proposed extensions without treating either the solver or the surrounding type system as a black box.

How to read this document

The text is deliberately layered.

Parts I and II

assume no logic background. Read them in order if terms such as *model*, *theory*, *quantifier-free*, or *SMT-LIB* are unfamiliar.

Part III

explains what changes when a solver becomes a component of a long-lived, security-conscious application rather than an interactive calculator.

Part IV

maps those ideas onto the current Leant and Djex modules and gives a complete Length-domain trace.

Part V

explains the planned extensions and which Z3 capabilities each extension needs.

Part VI and the appendices

are a practical reference for reading queries, debugging failures, navigating the source, and recalling syntax.

Each *Mental model* box compresses a concept to one sentence. Each *In Leant/Djex* box explains why the concept matters to this project. *Trust boundary* boxes distinguish a useful observation from evidence that the system is authorized to act upon. Source trails use commit-pinned links; line numbers are intentionally omitted because these repositories are changing rapidly.

Current-tree status

The older Z3 behavioral-synthesis proposal described Length as ranking-only at its August 15 snapshot. The Leant tree inspected here has already added an explicit `--behavior-mode filter` path, occurrence-sealed hard selection, command-local counterexample banks, and progressive same-run filtering. This document therefore treats hard filtering as *implemented but narrow*, while full CEGIS, solver-selected typed sketches, sound prefix pruning, richer domains, and proof-producing positive verification remain future work.

Contents

Abstract	i
How to read this document	ii
I Constraint Solving from Zero	1
1 The problem Z3 solves	2
1.1 Programs compute; constraints describe	2
1.2 Formulas, assignments, and truth	2
1.3 Satisfiable and unsatisfiable	2
1.4 Validity is checked by searching for a counterexample	3
1.5 A first complete example	4
1.6 Synthesis adds another existential layer	4
2 The vocabulary of logic	5
2.1 Sorts are the logical analogue of types	5
2.2 Constants and functions	5
2.3 Terms and formulas	6
2.4 Interpreted and uninterpreted symbols	6
2.5 Free and bound variables	6
2.6 First-order logic and its boundary	6
2.7 Equality is sort-specific	7
2.8 Conditional expressions	7
2.9 Quantifiers change the difficulty class	7
3 From SAT to SMT	8
3.1 Propositional satisfiability	8
3.2 Theories add structured atoms	8
3.3 A high-level view of CDCL(T)	8
3.4 Logic names	9
3.5 Decidable does not mean cheap	9
3.6 Why narrow theories are valuable	10

4	Three answers, not two	11
4.1	The public result space	11
4.2	What sat does and does not mean	11
4.3	What unsat does and does not mean	11
4.4	What unknown means	12
4.5	Models are witnesses, not explanations	13
4.6	Evidence vocabulary used by Djex	13
II	Using Z3 through SMT-LIB	14
5	Z3 as an executable, library, and repository	15
5.1	What the Z3 repository contains	15
5.2	Four common interfaces	15
5.2.1	SMT-LIB files	15
5.2.2	Standard input	15
5.2.3	Language bindings	16
5.2.4	The fixedpoint and optimization APIs	16
5.3	Why Djex uses a subprocess protocol	16
5.4	A minimal local experiment	16
5.5	Version and capability observations	17
6	Reading and writing SMT-LIB	18
6.1	S-expressions	18
6.2	Scripts are command sequences	18
6.3	Declarations do not assert facts	19
6.4	Definitions are not assertions about unknown functions	19
6.5	Boolean connectives	19
6.6	Names and generated symbols	20
6.7	Numerals and negative values	20
6.8	Why get-value is often better than get-model	20
6.9	The model is valid only after a successful check	20
6.10	Push, pop, and reset	21
7	Boolean and linear integer arithmetic	22
7.1	Mathematical integers, not machine integers	22
7.2	Naturals are encoded inside integers	22
7.3	Linear expressions	22
7.4	Relations and disequality	23
7.5	Truncated natural subtraction	23
7.6	Natural quotient and modulo without div or mod	23
7.7	A complete counterexample query	24

7.8	Vacuity	25
7.9	Real arithmetic	25
8	Theories beyond QF_LIA	26
8.1	Equality with uninterpreted functions	26
8.2	Arrays	26
8.3	Algebraic datatypes	26
8.4	Sequences and strings	27
8.5	Bit-vectors	27
8.6	Floating point	28
8.7	Finite domains and enumerations	28
8.8	Theory combinations are not free	28
9	Quantifiers, recursion, and incompleteness	29
9.1	Universal properties	29
9.2	Why quantified reasoning is hard	29
9.3	Patterns and model-based instantiation	29
9.4	Recursive function definitions	30
9.5	When unknown is expected	30
10	Models and response parsing	31
10.1	A model interprets all relevant symbols	31
10.2	Partial display versus complete semantics	31
10.3	Exact parsing is part of correctness	31
10.4	Association matters as much as parsing	32
10.5	Independent replay	32
10.6	Counterexample simplification	32
10.7	Counterexample banks	33
11	Assumptions, unsatisfiable cores, and optimization	34
11.1	Temporary assumptions	34
11.2	Unsatisfiable cores	34
11.3	Core minimization	35
11.4	Optimization	35
11.5	Native Optimize versus iterative bounds	36
12	Constrained Horn clauses and fixedpoints	37
12.1	Why recursion needs a different view	37
12.2	A tiny Horn example	37
12.3	Counterexample trace or invariant	38
12.4	Why CHCs fit the roadmap	38

III Engineering a Solver Boundary	39
13 Compiling program behavior into formulas	40
13.1 A domain, not a universal translator	40
13.2 The semantic pipeline	40
13.3 Symbolic interpretation versus execution	41
13.4 Provider laws are assumptions with provenance	41
13.5 Exactness, over-approximation, and under-approximation	42
13.6 Contract roles	42
13.7 Closed interpretation policies	42
13.8 Failure is part of the semantics	42
14 Typed plans, canonical bytes, and fingerprints	43
14.1 Text must not become the semantic authority	43
14.2 A small typed command language	43
14.3 Canonicalization choices	44
14.4 The current Length preamble	44
14.5 Why separate check bytes and value-request bytes	45
14.6 Structural fingerprints	45
14.7 Fingerprints are association, not proof	45
14.8 Versioning strategy	45
14.9 Bounds before fingerprints	46
15 Owning the Z3 process	47
15.1 The process lifecycle is a state machine	47
15.2 No shell and no ambient environment	47
15.3 Executable admission and digest pinning	47
15.4 Capability probing	48
15.5 Causal framing	48
15.6 One query transaction	48
15.7 Budgets and deadlines	49
15.8 Query leases and maximum transactions	49
15.9 Cancellation and cleanup	49
15.10 Concurrency	49
16 From observations to evidence	51
16.1 A useful evidence ladder	51
16.2 Why counterexamples are easy to trust	51
16.3 Why positive results are harder	51
16.4 Behavioral statuses	52
16.5 Fail closed in semantics, preserve in policy	52

16.6	Selection needs stronger association than ranking	52
16.7	Batch failures preserve the original batch	52
16.8	Conditional claims	53
17	Incrementality, determinism, and CEGIS-ready state	54
17.1	Two kinds of reuse	54
17.2	Counterexample-guided inductive synthesis	54
17.3	Monotone sample growth	54
17.4	Semantic signatures	55
17.5	Determinism	55
17.6	Incremental state fingerprints	55
17.7	Recovery	56
IV	Z3 in the Current Leant/Djex System	57
18	The four-layer architecture	58
18.1	One pipeline, four different kinds of authority	58
18.2	Why type correctness is not behavioral correctness	59
18.3	Candidate groups, variants, and callback verification	59
18.4	The exact typed origin	59
18.5	What each repository contributes	60
19	The implemented finite-spine Length domain	61
19.1	Why start with length?	61
19.2	A spine rather than a built-in-name convention	61
19.3	Scalar and binary-product domains	62
19.4	The passive contract	62
19.5	Provider laws	63
19.6	Symbolic values in the Length interpreter	63
19.7	Sealing order	64
19.8	The Leant handoff in detail	64
20	From a checked Length problem to QF_LIA	66
20.1	The bad-state formula after symbolic interpretation	66
20.2	Why QF_LIA is sufficient	66
20.3	Canonical preamble and commands	67
20.4	Positive-literal quotient and modulo without SMT div or mod	67
20.5	Input-only models	68
20.6	Replay turns an assignment into a counterexample	68
20.7	What unsat means in the current system	69
20.8	Solver-independent checks before and after Z3	69

20.9	Counterexample banks	69
20.10	Canonical query identities and limits	70
20.11	An end-to-end example: one contract, four candidates	70
21	The live Z3 worker	73
21.1	Configuration is not execution	73
21.2	No shell, ambient environment, or inherited working directory	73
21.3	Lexically scoped ownership	74
21.4	Readiness probing	74
21.5	One query transaction	75
21.6	Budgets and cleanup	75
21.7	Causal parsing	76
21.8	Artifact policy	76
22	Leant's rank and filter modes	77
22.1	Behavioral assessment is disabled by default	77
22.2	Startup activation	77
22.3	Request syntax	78
22.4	Ranking mode	78
22.5	Filter mode	79
22.6	Same-run progressive filtering	79
22.7	A worked conceptual session	80
22.8	Failures are classified, not flattened	80
22.9	Current feature status	81
23	A maintainer's source map for one assessment	82
23.1	Trace from command line to verified batch	82
23.2	Trace from a verified occurrence to a query	82
23.3	Trace through live execution	82
23.4	Stage by stage: owning module and failure class	83
23.5	Trace through ranking or selection	84
23.6	Public facade versus private implementation	85
23.7	Changes between Leant's pinned Djex and Djex head	85
V	How Z3 Is Planned to Be Used Next	86
24	From post-verification assessment to behavioral synthesis	87
24.1	The sequential architecture today	87
24.2	Governing invariants	87
24.3	Five increasingly strong integration levels	88
24.4	A generic behavioral domain interface	89
24.5	Exactness is multidimensional	89
24.6	Planned mechanisms, Z3 features, and status at a glance	90

25 Counterexample-guided enumeration	91
25.1 Turning the current bank into generator feedback	91
25.2 A candidate-level CEGIS loop	91
25.3 Counterexample lifecycle	92
25.4 Sample selection and minimization	92
25.5 Distinguishing candidates with Z3	92
25.6 Interaction with completeness claims	93
25.7 A practical first milestone	93
26 Typed sketches and solver-selected productions	95
26.1 What a sketch is	95
26.2 Why selector variables are the first design	95
26.3 Exactly-one encodings	95
26.4 Semantic equations over samples	96
26.5 Worked sketch: Option.bind	96
26.6 Worked sketch: state threading	97
26.7 Model decoding and materialization	98
26.8 Why samples do not finish verification	98
26.9 Worked example: two rounds of sketch CEGIS on lengths	99
26.10 Unsat means different things in synthesis and verification	100
27 Sound pruning of incomplete search states	101
27.1 Why pruning is harder than checking a complete candidate	101
27.2 Over- and under-approximation	101
27.3 A Length interval summary	101
27.4 Abstract interpretation of a typed grammar	102
27.5 Encoding a pruning query	102
27.6 Pruning receipts	103
27.7 Nogoods over production choices	103
27.8 Monotonicity rules for reuse	104
27.9 A staged implementation	105
28 Richer behavioral domains and the Z3 theories they need	106
28.1 Domain design comes before solver selection	106
28.2 Generalized size and constructor tags	106
28.3 Sequences, indices, and order	107
28.4 Bags and permutation	107
28.5 Scalar arithmetic	108
28.6 Bit-vectors and machine arithmetic	108
28.7 State and provenance	108
28.8 Finite callbacks and higher-order observations	109

28.9	Cost and resource models	109
28.10	Finite-state protocols	109
28.11	Domain-priority order	110
29	Incremental solving, cores, optimization, and explanations	111
29.1	Why the current Length protocol should not be stretched	111
29.2	Assumptions versus push/pop	111
29.3	Contract diagnostics with unsat cores	111
29.4	Grammar cores and learned nogoods	112
29.5	Optimization	112
29.5.1	Iterative satisfiability	113
29.5.2	Native Optimize	113
29.6	Canonical models	113
29.7	Session rebuild and recovery	114
29.8	Protocol identity	114
30	Recursive programs, CHCs, and Lean-checked proofs	116
30.1	Why recursive functions are a different problem	116
30.2	Relational semantics	116
30.3	SMT-LIB fixedpoint shape	117
30.4	Counterexample traces	118
30.5	Invariant candidates	118
30.6	Lean theorem generation	119
30.7	Provider-law proof linkage	119
30.8	Termination remains a Lean concern	120
30.9	Recursive synthesis	120
30.10	Synthesiz3 as an adjacent experiment	120
30.11	What a positive result should say	120
31	A staged implementation and evaluation plan	122
31.1	Milestone 0: preserve and benchmark the current baseline	122
31.2	Milestone 1: semantic signatures	122
31.3	Milestone 2: candidate-level CEGIS	123
31.4	Milestone 3: one typed-sketch vertical slice	123
31.5	Milestone 4: incremental protocol, cores, and nogoods	123
31.6	Milestone 5: generalized observation domains	123
31.7	Milestone 6: proof artifacts	124
31.8	Milestone 7: CHC experiments	124
31.9	Benchmark scenarios	124
31.10	Metrics	124
31.11	Testing strategy	125
31.12	Non-goals	125

VI	Practical Reading and Debugging Guide	127
32	How to read one Z3 query end to end	128
32.1	Start with the semantic question	128
32.2	Identify the logic and theory profile	128
32.3	List declarations in order	128
32.4	Separate domain axioms from the bad state	129
32.5	Predict trivial models by hand	129
32.6	Read status before values	130
32.7	Replay the model manually	130
32.8	Run a standalone SMT-LIB file	130
32.8.1	Reproducing the embedded launch by hand	130
32.9	Minimize without changing the claim	131
32.10	Compare two queries structurally	131
32.11	Reading a real query and its fingerprint	132
33	Troubleshooting by symptom	134
33.1	Z3 was not launched	136
33.2	Z3 reported sat, but the candidate was retained	136
33.3	Z3 reported unsat, but Leant did not call the candidate proved	137
33.4	The same candidate receives different models	137
33.5	A candidate is unassessed	137
33.6	Filtering changed between batches	138
33.7	A valid provider is reported unavailable	138
33.8	Standalone Z3 works, embedded Z3 fails	138
33.9	The query unexpectedly returns unknown	139
33.10	The candidate is rejected in filter mode but seems correct	139
33.11	The candidate seems behaviorally wrong but survives	140
33.12	Leant refuses at startup or at the prompt	140
33.13	A batch-wide warning says assessment preserved all verified candidates	141
33.14	Reading a real assessment note	141
34	A concept crosswalk for Lean, Djex, and Z3	143
34.1	Types and sorts	143
34.2	Elaboration versus solving	144
34.3	Theorem proving versus counterexample search	144
34.4	Model versus value	144
34.5	Unsat core versus explanation	144
34.6	Optimization versus synthesis quality	144
34.7	Fixedpoint safety versus induction theorem	145
34.8	Lean automation versus Z3	145

A	SMT-LIB quick reference	146
A.1	Lexical structure	146
A.2	Core commands	146
A.3	Boolean terms	147
A.4	Integer and real arithmetic	147
A.5	Arrays	148
A.6	Bit-vectors	148
A.7	Algebraic datatypes	148
A.8	Quantifiers	148
A.9	Typical responses	148
A.10	Logic-name orientation	149
B	Complete small scripts	150
B.1	A satisfiable arithmetic puzzle	150
B.2	Proving a linear implication by refuting its negation	150
B.3	Length preservation: constant empty candidate	151
B.4	Length preservation: identity candidate	151
B.5	Length preservation: exact tail behavior	151
B.6	Euclidean quotient/remainder witnesses	151
B.7	Assumption literals and an unsat core	152
B.8	A tiny optimization problem	152
B.9	A bit-vector overflow witness	153
B.10	A fixedpoint safety query	153
C	An end-to-end Length counterexample walkthrough	154
C.1	Target and contract	154
C.2	Candidate	154
C.3	Bad state	154
C.4	Canonical translation stages	155
C.5	Live transaction	155
C.6	Response decoding	155
C.7	Independent replay	155
C.8	Ranking result	156
C.9	Filter result	156
C.10	Bank reuse	156
C.11	What was and was not established	156
D	Glossary	157

E	Repository snapshots and document map	160
E.1	Exact revisions inspected	160
E.2	Existing Leant LaTeX documents	160
E.3	Recommended reading paths	161
E.4	Where each key concept is introduced	161
E.5	Primary source paths	162
E.5.1	Leant	162
E.5.2	Djex	163
E.5.3	Z3	163
F	Further reading	164
F.1	Official Z3 material	164
F.2	Foundational perspective	164
F.3	Citation list	164
	Conclusion	166

Part I

Constraint Solving from Zero

The problem Z3 solves

1.1 Programs compute; constraints describe

Most programs are written in a forward direction. Given an input, execute a sequence of operations and produce an output. If we want the sum of two integers, we write a function such as

$$\text{add}(x, y) = x + y.$$

The caller supplies x and y ; the program determines the result.

A constraint problem reverses the emphasis. We introduce unknown values and describe relationships they must satisfy. For example:

$$x + y = 10, \quad x \geq 0, \quad y \geq 0, \quad x < y.$$

We have not specified an algorithm for choosing x and y . We have described the set of acceptable choices. A solver searches for one assignment, such as $x = 4, y = 6$, or establishes that no assignment exists.

This style is called *declarative*: state what must be true, not how to find it. It is useful whenever the unknowns are easier to characterize than to construct directly. Scheduling, configuration, symbolic execution, test generation, program verification, and bounded program synthesis all have this shape.

Mental model

A conventional function maps known inputs to an output. A constraint solver fills in unknowns so that a collection of statements becomes simultaneously true.

1.2 Formulas, assignments, and truth

A logical **formula** is an expression whose value is true or false after its free symbols have been interpreted. The expression

$$x + y = 10 \wedge x < y$$

is neither unconditionally true nor unconditionally false. It is true under $x = 4, y = 6$ and false under $x = 7, y = 3$.

An **assignment** gives values to free constants such as x and y . More generally, an **interpretation** also gives meanings to declared functions, relations, arrays, and abstract sorts. A complete interpretation under which every asserted formula is true is called a **model**.

The word “model” here does not mean a machine-learning model. It means a mathematical world that satisfies the constraints.

1.3 Satisfiable and unsatisfiable

A set of formulas is **satisfiable** if at least one model exists. It is **unsatisfiable** if no model exists.

The constraints

$$x > 0, \quad x < 3, \quad x \in \mathbb{Z}$$

are satisfiable because $x = 1$ or $x = 2$ works. The constraints

$$x > 0, \quad x < 1, \quad x \in \mathbb{Z}$$

are unsatisfiable because there is no integer strictly between 0 and 1.

Satisfiability is an existential question:

Does there exist an interpretation making all assertions true?

A solver that answers `sat` should normally be able to provide a model. A solver that answers `unsat` says the asserted conjunction has no model.

1.4 Validity is checked by searching for a counterexample

A formula F is **valid** if it is true in every interpretation. Validity is universal, whereas satisfiability is existential. Solvers are commonly used to check validity by negation:

$$F \text{ is valid} \iff \neg F \text{ is unsatisfiable.}$$

Suppose we want to establish

$$(x \geq 0 \wedge y \geq 0) \implies x + y \geq 0.$$

Instead of asking Z3 to “prove” the implication directly, assert its negation:

$$x \geq 0 \wedge y \geq 0 \wedge x + y < 0.$$

If this counterexample condition is unsatisfiable, no counterexample exists in the chosen mathematical theory. Here is the same derivation as an executable script. Each conjunct of the negation becomes its own `assert`; a set of assertions is always read as a conjunction:

```
(set-logic QF_LIA)
(declare-const x Int)
(declare-const y Int)
(assert (>= x 0))
(assert (>= y 0))
(assert (< (+ x y) 0))
(check-sat)
```

Listing 1.1: Checking validity by asserting the negation.

Z3 answers `unsat`: no nonnegative integers have a negative sum, so the implication is valid. It is instructive to break the property. Replace the last assertion by `(assert (not (> (+ x y) 0)))`, that is, ask whether $x + y > 0$ can fail, and add `(get-value (x y))` after the check. Z3 now answers

```
sat
((x 0)
 (y 0))
```

The strict version of the claim is false, and the model is exactly the counterexample a human would give. (Writing the negation as one formula, `(assert (not (=> (and (>= x 0) (>= y 0)) (>= (+ x y) 0))))`, is equivalent and also yields `unsat`; splitting it into conjuncts merely makes each hypothesis visible.)

This transformation is the bridge to behavioral checking. Let $P(x)$ be a precondition, let $C(x)$ be the result computed by a candidate, and let $Q(x, C(x))$ be the desired postcondition. The candidate is wrong exactly when

$$\text{Bad}_C(x) \equiv P(x) \wedge \neg Q(x, C(x)). \quad (1.1)$$

Ask whether Bad_C is satisfiable.

- If the answer is sat, the model proposes an input x on which the candidate violates the contract.
- If the answer is unsat, no violating input exists *in the encoded model*, assuming the translation is exact and the solver result is trusted for that purpose.
- If the answer is unknown, the attempt produced no yes/no conclusion.

In Leant/Djex

Djex's Length query is precisely a sealed instance of (1.1). The checked problem already owns the candidate's symbolic length result, the contract precondition, and the contract postcondition. The SMT-LIB layer receives the combined counterexample condition; it does not accept an arbitrary caller-written formula and an unrelated candidate.

1.5 A first complete example

Imagine a candidate function on natural numbers:

$$C(n) = n + 1$$

and a contract saying the result must equal the input:

$$Q(n, r) \equiv r = n.$$

The precondition is simply $n \geq 0$. The counterexample condition is

$$n \geq 0 \wedge n + 1 \neq n.$$

It is satisfiable; indeed every natural number is a counterexample. Z3 may return $n = 0$. The model need not be unique and need not be the smallest unless we explicitly optimize for that.

Now let $C(n) = n$. The bad state becomes

$$n \geq 0 \wedge n \neq n,$$

which is unsatisfiable.

Checkpoint

The most important equation in this document is $P \wedge \neg Q$. Whenever the later architecture seems complicated, ask: (1) who defined P and Q ; (2) how was the candidate result substituted; (3) what does a satisfying model denote; and (4) who independently checks that interpretation?

1.6 Synthesis adds another existential layer

Verification asks whether one fixed candidate has a counterexample. Synthesis asks for a candidate as well. In a bounded grammar with choice variables $\vec{s}$, samples $\vec{x}$, and interpreter Eval, a synthesis constraint has the rough shape

$$\exists \vec{s}. \bigwedge_{x \in \text{samples}} Q(x, \text{Eval}(\vec{s}, x)).$$

A model for $\vec{s}$ selects grammar alternatives. The selected term is then materialized, type-checked by Lean, and checked against additional inputs. If a new counterexample appears, add it to the sample set and solve again. This is the essence of counterexample-guided inductive synthesis, developed in Part V.

Trust boundary

A model of selector variables is not itself a Lean program. It is untrusted data naming alternatives in a grammar owned by Djex. The application must decode the choices, reconstruct the exact typed candidate, render or lower it, and ask Lean to elaborate it.

The vocabulary of logic

2.1 Sorts are the logical analogue of types

Every well-formed Z3 term has a **sort**. Familiar built-in sorts include:

`Bool, Int, Real, (_ BitVec 32), String.`

Parameterized sorts include arrays and sequences:

`(Array Int Bool), (Seq Int).`

Users can also declare uninterpreted sorts and algebraic datatypes.

Sort checking rejects ill-sorted terms: with $x : \text{Int}$ and $p : \text{Bool}$, the term $(\text{and } x \ p)$ is refused with a “Sort mismatch” error, and an `Int` can never be equated with a bit-vector (“Sorts `Int` and `(_ BitVec 8)` are incompatible”). Z3’s front end is, however, more lenient than the SMT-LIB standard in a few places: it silently inserts `to_real` when an `Int` meets a `Real`, and Z3 5.0.0 even accepts $(+ \ x \ p)$ with $p : \text{Bool}$, reading it as $x + \text{ite}(p, 1, 0)$ instead of reporting an error. Generated queries should never rely on such coercions. Sort checking resembles static type checking, but Z3’s sort system is much simpler than Lean’s dependent type theory. It has no universe polymorphism, dependent function type, implicit argument elaboration, or type-class synthesis.

Common mistake

Do not identify a Z3 sort with an arbitrary Lean type. A domain designer chooses a mathematical abstraction. A Lean `List a` may be represented only by its natural-number length; an arbitrary state type may be represented by an uninterpreted sort; a `UInt32` may be represented by a 32-bit bit-vector. The abstraction is part of the contract.

2.2 Constants and functions

A logical constant is a function of zero arguments. Declaring

$$x : \mathbb{Z}$$

introduces a symbol whose value is to be chosen by a model. A function declaration

$$f : \mathbb{Z} \rightarrow \mathbb{Z}$$

introduces a total mathematical function. “Uninterpreted” means no predefined meaning is attached beyond the constraints and the ordinary equality laws.

For example, from $x = y$ a solver may infer $f(x) = f(y)$. This is **congruence**. It cannot infer $f(x) = x$ unless that is asserted or follows from other axioms.

Logical functions are pure and total. They do not throw exceptions, mutate state, perform IO, or diverge. Even operations that programming languages may leave undefined are assigned a logical meaning by their theory, sometimes an under-specified but still total one.

In Leant/Djex

The planned state/provenance domain can model callback arguments as uninterpreted total functions. This is excellent for distinguishing “pass the updated state” from “reuse the old state.” It is not automatically a theorem about effects, exceptions, or nontermination of arbitrary Lean functions.

2.3 Terms and formulas

A **term** denotes a value. Examples are

$$x + 1, \quad f(x), \quad \text{ite}(p, a, b).$$

A **formula** is simply a term of sort `Bool`. Examples are

$$x < 10, \quad f(x) = y, \quad p \wedge \neg q.$$

Z3’s abstract syntax is a directed acyclic graph internally; shared subexpressions need not be duplicated. SMT-LIB presents expressions as nested S-expressions.

2.4 Interpreted and uninterpreted symbols

Symbols such as integer addition and order have fixed meanings supplied by a theory. They are **interpreted**. A user-declared function f has no fixed meaning and is **uninterpreted**. Formulas may combine both:

$$x < y \wedge f(x) = f(y) + 1.$$

The arithmetic theory constrains $<$ and $+$; equality with uninterpreted functions constrains f through congruence and the explicit equation.

2.5 Free and bound variables

A symbol such as x in an assertion is free: the solver chooses its interpretation. A quantifier binds variables:

$$\forall x : \mathbb{Z}. f(x) \geq 0.$$

The x inside the body is bound by $\forall$. An existential quantifier similarly states that some value exists.

At the top level, free constants are already existential for satisfiability purposes. Declaring x and asserting $x > 0$ asks whether there exists a value of x greater than zero. There is no need to write an explicit existential quantifier.

2.6 First-order logic and its boundary

Ordinary SMT reasoning is predominantly first-order. Functions can be declared and applied, but functions are not generally passed as ordinary values, quantified over without restrictions, or returned as arbitrary higher-order results. Arrays and lambda expressions provide function-like values in specialized ways, and Z3 supports extensions, but the dependable core remains first-order.

Lean is fundamentally higher-order and dependent. The Leant/Djex bridge therefore does not translate arbitrary Lean propositions wholesale. Each semantic domain defines a first-order observation language and refuses terms whose demanded behavior lies outside it.

Mental model

Lean reasons about programs and proofs in a rich dependent type theory. Z3 reasons about a deliberately chosen first-order mathematical shadow of the program.

2.7 Equality is sort-specific

Equality compares two terms of the same sort. For Booleans, equality is logical equivalence. For algebraic datatypes it is structural equality in the datatype theory. For arrays it is extensional equality: arrays are equal when all indexed values agree. For uninterpreted sorts it is an equivalence relation with no extra structure.

This is powerful, but it makes modeling choices consequential. If two Lean values are represented by one opaque token, equality of those tokens means equality in the abstraction, not necessarily definitional or propositional equality in Lean.

2.8 Conditional expressions

The expression

$$\text{ite}(c, t, e)$$

means “if c then t else e .” Both branches must have the same sort. Unlike an imperative branch, it is a pure expression. Conditional terms are central to symbolic interpretation of pattern matching.

For a list-spine case whose only observation is length n , the current Length semantics has the conceptual form

$$\text{caseLen}(n, z, s) = \text{ite}(n = 0, z, s(n \dot{-} 1)),$$

where $\dot{-}$ is natural monus, or truncated subtraction.

2.9 Quantifiers change the difficulty class

Quantifier-free fragments over supported theories often have complete decision procedures. General quantified first-order logic is undecidable. Z3 can still solve many quantified problems using pattern-based instantiation, model-based quantifier instantiation, and special fragments, but termination and completeness are no longer generally available.

This is why the implemented Length translator emits QF_LIA: quantifier-free linear integer arithmetic. It converts naturals to integers plus nonnegativity constraints and even lowers natural quotient/modulo by explicit witness equations rather than emitting general quantifiers.

Checkpoint

By this point, “Z3 variable” should mean a free logical constant, not a mutable storage cell; “function” should mean a total mathematical mapping; and “type” in the Z3 layer should normally be called a sort.

From SAT to SMT

3.1 Propositional satisfiability

Propositional logic has Boolean variables and connectives such as not, and, or, and implication. A SAT problem asks whether a Boolean formula has a satisfying truth assignment. For example,

$$(p \vee q) \wedge (\neg p \vee q) \wedge (p \vee \neg q)$$

is satisfiable with $p = q = \text{true}$.

Modern SAT solvers use conflict-driven clause learning (CDCL). Very roughly, they choose truth values, propagate consequences, detect conflicts, learn clauses that prevent repeating the same failed region, and backtrack nonchronologically. SAT is NP-complete, yet industrial solvers routinely handle enormous structured instances.

3.2 Theories add structured atoms

In SAT, an atom is opaque: a is just true or false. In SMT, atoms may have theory meaning:

$$x < y, \quad f(x) = z, \quad \text{select}(A, i) = 3.$$

A SAT-level assignment could set both $x < y$ and $y < x$ to true, but the arithmetic theory knows they cannot both hold. “Satisfiability modulo theories” means satisfiability while respecting such background semantics.

Common theories include:

- equality with uninterpreted functions (EUF);
- integer and real arithmetic;
- fixed-width bit-vectors;
- arrays;
- algebraic datatypes;
- strings and sequences;
- floating-point arithmetic.

3.3 A high-level view of CDCL(T)

Z3’s general SMT solver combines a Boolean search core with theory solvers. The Boolean core reasons about which atoms are true or false. Theory solvers check whether the currently selected atoms are jointly consistent in their domains. When a theory detects a contradiction, it explains the conflict back to the Boolean core, which learns a clause.

For example, the Boolean core may tentatively select

$$x < y, \quad y < z, \quad z \leq x.$$

The arithmetic solver detects inconsistency and returns a theory conflict. The core learns not to select that combination again.

The actual architecture is substantially more sophisticated: preprocessing, equality propagation, specialized solvers, tactics, model construction, and theory combination all matter. The important beginner-level point is that Z3 is not enumerating all integers. It performs symbolic search with domain-specific reasoning.

3.4 Logic names

SMT-LIB names standard fragments with compact identifiers. The name QF_LIA decomposes as follows:

QF	quantifier-free
LIA	linear integer arithmetic

“Linear” means variables are not multiplied by one another and do not occur with variable exponents. Constant scaling such as $7x$ is linear; xy and x^2 are nonlinear. Related names include QF_LRA for linear real arithmetic, QF_UFLIA for uninterpreted functions plus linear integer arithmetic, QF_BV for quantifier-free bit-vectors, and QF_AUFLIA for arrays plus uninterpreted functions plus linear integer arithmetic. Two Z3-specific names are also common: ALL enables every theory Z3 knows, and HORN selects the constrained-Horn-clause engine of Chapter 12.

The set-logic command communicates the intended fragment. It can select specialized behavior, validate feature use, and document the encoding. It is not merely a comment.

Common mistake

Z3 really does check the declared logic, and it does not stop after complaining. Under (set-logic QF_LIA), declaring an uninterpreted function (declare-fun f (Int) Int) yields (error "... logic does not support uninterpreted functions"), a quantifier yields logic does not support quantifiers, and a product of two variables yields logic does not support nonlinear arithmetic. The dangerous part is what happens next: the offending command is discarded and the script continues.

```
(set-logic QF_LIA)
(declare-const x Int)
(assert (> x 5))
(assert (< (* x x) 10))
(check-sat)
(get-value (x))
```

Z3 5.0.0 prints

```
(error "line 4 column 22: logic does not support nonlinear arithmetic")
sat
((x 6))
```

The sat and the “model” $x = 6$ refer to the remaining assertion only; the intended problem $(x > 5 \wedge x^2 < 10)$ is unsatisfiable. A driver that scans stdout for sat and then reads values would accept a counterexample for a query that was never checked. Any (error ...) line must abort the whole transaction, and the exit code (Z3 exits with 1 after an error) is not enough because the wrong answers have already been printed. Djex’s response parser rejects such transcripts; a hand-written harness must do the same.

3.5 Decidable does not mean cheap

A fragment may have a decision procedure and still be computationally difficult. Conversely, an undecidable language may be solved quickly on many practical instances. Three questions must remain separate:

1. Is the fragment decidable in principle?
2. Is Z3 complete for the exact combination and options being used?
3. Will this particular instance finish within the application's resource budget?

Leant/Djex imposes explicit byte, node, numeral, query-count, solver-time, resource, and host-deadline bounds because the third question is operational, not merely logical.

3.6 Why narrow theories are valuable

A rich encoding can be shorter to write but harder to solve, harder to audit, and harder to replay. The Length translator deliberately uses a small arithmetic language. This yields:

- a simple typed intermediate representation;
- deterministic rendering;
- a bounded response grammar;
- straightforward independent evaluation;
- fewer solver-version surprises;
- clearer exactness claims.

In Leant/Djex

Z3's breadth is a menu, not a mandate. Each future behavioral domain should select the smallest theory profile that represents its observations faithfully. Bit-vector synthesis should use bit-vectors; state wiring may need only EUF and tuples; list-length behavior should stay in linear arithmetic.

Three answers, not two

4.1 The public result space

A Z3 satisfiability check has three semantically important outcomes:

sat A satisfying model was found for the asserted formulas.

unsat

Z3 determined that no satisfying model exists.

unknown

Z3 did not establish either result under the selected procedure and resources.

A timeout, cancellation, resource limit, transport failure, malformed response, or capability mismatch is not logically identical to unknown. An application should preserve these operational distinctions even if its user-facing policy ultimately groups them as “inconclusive.”

4.2 What sat does and does not mean

A sat result means the formula has a model according to the solver. It does not mean:

- the model is unique;
- values are minimal or aesthetically simple;
- the application encoded its intended semantics correctly;
- parsed output is well-associated with the candidate the caller thinks it checked;
- a model fragment remains valid after the solver state changes.

For counterexample search, sat is exceptionally useful because the application can ask for the exact input values, decode them, and replay the candidate independently. Replay turns a solver suggestion into domain-owned evidence.

4.3 What unsat does and does not mean

Within a correct exact encoding and a trusted decision procedure, unsat means no counterexample exists in the model. But several layers can invalidate the leap from this statement to “the Lean program satisfies the user’s intention”:

- the domain may observe only length, not elements;
- provider transfer laws may be assumptions;
- unsupported source constructs may have been approximated;

- the contract may be vacuous because its precondition has no satisfying inputs;
- the arithmetic model may differ from Lean’s runtime representation;
- the application may not treat the external solver as proof authority.

The current Djex generic behavioral envelope therefore classifies every raw solver result, including `unsat`, as heuristic-ranking-only. Stronger positive evidence comes from exhaustive domain replay or from a proof checked by Lean.

Trust boundary

A replayed counterexample is asymmetric evidence: one concrete witness is enough to refute a universal claim. A raw `unsat` result is a global positive claim whose force depends on the entire encoding and trust chain. The current architecture exploits this asymmetry deliberately.

4.4 What unknown means

unknown means “the selected procedure did not return `sat` or `unsat`.” In SMT-LIB, the command `(get-info :reason-unknown)` issued after an unknown answer returns a short diagnostic string—in Z3 5.0.0 typically “timeout” (a soft `:timeout` expired), “canceled” (an `:rlimit` resource budget was exhausted), or a text such as “(incomplete quantifiers)” (the procedure gave up)—but the reason is diagnostic, not a theorem. Quantifiers, nonlinear arithmetic, incomplete theory combinations, and configured time/resource limits are common causes.

Unknown should be a first-class policy case. For hard behavioral filtering, the safe default is to retain the candidate. For ranking, an unknown observation may place the candidate in a neutral tier. For an explicitly strict mode, the user may choose to reject inconclusive candidates, but that is a policy decision, not logical evidence of failure.

What the outcomes look like on the wire

Table 4.1 lists what a driver actually observes for each outcome, verified with Z3 5.0.0. The scripts are small: the “timeout” and “canceled” rows use the nonlinear query $x^3 + y^3 = z^3$ with $x, y, z > 0$ (Fermat’s equation for exponent 3, which Z3 cannot decide), preceded by `(set-option :timeout 1000)` or `(set-option :rlimit 100000)`; the “incomplete quantifiers” row uses

```
(set-option :smt.mbqi false)
(set-option :auto_config false)
(declare-fun f (Int) Int)
(assert (forall ((x Int)) (! (>= (f x) 0) :pattern ((f x)))))
(assert (= (f 3) 5))
(check-sat)
(get-info :reason-unknown)
```

which is satisfiable (with model-based instantiation left on, Z3 answers `sat` and prints the model $f = [3 \mapsto 5, \text{else} \mapsto 0]$), but pattern-based instantiation alone cannot establish either answer.

Outcome	Observed output	What it does and does not tell you
sat	sat, then values on request	A model exists for the assertions as encoded. Nothing about uniqueness, minimality, or the fidelity of the encoding.
unsat	unsat; get-model now errors	No model exists for the assertions as encoded. Nothing about vacuity or encoding fidelity.
unknown (soft timeout)	unknown, then (:reason-unknown "timeout")	The :timeout budget expired. The instance may be trivially sat or unsat under a larger budget.
unknown (resource limit)	unknown, then (:reason-unknown "canceled")	The :rlimit budget expired. Deterministic across runs of the same build, unlike wall-clock timeouts.
unknown (incompleteness)	unknown, then e.g. (:reason-unknown "smt tactic failed to show goal to be sat/unsat (incomplete quantifiers)")	The procedure gave up without exhausting a budget; more time will not help, a different encoding or option might.
Hard timeout (z3 -T:5)	The single line timeout; the process exits	Not an SMT-LIB status at all. No get-info is possible because the process is gone.
Error	(error "...") lines interleaved with normal answers	A command was rejected; later answers refer to a different problem than intended.
No output / partial output	(nothing, or a truncated S-expression)	Crash, kill, or transport failure. Treat as a driver-level failure, not as unknown.

Table 4.1: Solver outcomes as observed by a driver, and their evidential value.

4.5 Models are witnesses, not explanations

A model demonstrates satisfiability, but it may not explain why the formula is satisfiable in a human-friendly way. Uninterpreted function models are finite tables with a default (“else”) value, which Z3 prints as a nested `ite` chain inside a `define-fun`. Arrays may be represented by stores over a constant array. Algebraic values may contain generated names. Z3 is free to choose any model.

If the application needs a small counterexample, it can simplify after replay, add optimization objectives, or search a bounded box in deterministic order. Djex’s Length pipeline includes query-owned counterexample simplification and finite input-box validation rather than assuming Z3 returns the nicest values.

4.6 Evidence vocabulary used by Djex

The current source distinguishes at least the following ideas:

- a raw `SolverObservation` with status-specific artifact;
- a raw `BehavioralObservation` such as established, violated, validated-within, or unknown;
- an `AssociatedObservation` bound to exact domain, inventory, encoding, candidate, and problem fingerprints;
- opaque `BehavioralEvidence` created only by a domain-owned verifier;
- replay gates that verify exact association before exposing the receipt.

Association is not certification. A perfectly associated `unsat` response can still be only a heuristic observation. This vocabulary prevents accidental authority inflation in ordinary Haskell code.

Source trail

See [synthesis/src/Language/Haskell/Synthesis/Semantic/Observation.hs](#), [synthesis/src/Language/Haskell/Synthesis/Semantic/Problem.hs](#), and [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib.hs](#).

Part II

Using Z3 through SMT-LIB

Z3 as an executable, library, and repository

5.1 What the Z3 repository contains

The Z3 repository is the implementation of a family of reasoning engines behind several interfaces. The top-level [README.md](#) documents source builds and language bindings. The most relevant areas for this document are:

src/

The C++ implementation: SAT and SMT cores, theory solvers, tactics, fixedpoint engines, optimization, parsers, command front ends, and APIs.

src/api/

The public C API and wrappers. Higher-level language bindings ultimately proxy this API or equivalent generated surfaces.

examples/

Examples for C, C++, .NET, Java, Python, OCaml, Go, SMT-LIB, MaxSAT, and user propagators.

scripts/ and build files

Source-generation and build tooling for Visual Studio, Make, CMake, Bazel, and packaging.

RELEASE_NOTES.md

Changes across releases. Solver behavior and supported parameters can evolve, so a production integration should bind the observed executable and protocol profile rather than rely on a bare command name.

A reader of Leant/Djex does not need to understand Z3's C++ internals. It is useful, however, to remember that “Z3” names a substantial toolkit rather than one algorithm. A general SMT solver, a SAT solver, optimization engines, a fixedpoint/CHC engine, and tactic pipelines coexist in the same distribution.

5.2 Four common interfaces

5.2.1 SMT-LIB files

Write a text file, conventionally with extension `.smt2`, and pass it to the executable:

```
z3 example.smt2
```

This is the easiest interface to archive, inspect, minimize, and reproduce. It is also language-neutral.

5.2.2 Standard input

Run Z3 in interactive input mode and send commands through a pipe:

```
z3 -in
```

A host process can keep the child alive, submit multiple transactions, and parse replies. This is the basic transport style used by the current Djex Length integration, wrapped in much stronger ownership and framing machinery than an ad hoc pipe.

5.2.3 Language bindings

Z3 ships or documents bindings for C, C++, Python, .NET, Java, OCaml, Go, Julia, and other environments. Bindings construct typed abstract syntax directly and avoid text parsing on the request side. They are convenient for application development and interactive exploration.

5.2.4 The fixedpoint and optimization APIs

Some capabilities have specialized APIs. A fixedpoint context declares relations, rules, and queries; an optimization context adds hard constraints, soft constraints, and objectives. SMT-LIB also exposes corresponding command languages, but their state machines differ from an ordinary solver.

5.3 Why Djex uses a subprocess protocol

An in-process binding is not automatically superior. The current design benefits from a process boundary:

- solver crashes, memory faults, and nontermination are isolated from Leant;
- the executable can be inspected and optionally pinned by digest;
- host deadlines and cleanup escalation can be enforced externally;
- request and response bytes are bounded and fingerprinted;
- the exact SMT-LIB program is reproducible independently;
- no Haskell FFI lifetime or Z3-context thread-affinity leaks into public semantic APIs;
- different solver builds can be tested without relinking Djex.

The cost is that the application must own process creation, protocol synchronization, output parsing, cancellation, resource limits, and cleanup. Djex's implementation treats these as semantic-adjacent correctness concerns, not incidental shell scripting.

In Leant/Djex

`synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Execution.hs` seals pure launch and resource policy. `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs` lends a capability-probed worker only within a lexical scope and exposes neither process handles nor raw transcripts to callers.

5.4 A minimal local experiment

Create `hello.smt2`:

```
(set-logic QF_LIA)
(set-option :produce-models true)

(declare-const x Int)
(declare-const y Int)

(assert (= (+ x y) 10))
(assert (>= x 0))
(assert (>= y 0))
(assert (< x y))

(check-sat)
```

```
(get-value (x y))  
(exit)
```

Listing 5.1: A complete first SMT-LIB script.

Z3 5.0.0 answers:

```
sat  
((x 4)  
 (y 6))
```

Note the layout: `get-value` prints one `(symbol value)` pair per line inside one outer pair of parentheses, so a response parser must read S-expressions, not lines. The model is allowed to choose any satisfying pair; another Z3 build or another random seed could legitimately print `((x 0) (y 10))`. If one needs x and y to be as close as possible, or y minimal, that preference must be encoded as an optimization objective (Section 11.4) or enforced by a deterministic outer search.

Two details of the script are worth knowing. `(set-option :produce-models true)` is required by the SMT-LIB standard before `get-value` may be used; Z3 happens to produce models by default, so the script works without it, but a portable script should keep it. `(exit)` is optional when running a file: Z3 stops at end of input anyway.

5.5 Version and capability observations

A robust application should distinguish at least four identities:

1. the path or file opened as the executable;
2. the bytes or digest observed for that file;
3. the solver's reported build/version information;
4. the capabilities actually demonstrated by a startup probe.

A version string alone does not prove that the process at a path is the expected executable, that startup options were accepted, or that the response framing expected by the host works. Conversely, a matching digest does not by itself prove that the configured resource arguments and host environment are correct. Djex binds these observations in a live execution envelope separate from the solver-neutral query fingerprint.

Reading and writing SMT-LIB

6.1 S-expressions

SMT-LIB uses S-expressions: a parenthesized operator followed by arguments. In conventional notation one writes

$$(x + 1) < y.$$

In SMT-LIB one writes

```
(< (+ x 1) y)
```

The outermost list applies `<` to two arguments. The first argument is itself an application of `+`. Prefix notation eliminates precedence rules: every grouping is explicit.

Atoms include symbols, keywords beginning with a colon, numerals, decimal/rational forms, bit-vector literals, and strings. A semicolon begins a line comment.

6.2 Scripts are command sequences

An SMT-LIB script is not a general-purpose program. It is a sequence of commands that mutate a solver context or request information. There are no ordinary loops, mutable variables, or user-defined effects. The principal commands are:

Command	Purpose
<code>(set-logic L)</code>	Select or declare the intended logic fragment.
<code>(set-option :k v)</code>	Configure a solver or output option.
<code>(declare-const x S)</code>	Declare a free constant of sort <i>S</i> .
<code>(declare-fun f (S1 ... Sn) R)</code>	Declare a total function from argument sorts to result sort.
<code>(define-fun f (...) R body)</code>	Define a nonrecursive macro/function.
<code>(assert p)</code>	Add Boolean formula <i>p</i> to the current assertion set.
<code>(check-sat)</code>	Ask whether the current assertions are satisfiable.
<code>(check-sat-assuming (...))</code>	Check under temporary Boolean assumptions.
<code>(get-value (t1 ... tn))</code>	Evaluate selected terms in the current model.
<code>(get-model)</code>	Request a representation of the complete model exposed by the solver.
<code>(push 1) / (pop 1)</code>	Save and restore assertion-stack scopes; a <code>pop</code> also removes declarations made inside the popped scope.
<code>(reset)</code>	Reset declarations, assertions, and much solver state. (In Z3 5.0.0, options set through <code>set-option</code> , such as <code>:timeout</code> or <code>:pp.decimal</code> , survive a <code>reset</code> .)
<code>(reset-assertions)</code>	Remove assertions while retaining declarations where supported.
<code>(get-unsat-core)</code>	After <code>unsat</code> , list the named assertions or assumptions used (Chapter 11).
<code>(get-info :reason-unknown)</code>	After <code>unknown</code> , return a diagnostic string.

Command	Purpose
(echo "text")	Print a string; useful for marking transaction boundaries in a transcript.
(define-fun-rec f (...) R body)	Define a recursive function (Chapter 9).
(declare-datatypes ...)	Declare algebraic datatypes (Chapter 8).
(minimize t) / (maximize t) / (assert-soft p)	Optimization objectives and soft constraints (Chapter 11).
(exit)	End the command session.

6.3 Declarations do not assert facts

The command

```
(declare-const n Int)
```

introduces an arbitrary mathematical integer. It does not initialize `n` to zero and does not say `n` is nonnegative. If `n` represents a natural number, add

```
(assert (>= n 0))
```

This small distinction is one of the most common sources of incorrect encodings.

6.4 Definitions are not assertions about unknown functions

A definition such as

```
(define-fun max2 ((x Int) (y Int)) Int
  (ite (>= x y) x y))
```

is an abbreviation with a fixed body. By contrast,

```
(declare-fun max2 (Int Int) Int)
```

introduces an arbitrary function named `max2`. It has no connection to maximum unless axioms are added.

6.5 Boolean connectives

The principal Boolean forms are:

```
(and p q r)
(or p q r)
(not p)
(=> p q)
(xor p q)
(= p q) ; Boolean equivalence when p and q are Bool
(distinct a b c)
(ite p t e)
```

Both `and` and `or` can accept several operands. The equality operator is overloaded by `sort` and compares same-sorted terms.

6.6 Names and generated symbols

Simple symbols have lexical restrictions. Arbitrary names can be quoted with vertical bars, for example `|a name with spaces|`. Generated integrations should avoid embedding source names directly when a small deterministic symbol scheme suffices. This reduces escaping bugs, response ambiguity, information leakage, and fingerprint instability.

Djex derives canonical input symbols solely from the sealed input arity. It asks Z3 to return values only for those symbols. Private Euclidean quotient/remainder witnesses may exist in the query, but they never appear in the public model-binding surface.

6.7 Numerals and negative values

Natural numerals such as 0, 17, and 1000 are atoms. A negative integer is conventionally represented as an application:

```
(- 3)
```

Z3's parser also tolerates `-3` on input, but on output it always prints negative values in the application form: `(get-value (x))` for a model with $x = -4$ prints `((x (- 4)))`. Response parsers should follow the accepted grammar rather than assume every integer arrives as a JSON-style signed token. They must also impose explicit bit-width or byte limits before converting attacker- or child-controlled decimal text into unbounded host integers.

6.8 Why `get-value` is often better than `get-model`

`get-model` can return interpretations for every relevant constant and function, with solver-chosen auxiliary names and representations. This is useful interactively but creates a broad parser and association surface.

`get-value` requests exactly the terms the application needs:

```
(get-value (in0 in1 in2))
```

For a counterexample, the Length domain needs only source-ordered input lengths. It independently recomputes the candidate result. Requesting only inputs makes the response smaller and prevents a solver-reported candidate result from bypassing replay.

Trust boundary

Never ask the solver for a derived result merely because recomputing it is inconvenient, then treat that result as evidence. The independent evaluator must own the semantics whose violation is being certified.

6.9 The model is valid only after a successful check

A value or model request is meaningful only after a satisfiable check in the same coherent solver state. Asking for a model after `unsat`, after `unknown`, after a `reset`, or after unrelated assertions have changed is a protocol error or at least application misuse. Z3 reports it as an error, and—importantly for a driver—keeps executing the rest of the script:

```
(declare-const x Int)
(assert (> x 0))
(get-value (x))
(check-sat)
(get-value (x))
```

Listing 6.1: Model requests outside a satisfiable state. Z3 5.0.0 prints an error line for the premature request, then continues.

```
(error "line 3 column 14: model is not available")
sat
```

```
((x 1))
```

The same `model is not available` error is printed for `(get-model)` after an `unsat` answer, for `(get-value ...)` after an unknown answer that produced no candidate model, and for `(get-value ...)` issued after a `pop` that discarded the scope in which the last `check-sat` ran. The process exit code becomes 1 once any error has been printed, but the remaining commands still run and still print answers.

A robust driver therefore treats “check, inspect status, conditionally request values, consume exact response” as one transaction. Djex’s causal protocol associates output with an ordinal/barrier internally so that stale output cannot be mistaken for the current query.

6.10 Push, pop, and reset

Incremental solving reuses a stable base:

```
(declare-const x Int)
(assert (>= x 0))

(push 1)
(assert (< x 5))
(check-sat)
(pop 1)

(push 1)
(assert (> x 100))
(check-sat)
(pop 1)
```

Declarations and base assertions remain while scoped assertions are removed. This can be much faster than recreating a solver, but it also introduces state-contamination risks. A lost `pop`, an unexpected error, or a partially consumed response can corrupt later transactions. Z3 5.0.0 answers `sat` twice for the script above ($x = 0$ satisfies the first scope, $x = 101$ the second); without the `pop` between them, the second check would be `unsat` because $x < 5$ and $x > 100$ would both be in force. Two consequences of scoping are easy to overlook. First, a declaration made inside a scope disappears with it: declaring `y` after `(push 1)` and mentioning it after `(pop 1)` produces (error “... unknown constant y”). Second, the model of a check performed inside a scope is gone after the `pop`; a `get-value` issued then produces `model is not available` until a new `check-sat` runs (Section 6.9). Popping more levels than were pushed is also an error (invalid `pop` command, argument is greater than the current stack depth).

The current Length live protocol uses a reset-oriented, serialized transaction profile and probes that profile at startup. The planned typed-sketch/CEGIS protocol will need a new explicitly fingerprinted incremental state machine; it should not quietly repurpose the Length protocol.

Boolean and linear integer arithmetic

7.1 Mathematical integers, not machine integers

SMT-LIB `Int` denotes unbounded mathematical integers. Addition never overflows. Multiplication by a constant is exact. This is appropriate for list lengths and many symbolic size measures.

It is not appropriate for exact `UInt32` or `Int64` behavior when wraparound, truncation, signed shifts, or overflow matter. Those require bit-vectors or an explicit bounded-integer model.

7.2 Naturals are encoded inside integers

SMT-LIB has no universal built-in `Nat` sort in the ordinary arithmetic logics. Encode $n \in \mathbb{N}$ as

$$n \in \mathbb{Z} \quad \wedge \quad n \geq 0.$$

For several inputs, assert nonnegativity separately:

```
(declare-const in0 Int)
(declare-const in1 Int)
(assert (<= 0 in0))
(assert (<= 0 in1))
```

After decoding a model, Djex rejects any negative input even though the query already asserted nonnegativity. This is deliberate defense in depth: the response is untrusted and must match the exact requested representation.

7.3 Linear expressions

Linear integer expressions are built from constants, variables, addition, subtraction, and multiplication by fixed integer coefficients:

$$3x - 2y + 7.$$

The following are nonlinear:

$$xy, \quad x^2, \quad 2^x.$$

An `ite` whose branches are linear expressions is allowed in Z3's broader input language and can be compiled into linear constraints by the solver. The current translator represents conditionals explicitly in its typed QF-LIA AST.

7.4 Relations and disequality

Arithmetic atoms include:

```
(= x y)
(not (= x y))
(< x y)
(<= x y)
(> x y)
(>= x y)
```

SMT-LIB does not require a separate `!=` operator; disequality is commonly `(not (= ...))` or `distinct`.

7.5 Truncated natural subtraction

Ordinary integer subtraction can be negative. Natural monus is

$$x \dot{-} y = \max(x - y, 0).$$

It can be defined without leaving linear arithmetic:

```
(define-fun nat-monus ((x Int) (y Int)) Int
  (ite (<= y x) (- x y) 0))
```

Djex's canonical preamble defines helpers for natural monus, integer minimum, and integer maximum. The exact helper definitions are part of the typed query plan and query fingerprint.

7.6 Natural quotient and modulo without div or mod

For a positive constant divisor k , Euclidean division is characterized uniquely by fresh integers q, r :

$$e = kq + r, \quad q \geq 0, \quad 0 \leq r < k.$$

Then $q = \lfloor e/k \rfloor$ and $r = e \bmod k$ for nonnegative e .

The Length translator uses deterministic private witnesses of this form for positive-literal natural quotient and modulo. It deliberately does not emit SMT-LIB `div` or `mod`. This keeps the target logic within its chosen typed lowering discipline and makes witness semantics explicit.

A schematic lowering is:

```
(declare-const q!0 Int)
(declare-const r!0 Int)
(assert (<= 0 q!0))
(assert (<= 0 r!0))
(assert (< r!0 3))
(assert (= e (+ (* 3 q!0) r!0)))
```

Only original input symbols are requested from the model. The evaluator recomputes quotient/modulo independently during replay.

Common mistake

Beginners often reach for the built-in `div` and `mod` instead of witness equations, and are then surprised by their semantics. SMT-LIB integer division is *Euclidean*: the remainder is always nonnegative, so the quotient rounds toward $-\infty$ for a positive divisor and toward $+\infty$ for a negative one. Verified with Z3 5.0.0:

```
(declare-const a Int)
(declare-const b Int)
(assert (= a (div (- 7) 2)))
(assert (= b (mod (- 7) 2)))
(check-sat)
(get-value (a b (div 7 (- 2)) (mod 7 (- 2))))
```

```
sat
((a (- 4))
 (b 1)
 ((div 7 (- 2)) (- 3))
 ((mod 7 (- 2)) 1))
```

C, Java, and Haskell’s quot/rem truncate toward zero ($-7/2 = -3$ with remainder -1); Haskell’s div/mod floor ($7 \text{ ‘div’ } (-2)$ is -4 with remainder -1). Neither matches SMT-LIB in general, so “the same” expression can denote different integers in the query and in the replay evaluator. Division by zero is worse: SMT-LIB makes $(\text{div } x \ 0)$ and $(\text{mod } x \ 0)$ *total but unconstrained*, so the solver may pick any value. The script

```
(declare-const a Int)
(assert (= a (div 7 0)))
(assert (= (div 8 0) 42))
(check-sat)
(get-value (a (div 8 0)))
```

is sat (Z3 prints $((a \ 42) ((div \ 8 \ 0) \ 42))$); a query with an unguarded division can thus be satisfied by an assignment that no program would ever compute. Explicit witness equations with a positive literal divisor, as in the lowering above, sidestep both problems and keep the query in a fragment whose semantics the replay evaluator can reproduce exactly.

7.7 A complete counterexample query

Suppose the input is a list length n , and a candidate behaves like `tail`: it returns length 0 when $n = 0$, otherwise $n - 1$. The contract requires length preservation. The bad state is

$$n \geq 0 \wedge \text{ite}(n = 0, 0, n - 1) \neq n.$$

A pedagogical SMT-LIB script is:

```
(set-option :produce-models true)
(set-option :random-seed 1)
(set-logic QF_LIA)

(declare-const in0 Int)
(assert (<= 0 in0))

(define-fun candidate-result () Int
  (ite (= in0 0) 0 (- in0 1)))

(assert (not (= candidate-result in0)))
(check-sat)
(get-value (in0))
```

Listing 7.1: Counterexample search for a tail-like candidate under length preservation.

Z3 may answer:

```
sat
((in0 1))
```

Independent replay evaluates the actual checked candidate graph at length 1, obtains result length 0, verifies the precondition, and verifies $0 \neq 1$. Only then does the input become a validated counterexample.

For an identity-like or reverse-like candidate, the bad state normalizes to a contradiction and Z3 returns `unsat`. In the current trust policy this is useful ranking information, not a Lean proof of length preservation.

7.8 Vacuity

Consider a contract with precondition $n < 0$ while n is modeled as natural. No input is applicable. Then

$$P(n) \wedge \neg Q(n, C(n))$$

is unsatisfiable for every candidate. A naive system might call every candidate correct.

Djex’s applicable-domain validators separately track whether any assignment satisfied the precondition. Positive receipts record applicable-assignment counts, allowing a non-vacuous result to be preferred over a vacuous one. Contract design must make this distinction visible.

Common mistake

“No counterexample found” can mean at least four different things: the property holds; the checked finite region contains no counterexample; the precondition is empty; or the solver did not complete. Never collapse these into one Boolean.

7.9 Real arithmetic

SMT-LIB `Real` denotes exact mathematical reals, represented symbolically by rational and algebraic values. It is not IEEE floating point; Z3 prints the rational one half as `(/ 1.0 2.0)` unless `:pp.decimal` is set. The SMT-LIB standard requires explicit coercions between the sorts: `(to_real n)` embeds an integer, `(to_int r)` rounds toward $-\infty$ (so `(to_int (- 2.7))` is `-3`), and `(is_int r)` tests integrality. Z3’s parser is lenient and silently inserts `to_real` when an `Int` term meets a `Real` term (`(= r (+ x 0.5))` with `x : Int` is accepted), but generated encodings should write the coercion explicitly rather than rely on that leniency or on programming-language intuitions.

Linear real arithmetic is useful for geometric constraints, rates, and rational coefficients. It is not currently needed by the Length domain, whose semantic values are naturals.

Theories beyond QF_LIA

8.1 Equality with uninterpreted functions

EUF treats user-declared functions as arbitrary total functions constrained by equality. Its key reasoning principle is congruence:

$$x = y \implies f(x) = f(y).$$

It is useful for abstracting computations whose internal implementation is irrelevant but whose wiring matters.

For a state-bind candidate, declare abstract sorts S, A, B , an input state $s_0 : S$, and functions

$$F : S \rightarrow A \times S, \quad G : A \times S \rightarrow B \times S.$$

The correct candidate calls G with the state projected from $F(s_0)$; a wrong candidate calls G with s_0 . Z3 can choose an interpretation separating the two states and results.

In Leant/Djex

This is the foundation of the planned state/provenance domain. Djex still owns the exact typed call graph and determines which application denotes the candidate. Z3 chooses a separating first-order interpretation; it does not inspect Lean closures.

8.2 Arrays

The array sort

$$(\text{Array } I \ E)$$

models a total function from index sort I to element sort E , with operations `select` and `store`. Array equality is extensional.

An array-plus-length representation of a sequence uses a pair (n, a) where n is the logical length and $a : \mathbb{Z} \rightarrow E$ stores elements. Every read must be guarded by $0 \leq i < n$. This profile supports symbolic counterexample indices and combines naturally with arithmetic.

Arrays are not mutable memory objects. `store` returns a new array value; it does not update an existing one in place.

8.3 Algebraic datatypes

Z3 algebraic datatypes model constructor-based values such as options, lists, trees, tuples, and enumerations. The theory understands constructor disjointness, injectivity, recognizers, and selectors.

Datatypes can serve two roles:

1. model object values and constructor tags;

2. represent a grammar or syntax tree whose constructors are solver choices.

For Leant/Djex, the first role is useful for first-order semantic domains. The second should initially be avoided for the main synthesis grammar because Djex already owns a checked typed DAG, including information that would be expensive and risky to duplicate in Z3. Finite selector variables over Djex-prepared alternatives are a cleaner first sketch encoding. The SMT-LIB syntax names each datatype with its number of sort parameters, then lists constructors with their named selectors. A tester is written (`_ is C`):

```
(declare-datatypes ((Shape 0))
  (((circle (radius Int))
    (rect (w Int) (h Int)))))
(declare-const s Shape)
(declare-const t Shape)
(assert (= (radius s) 3))
(assert (not ((_ is circle) t)))
(assert (= (w t) (radius s)))
(check-sat)
(get-model)
```

Listing 8.1: A two-constructor datatype with selectors and a tester.

Z3 5.0.0 answers

```
sat
(
  (define-fun s () Shape
    (circle 3))
  (define-fun t () Shape
    (rect 3 2))
)
```

One caveat matters for encodings: applying a selector to the wrong constructor is not an error but an under-specified value. Adding `(assert ((_ is rect) s))` to the script keeps it sat (Z3 chooses `(rect 2 4)` for `s`, and `(radius s)` is then some integer that happens to be 3). Encodings should guard every selector by the corresponding tester, exactly as array and sequence reads must be guarded by bounds.

8.4 Sequences and strings

Z3 sequences support length, concatenation, extraction, indexing, containment, prefixes, suffixes, and related operations over an element sort. Strings are sequences of characters with additional operations.

They can express list-like contracts concisely, but modelers must guard indices. Out-of-bounds sequence indexing is not a programming-language exception; it is under-specified in the logical semantics. A missing guard can make a false property appear satisfiable or permit a model that has no intended source-level meaning.

The planned sequence domain should therefore support an explicit native-sequence profile and an array-plus-length profile. The selected profile and every bounds convention belong to the encoding fingerprint.

8.5 Bit-vectors

A bit-vector sort `(_ BitVec w)` contains exactly 2^w fixed-width bit patterns. Arithmetic wraps modulo 2^w . Signed and unsigned comparisons are distinct. Shifts, rotates, extraction, concatenation, extension, and bitwise operators are exact.

Bit-vectors are the correct theory for:

- machine-word masks and constants;
- wraparound arithmetic;

- overflow detection;
- saturating operations;
- packing and unpacking fields;
- branchless bit tricks;
- finite-width cryptographic or protocol transformations.

A planned bit-vector synthesis domain must explicitly bind width, signedness, source coercions, shift behavior, division-by-zero policy, and the Lean target type. A Z3 model choosing a constant is decoded into a grammar-owned literal, then Lean elaborates and proves the materialized term where possible, for example with `bv_decide`.

8.6 Floating point

Z3 has a theory of IEEE floating-point formats, including rounding modes, NaNs, infinities, signed zero, and conversions. This is different from `Real`. It is appropriate only when the source semantics match the selected IEEE profile exactly.

Floating-point synthesis is a plausible future domain but is not part of the present Leant/Djex roadmap. It would require unusually careful replay and proof support.

8.7 Finite domains and enumerations

Booleans, bit-vectors, enumeration datatypes, and bounded integers can encode finite choices. These are central to typed sketches. A selector may be a Boolean for two alternatives, an integer with bounds $0 \leq s < k$, a bit-vector wide enough for k cases, or an enumeration datatype with k constructors.

Integer selectors are simple to combine with arithmetic objectives; enumeration sorts make illegal values unrepresentable; one-hot Booleans can make unsat cores and cardinality constraints convenient. The grammar compiler should choose and fingerprint one canonical representation.

8.8 Theory combinations are not free

Z3 can combine many theories, but every added theory enlarges the solver and parser contract. A formula using arrays, quantifiers, nonlinear arithmetic, and recursive functions may be concise yet unpredictable. Domains should document:

- exact sorts and operators admitted;
- whether quantifiers are used;
- whether the fragment is decidable and whether Z3's procedure is complete;
- model shapes the decoder accepts;
- replay semantics;
- timeout and fallback policy.

Quantifiers, recursion, and incompleteness

9.1 Universal properties

Many specifications are naturally universal:

$$\forall n \in \mathbb{N}. C(n) = n.$$

Counterexample search removes the outer universal by introducing a free constant:

$$n \geq 0 \wedge C(n) \neq n.$$

This is one reason verification encodings can remain quantifier-free even when the source contract is universally quantified.

More complex properties may quantify over indices, elements, functions, or recursive executions. The domain should look for a counterexample-oriented transformation before emitting a general quantifier. Examples include:

- replace “for every valid index” by one free index plus bounds and a disequality;
- expand a bounded finite universal into a conjunction;
- represent “for every element” by a symbolic atom and bag-count equation;
- represent recursive reachability with Horn clauses rather than a recursively quantified axiom.

9.2 Why quantified reasoning is hard

First-order logic with unrestricted quantifiers is undecidable. Z3 uses heuristic and fragment-specific methods such as E-matching and model-based quantifier instantiation. These can be extremely effective, but they may diverge, return unknown, or be sensitive to syntactic triggers.

A quantified axiom can also cause an instantiation loop, producing ever more terms. Small changes in patterns or preprocessing can change performance dramatically. This is unacceptable as an implicit foundation for a user-visible proof claim.

9.3 Patterns and model-based instantiation

A pattern tells Z3 which ground terms should trigger an instance of a quantified formula. Patterns must cover bound variables and are matched modulo current equalities. Model-based quantifier instantiation instead builds a candidate model and looks for quantified counterexamples.

These mechanisms are valuable for specialized domains, but the pattern set, solver options, and exact quantified axioms become part of the encoding profile. They must not be hidden inside a generic “Z3 proved it” flag.

9.4 Recursive function definitions

SMT-LIB and Z3 support recursive function definitions. Z3 reasons by incremental unfolding and other techniques; it does not thereby perform mathematical induction over arbitrary recursive definitions. This can find bounded counterexamples and solve useful instances, but it is not a general positive-proof method.

For a recursive candidate, bounded unfolding is an under-approximation of all executions: a found counterexample may be replayable, but failure to find one does not establish the property for all recursion depths. A small experiment makes the asymmetry concrete. Define the triangular-number function recursively and ask for a counterexample to a false claim:

```
(define-fun-rec sumTo ((n Int)) Int
  (ite (<= n 0) 0 (+ n (sumTo (- n 1)))))
(declare-const n Int)
(assert (>= n 0))
(assert (not (= (sumTo n) (* 3 n))))
(check-sat)
(get-value (n (sumTo n)))
```

Listing 9.1: A recursive definition and a falsifiable claim.

Z3 5.0.0 answers immediately:

```
sat
((n 1)
 ((sumTo n) 1))
```

One unfolding sufficed: $\text{sumTo}(1) = 1 \neq 3$. Now replace the claim by the true identity $2 \cdot \text{sumTo}(n) = n(n+1)$, that is, `(assert (not (= (* 2 (sumTo n)) (* n (+ n 1)))))`, and add `(set-option :timeout 5000)` at the top. Z3 unfolds and unfolds, never finds a counterexample, and eventually prints `unknown` with `(:reason-unknown "timeout")`: it has no induction principle with which to close the argument. Only after adding the bound `(assert (<= n 20))` does it answer `unsat`, in about a tenth of a second, because twenty-one unfoldings then exhaust the domain. That `unsat` is a statement about $n \leq 20$, not about all naturals; the general theorem needs induction, which is Lean's job.

9.5 When unknown is expected

Unknown or resource exhaustion becomes more likely with:

- nonlinear integer arithmetic;
- unrestricted quantifiers;
- difficult quantified array or sequence axioms;
- recursive definitions requiring induction;
- mixed theories with no complete combination procedure;
- aggressive resource caps.

The right response is often to redesign the abstraction, split the proof, add a finite bound, move recursion to CHCs, or generate a Lean obligation—not merely increase the timeout.

Mental model

Z3 is strongest when the application compiles a rich source question into a small decidable theory. Throwing the original theorem over the wall is usually the least reliable integration strategy.

Models and response parsing

10.1 A model interprets all relevant symbols

When a formula is satisfiable, a model assigns values to free constants and interpretations to free functions. A function model is often shown as a finite map plus an else case:

$$f = [0 \mapsto 3, 3 \mapsto 0, \text{ else } \mapsto 1].$$

This is a compact representation of a total function, not a claim that only three arguments exist. Z3 5.0.0 prints such a function as a define-fun whose body is a nested ite; for the assertions $f(0) = 3$, $f(3) = 0$, $f(5) = f(7)$, $f(5) \neq 0$ it prints

```
(
  (define-fun f ((x!0 Int)) Int
    (ite (= x!0 0) 3
      (ite (= x!0 3) 0
        1)))
)
```

where $x!0$ is a solver-generated bound-variable name and the final 1 is the default. Older Z3 versions printed the same information with an explicit else keyword; parsers must not assume either spelling.

The printed model is a serialization chosen by the solver. Applications should prefer term-specific values and a narrow parser whenever possible.

10.2 Partial display versus complete semantics

A model display may omit unconstrained constants or choose arbitrary default values. (Z3 5.0.0's `get-model` happens to list every declared constant, assigning a default such as 0 to one that no assertion mentions; that is a presentation choice of this version, not a guarantee.) API calls often support model completion when evaluating a term. This does not make the completed values semantically important; it merely extends a partial presentation to one total interpretation consistent with the assertions.

A counterexample decoder should request every symbol needed for replay and reject missing values rather than silently invent defaults.

10.3 Exact parsing is part of correctness

A robust response parser must handle more than balanced parentheses. It needs explicit policies for:

- total bytes and nesting depth;
- token count and token byte length;
- integer numeral bit length;

- legal symbol grammar and exact expected symbol set;
- duplicate and missing bindings;
- status-specific response shape;
- error responses and unsupported outputs;
- trailing output and transaction boundaries.

The child process is an external component. Even a benign solver can emit unexpected diagnostics, warnings, version-dependent formatting, or enormous artifacts. Treat stdout as untrusted structured input.

10.4 Association matters as much as parsing

Suppose a parser correctly decodes $((in\ 3))$. The value is still useless unless the application knows:

- which exact query requested in ;
- which candidate and contract that query represented;
- which inventory and provider laws were used;
- which solver process and transaction produced it;
- whether the status was sat;
- whether the response arrived before a reset or cancellation boundary.

Djex binds checked artifacts through nominal types and structural fingerprints. Public replay gates compare complete problem/query association before exposing a receipt.

10.5 Independent replay

For a Length model, replay performs the following checks:

1. the number of bindings equals the sealed input arity;
2. every symbol is known, appears once, and has bounded bytes;
3. every value is an integer and nonnegative;
4. bindings are reordered into source input order;
5. the checked candidate graph is evaluated under the same Length semantics;
6. the precondition holds;
7. the postcondition fails;
8. the resulting evidence is associated back to the exact behavioral problem.

If the assignment does not reproduce the bad state, the result is Nothing or a replay refusal, not a weaker counterexample.

10.6 Counterexample simplification

A raw model may be larger than necessary. After validating it, a domain can search for a deterministic smaller witness. For a natural input vector $\vec{n}$, one strategy explores the dominated box

$$0 \leq m_i \leq n_i$$

in a fixed order and returns the first strict improvement that still violates the contract. This is solver-independent and replay-owned.

Alternative strategies include lexicographic optimization, minimizing the sum or maximum of inputs, or delta debugging over discrete structure. The simplifier's algorithm and bounds become part of the receipt if its minimality claim is user-visible.

10.7 Counterexample banks

A counterexample bank stores input vectors, not verdicts. When checking a new candidate, each sample is replayed against that candidate's newly sealed problem. A sample that refuted one candidate may satisfy another. The bank is scoped to candidate-independent identities such as the exact inventory, contract, encoding policy, and input representation.

Current scalar and binary-product Length banks are nominally distinct. They charge bounded replay attempts, preserve origins as coarse labels, and never let a stored sample itself carry candidate-specific evidence.

In Leant/Djex

The newest Leant filter context owns one command-local nominal bank and reuses it across batches within that context. This is the first concrete piece of the future CEGIS feedback loop: newly replayed inputs can cheaply challenge later candidates before a live Z3 query is needed.

Assumptions, unsatisfiable cores, and optimization

11.1 Temporary assumptions

`check-sat-assuming` adds Boolean literals for one check without permanently asserting them. A common pattern guards clauses by named assumption literals:

```
(set-option :produce-unsat-cores true)
(declare-const x Int)
(declare-const a_clause Bool)
(declare-const b_clause Bool)
(declare-const c_clause Bool)
(assert (=> a_clause (> x 0)))
(assert (=> b_clause (< x 0)))
(assert (=> c_clause (< x 100)))
(check-sat-assuming (a_clause b_clause c_clause))
(get-unsat-core)
(check-sat-assuming (a_clause c_clause))
(get-value (x))
```

Listing 11.1: Guarding clauses by assumption literals.

Z3 5.0.0 answers:

```
unsat
(a_clause b_clause)
sat
((x 1))
```

The first check activates two incompatible clauses and is `unsat`; the core names only the two assumptions actually needed (`c_clause` is not blamed). The second check drops `b_clause` without touching the assertion set and succeeds. Assumptions can thus be changed freely between checks without rebuilding declarations. Note that an unset guard does not deactivate its clause by force: an unguarded `check-sat` may still set `a_clause` to true or false as it pleases, so a guard that must be off has to be assumed as `(not a_clause)`.

11.2 Unsatisfiable cores

An unsatisfiable core is a subset of tracked assertions or assumptions that is itself unsatisfiable. It is not necessarily minimal. A core can explain that a contract is inconsistent or that a particular combination of grammar choices cannot satisfy the samples. Besides assumption literals, SMT-LIB lets an assertion be tracked by name with the `!` annotation. Cores must be

enabled explicitly, otherwise `get-unsat-core` fails with `unsat core construction is not enabled`, use command `(set-option :produce-unsat-cores true)`:

```
(set-option :produce-unsat-cores true)
(declare-const x Int)
(assert (! (> x 0) :named pos))
(assert (! (< x 0) :named neg))
(assert (! (< x 100) :named small))
(check-sat)
(get-unsat-core)
```

Listing 11.2: Named assertions and an unsatisfiable core.

Z3 5.0.0 answers

```
unsat
(pos neg)
```

The core is the list of names; `small` is not blamed because it played no part in the conflict.

Potential Leant/Djex uses include:

- identify conflicting contract clauses and provider assumptions;
- learn a nogood over typed-sketch selector choices;
- minimize the hypotheses of a generated Lean proof attempt;
- explain why no candidate exists within a bounded grammar.

A core is meaningful only under the exact assertion-to-source mapping and exact grammar/sample fingerprints. If the grammar or semantics changes, old nogoods cannot be reused blindly.

11.3 Core minimization

Because Z3 need not return a minimal core, the host may perform deletion-based minimization: remove one assumption, recheck, and keep it removed if unsatisfiability remains. This can require many solver calls, so it needs its own budget and deterministic order.

Even a minimal core is not necessarily the clearest explanation. The domain should map internal literals back to contract source spans and semantic labels.

11.4 Optimization

Z3 supports objectives such as:

```
(minimize cost)
(maximize score)
(assert-soft preferred :weight 5)
```

Multiple objectives can be lexicographic, Pareto, or independent depending on the interface and options. Optimization chooses among satisfying models; it does not weaken hard constraints. Objectives are ordinary commands in Z3's SMT-LIB front end; `(get-objectives)` reports the optimum reached. Revisiting the first script of Chapter 5 with a preference for a small `y`:

```
(declare-const x Int)
(declare-const y Int)
(assert (= (+ x y) 10))
(assert (>= x 0))
(assert (>= y 0))
```

```
(assert (< x y))
(minimize y)
(check-sat)
(get-value (x y))
(get-objectives)
```

Listing 11.3: An optimization objective added to the hello script.

Z3 5.0.0 answers

```
sat
((x 4)
 (y 6))
(objectives
 (y 6)
 )
```

Here $y = 6$ is genuinely optimal ($x < y$ and $x + y = 10$ force $y \geq 6$), and the reported value now carries that meaning, whereas the identical-looking model in Chapter 5 was an arbitrary choice.

For synthesis, useful objectives include:

- minimize AST nodes or weighted grammar cost;
- minimize provider calls, branches, or duplicated arguments;
- minimize symbolic output size or modeled resource cost;
- maximize weighted soft examples;
- lexicographically minimize branch count, then size, then provider penalty.

11.5 Native Optimize versus iterative bounds

A native optimization context is concise and supports soft constraints. An ordinary solver with iterative bounds can be easier to combine with assumptions and cores:

1. assert that a solution has cost at most k ;
2. check satisfiability;
3. if satisfiable, decrease k or binary-search the bound;
4. materialize the best model found.

The existing behavioral proposal recommends iterative SAT first for syntactic size and native optimization later for genuinely multiobjective problems. Both approaches require deterministic objective ordering and fingerprinted solver parameters.

Common mistake

An optimal model is optimal only for the encoded objective. “Smallest SMT grammar cost” need not mean shortest Lean source, fastest runtime, simplest proof, or best human readability unless those notions were modeled.

Constrained Horn clauses and fixedpoints

12.1 Why recursion needs a different view

A recursive program is naturally described as a relation between inputs and outputs rather than as a finite arithmetic expression. Let $\text{Eval}_f(x, y)$ mean that candidate f can evaluate input x to output y . Base and recursive cases become rules:

$$\begin{aligned}\text{Eval}_f(x, y) &\leftarrow \text{Base}(x, y), \\ \text{Eval}_f(x, y) &\leftarrow \text{Step}(x, x', z) \wedge \text{Eval}_f(x', y') \wedge \text{Combine}(z, y', y).\end{aligned}$$

A bad state is another rule:

$$\text{Bad} \leftarrow \text{Eval}_f(x, y) \wedge P(x) \wedge \neg Q(x, y).$$

A constrained Horn clause has at most one positive occurrence of an uninterpreted relation in its head, with theory constraints such as arithmetic in the body. Z3's fixedpoint extension, often called muZ, includes Spacer, a property-directed reachability engine for Horn clauses over arithmetic and other theories.

12.2 A tiny Horn example

A reachability relation over nonnegative integers can be written schematically as:

```
(set-logic HORN)
(declare-fun Reach (Int) Bool)

(assert (Reach 0))
(assert (forall ((x Int))
  (=> (and (Reach x) (< x 10))
    (Reach (+ x 1)))))

; Bad: a reachable value is negative.
(assert (forall ((x Int))
  (=> (and (Reach x) (< x 0)) false)))

(check-sat)
```

In the pure SMT-LIB Horn style, a clause with no positive relation in the head (a clause whose conclusion is false) represents a safety query. The fixedpoint API also supports explicit rule and query commands.

The meaning of the answer is inverted relative to counterexample search, and this trips up almost every newcomer. In HORN logic, *sat* means “there is an interpretation of the relations that makes every clause true,” that is, an inductive invariant that keeps *Reach* away from the bad states: the system is *safe*. *unsat* means the clauses can only be satisfied by making the bad clause’s premise reachable, so the bad state *is* reachable. For the script above, Z3 5.0.0 prints

```
sat
(
  (define-fun Reach ((x!0 Int)) Bool
    (not (<= x!0 (- 1))))
)
```

The “model” is the invariant $\text{Reach}(x) \equiv x \geq 0$: it contains 0, is preserved by the step rule, and excludes every negative value. If the bad clause is changed to blame values above 5,

```
; Bad: a reachable value exceeds 5.
(assert (forall ((x Int))
  (=> (and (Reach x) (> x 5)) false)))
```

the same script prints `unsat`, because $0, 1, \dots, 10$ really are reachable and 6 violates the property. A driver for the fixedpoint engine must therefore not reuse the “sat means counterexample” branch of the ordinary SMT protocol.

12.3 Counterexample trace or invariant

A CHC solver may show that the bad relation is reachable, yielding a derivation that can be decoded into a trace. Or it may establish safety by discovering an inductive invariant. The two outputs have different application paths:

Trace

Decode bounded states and steps, replay them against the exact relational semantics, and produce a candidate-specific counterexample.

Invariant

Translate the invariant into a Lean proposition, prove that it is inductive and implies the contract, and let Lean check the proof.

A raw “safe” fixedpoint status should remain a solver observation until the invariant is checked or an explicit external-solver trust policy is selected.

12.4 Why CHCs fit the roadmap

CHCs can express recursive relational semantics without requiring Z3 to invent or inspect Lean syntax. Djex can generate rules from a checked recursive candidate schema, provider relations, and a domain abstraction. Z3 searches the space of inductive facts. Lean remains responsible for termination and the final theorem.

This is a later-stage feature because it requires:

- exact extraction of recursive call structure;
- a relational value abstraction;
- a stable invariant grammar translatable to Lean;
- trace and invariant decoders;
- a new fixedpoint process protocol and capability profile;
- proof-generation support.

Checkpoint

The ordinary SMT solver searches for values satisfying a finite formula. The fixedpoint engine searches for relations or invariants satisfying recursive rules. Both are Z3, but they have different state machines, artifacts, and trust stories.

Part III

Engineering a Solver Boundary

Compiling program behavior into formulas

13.1 A domain, not a universal translator

There is no faithful automatic translation from arbitrary Lean programs and propositions into one convenient Z3 fragment. A practical integration defines a **behavioral domain**. A domain specifies:

- which source types and term-graph constructs it can observe;
- the mathematical values used for those observations;
- how constructors, cases, functions, providers, and callbacks transform observations;
- the contract language;
- the solver theory and lowering profile;
- the concrete replay evaluator;
- what exactness and assumptions accompany a result.

The implemented Length domain observes a unary recursive spine only through its finite length. Elements are opaque. It supports a restricted candidate-graph language and explicit provider transfer laws. This narrowness is a feature: every supported operation has a clear symbolic and concrete interpretation.

13.2 The semantic pipeline

A well-factored behavioral check has the following stages:

1. **Source authority**: obtain a candidate whose exact typed origin and inventory are known.
2. **Domain admission**: verify that the target type, roles, datatype spine, providers, and candidate constructs belong to the domain.
3. **Symbolic interpretation**: compute a domain expression for the candidate result in terms of modeled inputs.
4. **Contract substitution**: form the checked bad-state formula $P \wedge \neg Q$.
5. **Typed lowering**: convert the domain formula to a solver-specific typed plan.
6. **Canonical rendering**: serialize the plan to bounded SMT-LIB bytes.
7. **Execution**: run the exact transaction under an observed solver envelope.
8. **Decoding**: parse only expected status and values under hard limits.
9. **Replay**: evaluate the checked problem independently and reproduce the violation or bounded positive claim.
10. **Policy**: rank, retain, reject, request a proof, or report inconclusive according to explicit mode.

Each boundary should have an opaque successful type and a closed error vocabulary. This prevents later code from skipping checks by constructing a record directly.

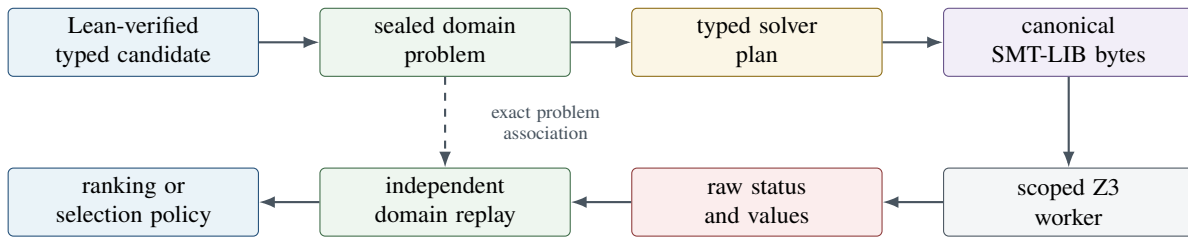


Figure 13.1: The solver is one stage in a larger checked pipeline.

13.3 Symbolic interpretation versus execution

A symbolic interpreter does not run the candidate on concrete Lean values. It traverses the exact typed term graph and computes expressions in the domain algebra. For `Length`:

- an observed list input becomes a natural variable;
- `nil` has length 0;
- `cons` adds 1 to the tail length;
- a supported exact spine case becomes an `ite`;
- a provider occurrence uses its explicit transfer law after occurrence-specific authority checks;
- an opaque argument may be forwarded but cannot be inspected;
- unsupported demand is a refusal.

The same checked problem also supports concrete replay. Given natural input lengths, the evaluator traverses the candidate graph and computes concrete result lengths. Symbolic and concrete semantics should be two views of one domain definition, tested against each other.

13.4 Provider laws are assumptions with provenance

A type signature rarely reveals behavior. The provider `List.reverse` has type `List a -> List a`, but length preservation is not derivable from that type alone. A domain may admit a law

$$\text{len}(\text{reverse}(xs)) = \text{len}(xs).$$

The law must be explicit and associated with the exact provider declaration and occurrence.

The current `Length` source distinguishes exactly two trust classes for a provider law (`LengthProviderTrust` in Djex’s private `Internal.Semantic.Length`): a context-free assumed law (`AssumedProviderLaw`), and a law on a class-constrained provider that becomes usable only after the occurrence’s ground constraints have been discharged independently (`AssumedProviderLawConditionalOnConstraintDischarge`). Leant’s handoff picks the class purely from the exact inventory scheme: an empty leading context yields the first, a nonempty one the second. A Lean-proved law is proposed, not implemented. The source never infers a transfer from a provider’s name, and Z3 is never type-class or dictionary authority.

A result conditional on provider laws should say so. If a provider law is false, both symbolic Z3 reasoning and concrete domain replay can agree with each other while disagreeing with the actual Lean function. A future proof path should link provider laws to Lean theorems whenever possible.

Trust boundary

Independent replay protects against solver and parser mistakes. It does not protect against a wrong domain semantics or false provider assumption shared by both translation and replay. Positive proof artifacts must ultimately connect those assumptions to Lean.

13.5 Exactness, over-approximation, and under-approximation

A domain interpretation may be exact, over-approximate, or under-approximate.

Exact

Every modeled behavior corresponds to the source abstraction, and every source behavior in scope is represented.

Over-approximation

The model admits all real behaviors and possibly extra behaviors. If no bad modeled behavior exists, no real bad behavior exists; spurious counterexamples are possible and must be replayed.

Under-approximation

The model represents only some real behaviors. Found counterexamples can be real after replay; absence of a counterexample cannot justify pruning or proof.

Sound prefix pruning requires an over-approximation of all completions. Bounded testing and bounded unfolding are under-approximations and may only produce witnesses. These directions are easy to reverse accidentally.

13.6 Contract roles

A contract should not infer semantics from a type constructor's spelling. It supplies a source-ordered role vector, arity-checked against the exact target. (The implemented Length domain admits exactly two target roles, observed-spine and unobserved-target, and two provider-argument roles, spine and unobserved; richer vocabularies with callbacks or result components are design space, not current code.)

For example, a function may accept two lists but observe only the first, or return a pair of observed lists. The product Length domain has a distinct nominal contract and problem type for two result spines; it is not a cast of scalar evidence.

13.7 Closed interpretation policies

The set of supported cases is itself policy. A candidate that pattern-matches on a list may require exact authority that the matched datatype is the configured spine and that constructor fields are understood. A provider with a nonempty class context may require a restricted resolver and independent ground discharge. A callback may be opaque in one domain and an uninterpreted function in another.

Policy should be represented by closed constructors, not arbitrary callbacks that can smuggle untracked semantics into the interpreter. Changing policy changes the encoding identity.

13.8 Failure is part of the semantics

A refusal to model a candidate is not a counterexample and not evidence that the candidate is wrong. Typical refusals include:

- no exact typed origin;
- unsupported target shape or role vector;
- unresolved or ambiguous spine/provider identity;
- unsupported term-graph node or demand on an opaque value;
- provider certificate mismatch;
- class-resolution authority unavailable;
- graph, syntax, numeral, or evaluation-step limit exceeded;
- fingerprint or rendering limit exceeded.

Ranking can retain the candidate in a neutral tier. Filtering must preserve it unless the explicit policy says inapplicability is fatal. The current Length filter retains preparation refusals.

Typed plans, canonical bytes, and fingerprints

14.1 Text must not become the semantic authority

It is tempting to construct an SMT-LIB string and later treat that string as the meaning of the query. This creates several problems:

- equivalent formulas may have many textual spellings;
- escaping and whitespace bugs can alter meaning;
- string concatenation can bypass sort checks;
- a renderer bug can make fingerprints identify the wrong plan;
- later code may parse its own output to rediscover semantics.

Djex constructs a typed QF-LIA plan first. Rendering is a bounded projection. The complete fingerprint binds both the structural plan and the resulting command bytes, but replay retains the checked problem rather than reparsing SMT-LIB.

14.2 A small typed command language

The internal AST lives in the private module `Language.Haskell.Synthesis.Internal.SMTLib.QFLIA`. Its three types have exactly these constructors (strictness annotations omitted):

```
data QFLIAIntegerExpression
  = QFLIAIntegerSymbol [Word8]
  | QFLIANaturalNumeral Natural
  | QFLIAIntegerSum [QFLIAIntegerExpression]
  | QFLIAIntegerScale Natural QFLIAIntegerExpression
  | QFLIAIntegerDifference QFLIAIntegerExpression QFLIAIntegerExpression
  | QFLIAIntegerBinaryApplication [Word8]
    QFLIAIntegerExpression QFLIAIntegerExpression
  | QFLIAIntegerIf QFLIABooleanExpression
    QFLIAIntegerExpression QFLIAIntegerExpression

data QFLIABooleanExpression
  = QFLIABooleanTruth Bool
  | QFLIAIntegerEqual QFLIAIntegerExpression QFLIAIntegerExpression
  | QFLIAIntegerAtMost QFLIAIntegerExpression QFLIAIntegerExpression
  | QFLIABooleanNot QFLIABooleanExpression
  | QFLIABooleanAll [QFLIABooleanExpression]

data QFLIACommand
  = QFLIASetLogic [Word8]
  | QFLIASetOption [Word8] [Word8]
  | QFLIADefineBinaryInteger [Word8] [Word8] [Word8] QFLIAIntegerExpression
```

```
| QFLIADeclareInteger [Word8]
| QFLIAAssert QFLIABooleanExpression
| QFLIACheckSatisfiable
| QFLIAGetValues [QFLIAIntegerExpression]
```

Listing 14.1: The typed QF-LIA plan, verbatim constructor names from the private Djex module `Internal.SMTLib.QFLIA`.

Notice what is absent: there is no disjunction, no strict comparison, no negative literal, and no general multiplication. Strict comparison is written as the negation of an at-most; “or” never arises because the Length formula language itself has only truth, equality, at-most, negation, and finite conjunction; and the only subtraction is the one inside the monus helper’s body. A constructor cannot accidentally place an integer expression where a Boolean formula is required, and the renderer (`renderQFLIACommand`) has one canonical spelling for each node: an empty sum renders as $\emptyset$, a one-term sum or one-clause conjunction renders as its only member, an empty conjunction renders as `true`, and every command ends with exactly one line feed.

14.3 Canonicalization choices

Canonical rendering should specify:

- declaration order;
- input-symbol naming from source positions;
- private-witness naming from deterministic traversal order;
- helper-definition order;
- operand order for associative forms;
- decimal numeral spelling;
- whitespace and line endings;
- whether redundant assertions are retained;
- exact status and value-request sequence.

Canonical does not mean mathematically simplest. It means one deterministic representation for one typed plan under one schema.

14.4 The current Length preamble

The scalar and binary-product translators emit an input-only QF_LIA program. The fixed preamble:

1. enables model production;
2. fixes the random seed to 1;
3. sets logic QF_LIA;
4. defines natural monus as `djex_nat_monus`;
5. defines binary minimum as `djex_nat_min`;
6. defines binary maximum as `djex_nat_max`.

The query then declares the source input symbols `djex_length_input_0`, `djex_length_input_1`, ... in source-position order, declares any private quotient/remainder witness pairs, asserts nonnegativity of every input, asserts the witness constraints, asserts the combined bad-state condition, and calls `check-sat` (see `sealLengthSMTLibQuery` in the private `Internal.Semantic.Length.SMTLib`). A separate optional command, `(get-value (djex_length_input_0 ...))`, requests values for all and only the source input symbols; a nullary problem has no such command.

The scalar translator schema is currently tagged `djex-length-z3-qf-lia-smtlib2/v2`; the binary-product sibling has a distinct schema. These tags identify translation formats, not the versionless Leant JSON configuration grammar.

14.5 Why separate check bytes and value-request bytes

The check program is valid regardless of status. The value request is sent only after sat. Keeping the artifacts separate prevents a model request from being blindly issued after unsat or unknown and makes the protocol state explicit.

For a nullary problem there are no input values to request. A satisfiable nullary bad state is replayed without model bindings.

14.6 Structural fingerprints

A fingerprint is a complete, versioned canonical encoding of tagged structural fields—a structural key, not a digest, so equality never depends on accepting a hash collision (SHA-256 appears only inside a fingerprint as the value of an executable-image field). Different subject types prevent accidental comparison of, for example, an inventory fingerprint with a candidate fingerprint.

The Length problem and query bind identities for:

- semantic domain;
- exact inventory and spine model;
- provider-law inventory;
- interpretation and encoding policy;
- contract;
- typed candidate graph;
- complete behavioral problem;
- typed query plan and schema;
- rendered check and value-request bytes.

Live execution adds separate identity for the observed executable, protocol, options, and resource limits. This avoids pretending that the same mathematical query run by two different solver envelopes is the same execution artifact.

14.7 Fingerprints are association, not proof

A fingerprint can establish that an artifact belongs to the exact problem that produced it. It cannot establish that the problem encoding is sound, that a provider law is true, or that an unsat response is trustworthy. This is why Djex’s generic module says “association is not certification.”

14.8 Versioning strategy

Every semantic change that can alter meaning or accepted artifacts must change a schema identity. Examples include:

- different input-symbol order;
- a changed monus definition;
- added or removed nonnegativity assumptions;
- new provider-law interpretation;
- a different sequence representation;
- changed fixedpoint rule generation;
- different objective priority;
- changed incremental scope protocol.

Leant’s current configuration files are deliberately versionless and experimental: superseded shapes live in Git history. That source-file policy is compatible with versioned internal semantic fingerprints. One governs user-facing decoder compatibility; the other prevents evidence reassociation.

14.9 Bounds before fingerprints

Fingerprint builders themselves consume resources. Djex bounds command bytes, fingerprint bytes, symbol bytes, syntax nodes, numeral bits, collection widths, and retained artifact payloads. A value too large to fingerprint safely is rejected before it enters an opaque checked object.

The default Length query limits in the inspected Djex tree admit 65,536 bytes per command artifact, 262,144 fingerprint-builder bytes, and 4,096 bits per rendered natural numeral. These defaults are policy, not mathematical constants.

Source trail

See [synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib.hs](#) and [synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/QFLIA.hs](#).

Owning the Z3 process

15.1 The process lifecycle is a state machine

A live solver integration is not merely write; read. It owns a sequence:

1. admit and validate pure execution policy;
2. resolve and inspect the executable;
3. create a fresh isolated workspace;
4. spawn with exact arguments and environment;
5. perform readiness/capability probes;
6. lend a scoped ready worker;
7. execute bounded serialized transactions;
8. close stdin or request exit;
9. wait, terminate, and escalate if necessary;
10. release descriptors and workspace;
11. report whether cleanup was incomplete.

Every transition can fail. Failures during cleanup must not overwrite a more important callback exception, but incomplete cleanup should remain observable.

15.2 No shell and no ambient environment

The current execution profile uses a fixed direct argument prefix, policy-derived resource arguments, and an exact empty child environment. It does not invoke a shell or inherit ambient variables. A fresh empty working directory is selected instead of inheriting the caller's directory.

These choices reduce accidental dependence on locale, configuration files, path search, user startup scripts, and environment secrets. They also make the execution identity easier to describe.

15.3 Executable admission and digest pinning

A configuration can contain an expected SHA-256 digest. Activation policy separately decides whether an unpinned executable is allowed. This separation is important: successfully decoding a path and digest field does not grant permission to execute it.

The live opener must defend resolution, hashing, and spawn against path replacement races. Djex exposes four launch strategies (`LengthSMTLibExecutableLaunchStrategy`): a portable path-snapshot-then-spawn strategy, and three Linux descriptor-bound strategies that open the executable once, copy its bytes into a sealed anonymous memory image, compare the

digest, and spawn from that descriptor, optionally after effective-ID or `AT_EXECVE_CHECK` access checks on the source and the staged image. The three descriptor-bound launches share one pipeline (`openDescriptorBoundProcessWith`, parameterized by a `DescriptorBoundLaunchPolicy` record, in the private `Internal.SMTLib.Z3.Process`) and one narrow C helper, `synthesis/cbits/z3_descriptor_spawn.c`, compiled only on Linux; elsewhere they fail closed before any spawn.

The public API reveals only closed classifications such as whether a digest expectation exists and which launch strategy was selected. It does not leak path or digest bytes through ordinary errors.

15.4 Capability probing

A process that launches successfully is not yet a ready Length solver. The readiness probe (private `Session/Capability.hs`) is a fixed four-write handshake; each write ends with a fresh positional echo barrier whose exact quoted string must come back before the next phase may begin:

1. the startup option (`set-option :print-success false`): nothing but the barrier echo may come back;
2. a reset, the canonical Length preamble, one declared integer asserted equal to 0, and (`check-sat`): the solver must answer `sat`;
3. (`get-value ...`) for that constant: the solver must answer exactly `((djex_capability_input 0))`;
4. a reset, the preamble, the constant asserted equal to both 0 and 1, and (`check-sat`): the solver must answer `unsat`.

Together these demonstrate acceptance of the startup options, `QF_LIA` status framing in both directions, value retrieval in the exact shape the decoder expects, reset behavior, whitespace and framing rules across write boundaries, and transport liveness. The probe establishes only this demonstrated profile. It does not certify arbitrary Z3 capabilities, resource-limit behavior, or semantic soundness.

15.5 Causal framing

A pipe is a byte stream, not a request/response message channel. Output may be chunked arbitrarily. Diagnostics can be delayed. A read can return half an S-expression or several responses at once.

Djex's internal causal driver uses barriers/markers and a bounded stream parser to associate complete responses with transaction order. Public callers cannot inspect or retain the process handle, transcript, barrier, ordinal, or decoded valuation. This prevents a later API layer from accidentally bypassing the transaction owner.

15.6 One query transaction

A scalar transaction conceptually proceeds as follows:

1. reset the solver to the probed clean state;
2. submit the sealed check bytes and a causal marker;
3. parse exactly one status response;
4. if `sat` and inputs exist, submit the sealed value-request bytes and another marker;
5. parse exactly the expected input bindings;
6. validate and replay the assignment;
7. associate the observation with exact query and execution identities;
8. make the session ready for the next reset or close it on protocol failure.

A malformed model, stale output, unexpected success token, extra payload, or replay failure is a transaction failure. The driver does not reinterpret it as unknown or as a harmless non-counterexample.

15.7 Budgets and deadlines

Solver-side timeout/resource options and host-side deadlines solve different problems. A cooperative solver timeout may not fire if the process is stuck outside the relevant check. A host deadline can interrupt transport and initiate cleanup, but cannot prove the child honored internal resource limits.

The execution policy binds finite values for solver timeout, solver resource limit, host deadline, response limits, and optional shared usable-work budget. The host deadline must exceed the solver timeout by a minimum margin (currently 100 ms, `z3SMTLibMinimumHostDeadlineMarginMilliseconds`) so there is time to receive and parse a cooperative response. The solver timeout and resource limit are passed on the command line as `timeout=` and `rlimit=` arguments, and both must be finite and nonzero.

A shared usable-work deadline can cover session opening and several queries. The newer scoped token checks owner thread and dynamic extent at runtime; rank- N polymorphism alone cannot prevent a closure or forked thread from retaining a phantom token.

15.8 Query leases and maximum transactions

A live scope admits a bounded number of scalar-plus-product transactions in total; the current public default is 64. The bound limits state growth and prevents an accidentally retained worker from becoming a general unbounded solver service.

A lease is spent even when a transaction fails after admission. Rolling back real solver work would make budgets dishonest and can enable retry amplification.

15.9 Cancellation and cleanup

Cancellation has at least three layers:

- stop admitting new transactions;
- interrupt or terminate the child process;
- reclaim pipes, descriptors, threads, and workspace.

Asynchronous exceptions complicate ownership. The scoped API begins durable cleanup before propagating callback exceptions across the boundary. Public errors expose a sanitized primary class and whether cleanup was incomplete, not child-controlled output or sensitive paths.

Common mistake

Killing a process is not the same as completing cancellation. If reader threads, descendants, descriptors, or temporary directories remain, later runs can observe interference or leak resources.

15.10 Concurrency

One ready worker serializes its protocol. Parallel candidate checks should use independent workers or a higher-level scheduler, not interleave commands on one stdin stream. Z3 contexts in language bindings also have thread-affinity/thread-safety constraints; a subprocess does not eliminate the need for ownership discipline.

The planned sketch solver may benefit from long-lived incremental sessions, but parallelism should occur across independently scoped sessions or solver clones with explicit identities.

Source trail

See [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Execution.hs](#), [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs](#), and the private [synthesis/internal/Language/Haskell/Synthesis/Inte](#)

rnal/SMTLib/Z3/Process.hs and `synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session/` modules.

From observations to evidence

16.1 A useful evidence ladder

Results can be arranged by what they authorize, not by how impressive they sound:

Artifact	What it supports	What it does not support
Raw sat + values	Candidate input for decoding	Rejection before association and replay
Associated solver observation	Exact provenance of the run	Behavioral correctness or violation by itself
Replayed counterexample	Rejection of that exact candidate under the domain model	Positive correctness of another candidate
Finite-box positive receipt	Property on an explicitly enumerated finite box	Universal correctness outside the box
Complete applicable-domain receipt	Property over an exactly traversed finite applicable domain	Claims beyond the domain/model/provider assumptions
Raw unsat	Useful heuristic that the encoded bad state has no model	Automatic proof authority in current Djex
Lean-checked theorem	Positive claim expressed and checked in Lean	Any stronger claim than the theorem states

16.2 Why counterexamples are easy to trust

A universal property is refuted by one witness. The host can evaluate that witness directly. Even if Z3 reached it through complex heuristics, the final check is small:

$$P(x) = \text{true}, \quad Q(x, C(x)) = \text{false}.$$

This asymmetry lets an untrusted or merely heuristic solver be operationally useful. The solver is a witness generator; the domain evaluator is the authority.

16.3 Why positive results are harder

A universal positive claim requires excluding every counterexample. There are three main ways to earn that authority:

1. **Finite exhaustion:** the domain is finite and every applicable assignment is replayed.
2. **Trusted exact decision procedure:** the application explicitly places the exact solver/encoding in its trusted base.
3. **Proof production:** generate a theorem or certificate checked by a trusted proof checker such as Lean.

The current architecture uses the first where bounded or complete finite domains are available and plans the third for proof-grade results. It deliberately does not silently choose the second.

16.4 Behavioral statuses

Djex’s solver-neutral vocabulary distinguishes:

Behavior established

A domain-owned verifier has established the property under its declared scope.

Behavior violated

A domain-owned counterexample has been reproduced.

Behavior validated within

A bounded region has been exhaustively checked.

Behavior unknown

No supported conclusion was reached.

These are report categories. Public construction of a raw observation is harmless because it carries no candidate and no authority. Opaque evidence requires private domain constructors and exact replay.

16.5 Fail closed in semantics, preserve in policy

Two superficially opposite rules coexist:

- The semantic adapter **fails closed**: it refuses to invent meaning for unsupported input.
- Conservative ranking/filtering **preserves candidates**: a refusal or operational failure does not masquerade as a counterexample.

Failing closed means “do not claim to understand.” Preserving means “do not punish the candidate for our lack of understanding.” An explicitly strict user policy may decide otherwise, but the distinction must remain visible.

16.6 Selection needs stronger association than ranking

A ranking is a permutation. A sealing layer can verify that every input occurrence appears exactly once and in the chosen order. Hard filtering is a partition: each verified occurrence must appear exactly once in either retained or rejected output, and every rejection must carry domain-authorized evidence.

The generic `BehavioralSelection` boundary in Leant checks occurrence membership, totality, and original-order association. It does not establish behavior itself. Length-specific private builders decide that only an independently replayed counterexample authorizes rejection.

16.7 Batch failures preserve the original batch

If a selection operation encounters an operational or sealing failure, the current `Length` path preserves the complete original callback batch. This prevents partial solver progress from causing nondeterministic omission.

Candidate-local outcomes remain visible: a preparation refusal, unknown solver status, positive bounded receipt, or unassessed candidate is retained with an explicit retention class.

16.8 Conditional claims

A receipt may be conditional on:

- the configured finite-spine model;
- provider laws and their ground discharges;
- an exact interpretation policy;
- finite input bounds;
- an over-approximation profile;
- a specific solver envelope if external trust is selected.

The user-facing presentation should carry these conditions instead of printing a context-free word such as “verified.”

Checkpoint

Ask of every result: What exact action may this artifact authorize? Ranking? Rejection? A bounded badge? A theorem?
If the answer is unclear, the type or presentation is too weak.

Incrementality, determinism, and CEGIS-ready state

17.1 Two kinds of reuse

A synthesis system can reuse work at two distinct levels:

Semantic reuse

Store concrete counterexample inputs and replay them against new candidates without a solver.

Solver-state reuse

Keep declarations, sample constraints, learned clauses, and scopes inside a live incremental Z3 session.

Semantic reuse is simpler and already present in the Length counterexample bank. Solver-state reuse is more powerful for typed sketches but needs a new protocol.

17.2 Counterexample-guided inductive synthesis

A canonical CEGIS loop is:

1. choose a candidate satisfying all known samples;
2. ask whether the candidate has any counterexample;
3. if a replayed counterexample is found, add it to the sample bank;
4. repeat;
5. stop with a positive proof/claim, an exhausted finite grammar, or an explicit budget result.

There are two solvers conceptually:

- the **synthesis solver** chooses candidate selectors consistent with samples;
- the **verification solver** searches for a fresh input violating the chosen candidate.

They may both be Z3 sessions, but they have different variables, assertions, artifacts, and trust directions. The verification model proposes a counterexample; the synthesis model proposes a program completion.

17.3 Monotone sample growth

If the grammar and semantics are fixed, adding counterexamples only strengthens synthesis constraints. Learned nogoods derived from unsatisfiable selector assumptions remain valid while the sample bank grows, but not after changing:

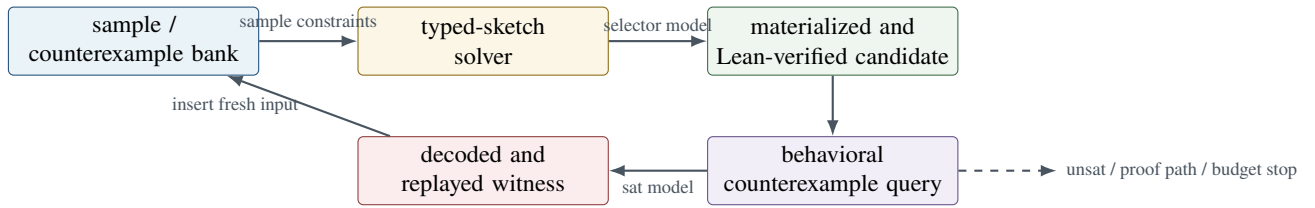


Figure 17.1: The planned CEGIS feedback loop.

- provider inventory;
- grammar productions or costs;
- semantic domain or encoding profile;
- type target or role vector;
- candidate-size bound;
- sample representation.

Nogoods and sample banks must therefore be scoped by exact fingerprints.

17.4 Semantic signatures

Before full solver-selected sketches, a smaller step is available in principle (it is not in the current tree): evaluate enumerated candidates on the current sample bank and compute a semantic signature:

$$\sigma(C) = (C(x_1), \dots, C(x_k))$$

under the chosen observation domain. Candidates with the same signature are indistinguishable on known samples.

The search can keep one representative per signature, postpone duplicate classes, or prioritize classes not yet explored. A new counterexample refines the partition. This is a low-risk bridge from post-verification filtering to CEGIS-aware enumeration.

17.5 Determinism

Reproducibility requires more than a fixed random seed. It includes:

- stable provider and production order;
- canonical symbol allocation;
- stable sample order and bank replacement policy;
- deterministic objective priority;
- fixed timeout/resource configuration;
- pinned or observed solver build;
- stable response normalization;
- deterministic counterexample simplification;
- stable tie-breaking after ranking.

A solver can still choose a different model after an implementation upgrade. Independent replay protects correctness; reproducibility tests detect presentation and search-order drift.

17.6 Incremental state fingerprints

A future incremental session should bind:

- base declarations and grammar fingerprint;
- every asserted sample in order;
- push/pop or assumption discipline;
- active selector assumptions;
- core-production settings;
- objective order and optimization mode;
- solver parameters and observed capabilities;
- transaction ordinal and reset generation.

It should expose a typed state transition API rather than generic “send SMT-LIB” access.

17.7 Recovery

Once an incremental protocol becomes desynchronized, continuing may produce plausible but stale answers. Conservative recovery is to discard the worker and rebuild from the canonical base plus retained sample bank. Rebuilding is slower but restores a known state.

The sample bank belongs outside the process and survives worker replacement. Learned clauses retained only inside Z3 are an optimization, not semantic authority.

Part IV

Z3 in the Current Leant/Djex System

The four-layer architecture

18.1 One pipeline, four different kinds of authority

The current system is easiest to understand as four cooperating components with deliberately noninterchangeable responsibilities:

Djex search.

Construct type-correct candidate graphs inside a checked synthesis inventory. Djinn searches a logical fragment; Exference searches a richer provider-oriented structural fragment. Djex owns the private names, schemes, term graphs, occurrence certificates, and completeness information needed to explain how a candidate was built.

Lean verification.

Re-elaborate a rendered candidate against the exact requested Lean type in the active project environment. This decides whether the spelling has the intended Lean meaning, resolves implicit arguments and type classes, checks universes and elaboration constraints, and ultimately relies on Lean’s kernel for type correctness.

Djex semantic interpretation.

Starting from the exact typed origin of a Lean-accepted candidate, seal a domain-specific behavioral problem. For the implemented Length domain, this means interpreting the candidate as a symbolic transformation of finite spine lengths under an explicit contract and explicit provider laws.

Z3 search.

Determine whether the sealed counterexample condition has a satisfying assignment in the selected SMT theory. In the current domain, the theory is quantifier-free linear integer arithmetic. A returned assignment is decoded and independently replayed by Djex before it can become counterexample evidence.

These responsibilities form a chain, but no component silently inherits the authority of the next one.

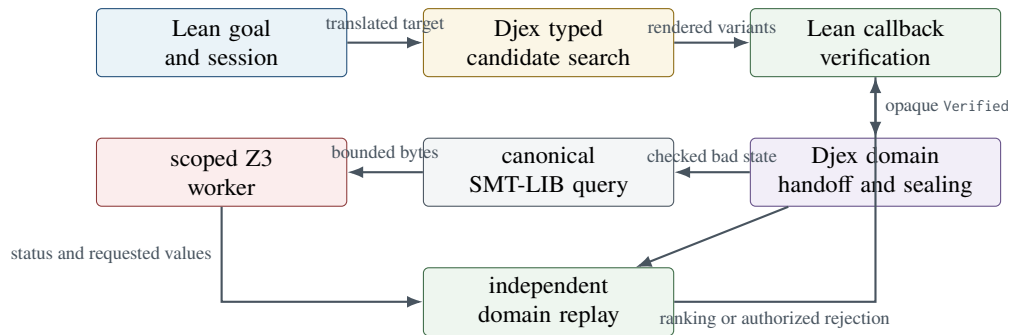


Figure 18.1: Current Leant/Djex behavioral-assessment pipeline. The return arrow does not alter Lean’s verification receipt; it associates a separate behavioral assessment with that exact occurrence.

Mental model

Lean answers “does this term have the requested type?” Djex answers “what checked candidate did that spelling come from, and what does it mean in this observation domain?” Z3 answers “is there a mathematical assignment satisfying the encoded bad state?” Independent replay answers “does that assignment really violate this exact sealed problem?”

18.2 Why type correctness is not behavioral correctness

A type can admit many inhabitants with very different behavior. For example,

$$\text{List } \alpha \rightarrow \text{List } \alpha$$

contains, among many others:

```
fun xs => xs
fun xs => xs.reverse
fun _ => []
fun xs => xs ++ xs
```

Listing 18.1: Four well-typed inhabitants with different behavior.

Lean correctly accepts all four at the same type. A type-directed synthesizer may therefore return all four unless additional intent is supplied. A length-preservation contract distinguishes the last two from the first two, but it still does not distinguish identity from reverse. A pointwise sequence contract could distinguish those. This is the central motivation for behavioral synthesis: a type determines the admissible interface; a contract describes some intended semantics within that interface.

The current implementation deliberately applies behavioral reasoning *after* Lean callback verification. Thus Z3 never repairs a type error and never makes an unelaborated candidate acceptable. It only assesses candidates that already crossed the exact Lean boundary.

18.3 Candidate groups, variants, and callback verification

A single semantic candidate graph can have several possible Lean spellings. Implicit arguments, qualification, coercions, constructor syntax, generated lets, and renderer alternatives may cause one spelling to elaborate while another does not. Leant therefore works with candidate groups and verification variants rather than assuming that one pretty-printer output is authoritative.

The callback tries variants in a controlled order. The first accepted variant produces an opaque `Verified DetailedVerificationVariant` receipt. A caller can inspect presentation information, but it cannot manufacture a receipt merely by holding the same text.

Trust boundary

A `Verified` receipt records callback acceptance of one occurrence. It is not a behavioral certificate, a proof that a provider law is true, or a solver result. Conversely, a behavioral counterexample does not invalidate Lean’s type-checking judgment. It says that a well-typed term violates an additional contract.

18.4 The exact typed origin

Behavioral interpretation must not reverse-engineer a term graph from rendered Lean text. The same text can elaborate differently in different environments, and pretty-printing can erase distinctions that matter to the synthesis engine. For typed Exference routes, Leant therefore retains an opaque `ExactTypedVariantOrigin` sidecar. It ties the accepted spelling to such facts as:

- the exact typed candidate graph;

- the checked synthesis inventory;
- source and search goals;
- provider bindings and nominal family identities;
- renderer ordinal and rendering route;
- premise layout and request context;
- the semantic-family provenance discovered during translation.

The Length handoff obtains that origin from the Verified receipt and rerenders it internally. The caller does not separately submit a graph and claim that it corresponds to the accepted text.

In Leant/Djex

This is why the behavioral layer is currently available only for candidate routes carrying sufficient typed provenance. A text-only candidate may be perfectly good Lean, but the Length interpreter refuses to guess which Djex graph or provider occurrences produced it. In practice this means selecting the Exference engine: Leant’s default synth-engine is djinn, which supplies no typed graph, so under the default engine every candidate is retained with the payload-free refusal code unsupported-route and no worker is opened. Use `:set synth-engine exference` (or both) to produce assessable candidates; Leant prints a reminder to that effect when a configuration is activated at startup.

18.5 What each repository contributes

At the inspected snapshots, the source ownership is roughly as follows.

Layer	Primary repository area	Responsibility
Leant REPL and orchestration	src/Main.hs , src/Leant/Options.hs	Session state, commands, candidate scheduling, call-back verification, optional Length configuration, result presentation.
Leant synthesis bridge	src/Leant/Synth/Engine.hs , src/Leant/Synth/Verification.hs	Candidate groups, exact typed sidecars, rendering, opaque verification receipts.
Leant Length adapter	src/Leant/Synth/Length/	Passive Lean-named contracts, exact handoff, startup/request policy, post-verification ranking, hard selection, counterexample-bank ownership, diagnostics.
Djex checked synthesis foundation	synthesis/src/Language/Haskell/Synthesis/ and private counterparts	Inventories, names, schemes, typed candidates, occurrence certificates, fingerprints, search products.
Djex Length semantics	synthesis/src/Language/Haskell/Synthesis/Semantic/Length.hs and synthesis/src/Language/Haskell/Synthesis/Semantic/Length/	Checked finite-spine contracts, symbolic interpretation, concrete evaluation, counterexample evidence, canonical queries.
Djex solver boundary	synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/ and private <code>Internal/SMTLib</code> modules	QF-LIA rendering, bounded parsing, protocol framing, live Z3 process ownership, replay association.
Z3	src/ plus the public executable/API	SMT solving, model construction, fixedpoint reasoning, optimization and other solver capabilities.
Lean project backend	External Lean project/toolchain driven by Leant	Elaboration, name resolution, type-class synthesis, generated declaration checking, kernel authority.

Source trail

For the broad architecture, compare the existing [docs/Leant_Overview/Leant_Overview.tex](#), [docs/Djex_Leant_Codebase_Walkthrough/Djex_Leant_Codebase_Walkthrough.tex](#), and [docs/Z3_Behavioral_Synthesis_Proposal/Z3_Behavioral_Synthesis_Proposal.tex](#). The current narrow behavioral entrances are concentrated in [src/Leant/Synth/Length/Handoff.hs](#), [src/Leant/Synth/Length/Integration.hs](#), [src/Leant/Synth/Length/Ranking.hs](#), and [src/Leant/Synth/Length/Selection.hs](#).

The implemented finite-spine Length domain

19.1 Why start with length?

List length is a useful first behavioral observation because it is simple enough for exact arithmetic reasoning yet discriminating enough to improve synthesis. It has several engineering advantages:

- element values can remain completely opaque;
- finite lists are summarized by natural numbers;
- constructors have simple transfers: zero contributes 0 and a step contributes $n + 1$;
- many library functions have affine or piecewise-linear length behavior;
- contract violation becomes a quantifier-free linear-integer formula;
- counterexamples are small vectors of natural numbers that are easy to replay and display.

It also exposes the hard parts that future domains share: exact datatype identity, provider-law assumptions, symbolic case analysis, typed candidate provenance, canonical translation, model replay, evidence association, and process isolation.

19.2 A spine rather than a built-in-name convention

The semantic object is a unary finite recursive *spine*, not necessarily Lean’s built-in `List`. A contract names:

- the family;
- its zero constructor;
- its step constructor;
- the structurally recursive field of the step constructor, validated by `Djex`;
- the roles of the target’s source-ordered arguments.

For ordinary lists, the intended pattern is morally

$$\text{nil} : F\alpha, \quad \text{cons} : \alpha \rightarrow F\alpha \rightarrow F\alpha.$$

The Length model observes only how many step constructors occur. Payload fields are opaque and cannot be inspected merely because the solver would find it convenient.

Common mistake

The spelling `List` does not grant list semantics. The contract’s Lean names are matched against the exact nominal family and constructor provenance retained from translation. A different declaration with a similar shape or name is not silently coerced into the model.

19.3 Scalar and binary-product domains

The current code has two nominally distinct domains.

Scalar finite-spine length.

The modeled result is one finite spine. A contract can relate its length to the modeled input lengths.

Binary-product finite-spine lengths.

The normalized result root must be canonical Lean Prod, and both source-ordered fields must be the configured spine family. The contract may relate both output lengths to the inputs, for example

$$r_0 + r_1 = n.$$

This supports early reasoning about functions such as partitioning or splitting even though element membership and order are not yet modeled.

The two domains reuse arithmetic machinery but retain different problem, query, evidence, bank, protocol, and fingerprint types. Identical rendered SMT-LIB bytes do not make scalar evidence valid for the product domain.

19.4 The passive contract

A Leant Length contract is a passive source value. It contains no process handle and grants no permission to run Z3. Its major fields are:

Field	Meaning
Spine identity	Exact Lean family, zero-constructor, and step-constructor names.
Target argument roles	A source-ordered vector with one entry per physical target argument, each either observed-spine (a modeled spine of the configured family; observed positions are numbered compactly from zero) or unobserved-target (an opaque token that may be forwarded but never inspected, called, or deconstructed). Roles are not guessed from types.
Candidate case policy	Either reject candidate case analysis or admit the exact finite-spine zero/step case semantics supported by the current interpreter.
Precondition	A solver-neutral Length formula describing when the contract applies.
Postcondition	A solver-neutral Length formula relating modeled inputs and result length or result pair.
Provider laws	Explicit assumptions for exact provider names, each with argument roles and a transfer expression.

A scalar contract with one modeled input can express length preservation as

$$P(n) = \top, \quad Q(n, r) \equiv r = n.$$

A nonempty-only contract can use

$$P(n) \equiv n > 0.$$

A binary-product split contract might say

$$P(n) = \top, \quad Q(n, r_0, r_1) \equiv r_0 + r_1 = n.$$

Trust boundary

A contract is an assertion of intended semantics, not evidence that the named declarations really have those semantics. Exact names and schemes are checked at handoff. Provider transfer laws remain explicit assumptions unless separately connected to Lean proofs.

19.5 Provider laws

A synthesized candidate often calls an existing function whose source is not symbolically expanded. To reason about such a call, the domain needs a transfer law. For example:

$$\begin{aligned}\text{len}(\text{reverse}(xs)) &= \text{len}(xs), \\ \text{len}(\text{append}(xs, ys)) &= \text{len}(xs) + \text{len}(ys), \\ \text{len}(\text{drop}(k, xs)) &= \text{len}(xs) \dot{-} k.\end{aligned}$$

Leant does not infer these laws from names, documentation, source text, or testing. A law identifies an exact provider occurrence through the checked inventory and supplies source-ordered argument roles plus a transfer expression. Djex verifies that the provider's scheme and occurrence certificate are compatible with use of the law.

Some providers have class constraints. The current associated-provider path may use one restricted closed scheme only after its ground obligations are discharged against the exact inventory and occurrence-specific certificate chain. Z3 is never asked to decide whether a Lean type-class instance exists.

In Leant/Djex

This separation is crucial. Arithmetic reasoning may depend on the assumption that an exact provider preserves length; dictionary resolution and occurrence identity remain Djex/Lean facts. “Z3 found a model” cannot authorize a type-class dictionary or retroactively validate a provider binding.

19.6 Symbolic values in the Length interpreter

For a target with modeled input lengths $n_0, \dots, n_{k-1}$, the interpreter evaluates the exact typed candidate graph to a symbolic result expression. Relevant nodes include:

- locals and lambda-bound variables;
- applications and explicit type applications;
- lets and supported tuple forms;
- zero and step constructors of the configured spine;
- exact spine case analysis under the selected case policy;
- calls to providers with admitted occurrence-specific laws;
- opaque tokens that may be transported but not inspected.

A zero constructor denotes 0. A step constructor whose recursive field has length e denotes $e + 1$. A supported exact case on a spine of symbolic length n has the essential form

$$\text{caseLen}(n, z, s) = \text{ite}(n = 0, z, s(n \dot{-} 1)),$$

where

$$a \dot{-} b = \max(a - b, 0)$$

is natural monus. Both branches are interpreted. The set of provider-law assumptions used by the candidate is conservatively retained even if arithmetic simplification later removes a branch.

Unsupported demand is a refusal. For example, if the candidate attempts to inspect an opaque element payload, call an unmodeled callback, open an unsupported recursive structure, or use an unrecognized provider occurrence, the interpreter does not invent an unconstrained arithmetic result and call it sound.

19.7 Sealing order

The checked values form a one-way chain:

1. seal an exact synthesis inventory and spine model;
2. seal an interpretation policy;
3. seal a Length session associating inventory, spine, provider-law table, and policy;
4. revalidate the passive contract inside that session;
5. seal the exact typed candidate against that contract;
6. derive the substituted counterexample condition;
7. seal a canonical query and its complete structural fingerprint.

Each opaque value is constructible only after the checks for that stage succeed. A downstream caller cannot take a checked candidate from one inventory, a contract from another session, and bytes from a third query and associate them by record construction.

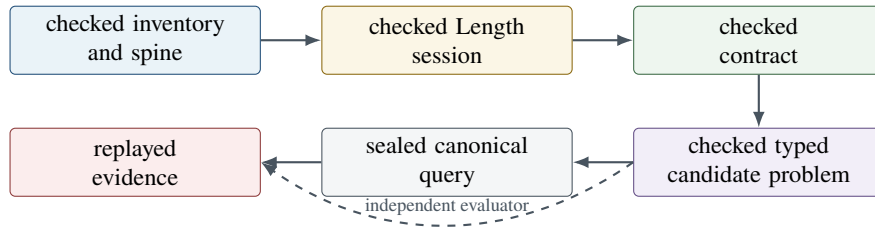


Figure 19.1: The current Length authority chain. Every arrow is a validating constructor or replay gate, not a cast.

19.8 The Leant handoff in detail

The public scalar entrance `prepareCheckedLengthProblem` takes a passive `LeanLengthContract` and an opaque callback-accepted receipt. Its fail-closed sequence is approximately:

1. require the typed candidate route and exact semantic sidecar;
2. require the source, fit, engine, and search fragments to retain the expected correspondence;
3. reject caller or constructor premises outside the current modeled entrance;
4. require the request context and goal to match the translated source goal;
5. rerender the opaque exact origin, require a unique expected rendering, and require the accepted text and renderer ordinal to match;
6. resolve the configured spine family and constructors against retained semantic-family provenance;
7. resolve every provider law against exact provider bindings, rejecting missing or ambiguous names;
8. seal the Djex session with the selected case and role policy;
9. seal the contract inside that session;
10. seal the typed candidate-specific problem.

The product entrance performs the shared checks and additionally requires a canonical `Lean Prod` result whose two fields are exact applications of the configured spine. It does not mistake `And`, `PProd`, a nested product, or a scalar spine for product-domain authority merely because synthesis used a structurally similar intermediate representation.

Common mistake

A handoff refusal is not evidence that the candidate violates the contract. It means the adapter could not soundly construct the exact modeled problem. Ranking preserves such candidates, and filtering retains them unless a separate explicit policy says that domain applicability itself is mandatory. The current public filtering path is deliberately conservative.

Source trail

Contract source vocabulary: [src/Leant/Synth/Length/Contract.hs](#). Exact origin-to-problem correspondence: [src/Leant/Synth/Length/Handoff.hs](#). Checked domain/session/problem interfaces: [synthesis/src/Language/Haskell/Synthesis/Semantic/Length.hs](#) and [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Problem.hs](#). The substantial private evaluator is under [synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/Problem/Candidate.hs](#).

From a checked Length problem to QF_LIA

20.1 The bad-state formula after symbolic interpretation

Once the candidate has been interpreted, its result is an arithmetic expression $R(\vec{n})$ over the modeled input lengths. The contract supplies $P(\vec{n})$ and $Q(\vec{n}, R(\vec{n}))$. Djex constructs the candidate-specific counterexample condition

$$\text{Bad}_C(\vec{n}) = P(\vec{n}) \wedge \neg Q(\vec{n}, R_C(\vec{n})).$$

The result variable is substituted away before SMT-LIB translation. Z3 sees only the compact modeled inputs and private arithmetic witnesses introduced by the translator.

For example, consider a one-input length-preservation contract and four symbolic candidate results:

Candidate behavior	$R(n)$	Bad state $R(n) \neq n$
Identity	n	$n \neq n$, impossible
Reverse with admitted law	n	$n \neq n$, impossible under that law
Constant empty	0	$0 \neq n$, satisfiable at $n = 1$
Duplicate	$2n$	$2n \neq n$, satisfiable at $n = 1$
Tail under exact spine case	$\text{ite}(n = 0, 0, n \div 1)$	satisfiable at $n = 1$

This table also illustrates the model's scope. Length preservation does not establish that identity and reverse return the same elements in the same order. The domain has intentionally forgotten that information.

20.2 Why QF_LIA is sufficient

The current translator emits exactly QF_LIA: quantifier-free linear integer arithmetic. The Length expression language (LengthExpression) has natural literals, input variables, sums, scaling by a literal, monus, minimum, maximum, conditionals, and positive-literal quotient and modulo; the formula language (LengthFormula) has only truth constants, equality, at-most, negation, and finite conjunction. Strict comparison is spelled as a negated at-most and there is no disjunction. Each of these has a direct QF_LIA lowering, with quotient and modulo handled by the witness scheme below.

Natural inputs are represented as SMT integers plus assertions

$$0 \leq n_i.$$

This avoids requiring a separate natural-number sort while preserving the intended domain.

The expression remains linear because variable-by-variable multiplication is not admitted. Multiplication by a literal is linear. Piecewise-linear operations are lowered with `ite` and comparisons.

20.3 Canonical preamble and commands

Every scalar and every product query begins with the same six-line preamble, rendered byte for byte as:

```
(set-option :produce-models true)
(set-option :random-seed 1)
(set-logic QF_LIA)
(define-fun djex_nat_monus ((x Int) (y Int)) Int (ite (<= y x) (- x y) 0))
(define-fun djex_nat_min ((x Int) (y Int)) Int (ite (<= x y) x y))
(define-fun djex_nat_max ((x Int) (y Int)) Int (ite (<= x y) y x))
```

Listing 20.1: The fixed preamble exactly as the current translator renders it (one command per line).

The symbol bytes (`djex_nat_monus`, `djex_length_input_0`, `djex_length_modulo_quotient_0`, ...) are chosen by the Length translator in the private `Internal.Semantic.Length.SMTLib`, not by the QF-LIA renderer, and are internal spellings rather than a stable public API. After the preamble, the query declares each input, declares any private Euclidean witnesses, asserts input nonnegativity and witness constraints, asserts the bad state, and calls `check-sat`.

The complete check program for the constant-empty candidate under unconditional length preservation ($P = \top$, $Q \equiv r = n$) is, byte for byte (this exact text is frozen by a Djex regression test in `synthesis/test-length/Spec.hs`):

```
(set-option :produce-models true)
(set-option :random-seed 1)
(set-logic QF_LIA)
(define-fun djex_nat_monus ((x Int) (y Int)) Int (ite (<= y x) (- x y) 0))
(define-fun djex_nat_min ((x Int) (y Int)) Int (ite (<= x y) x y))
(define-fun djex_nat_max ((x Int) (y Int)) Int (ite (<= x y) y x))
(declare-const djex_length_input_0 Int)
(assert (<= 0 djex_length_input_0))
(assert (not (= djex_length_input_0 0)))
(check-sat)
```

Listing 20.2: Exact scalar Length counterexample query for the constant-empty candidate.

The bad-state line is the whole of $P \wedge \neg Q$ after substitution and normalization: the precondition $\top$ has been dropped from the conjunction, and the equation is oriented input-before-literal by the normalizer's fixed term ordering. Z3 returns `sat`. Djex then sends the separate input-value request

```
(get-value (djex_length_input_0))
```

and receives a valuation such as `((djex_length_input_0 1))`.

In Leant/Djex

Although the listing above is the exact rendered text, Djex does not assemble it from free-form strings. It constructs a typed QF-LIA plan, renders canonical bytes under limits, fingerprints the plan and bytes together, and retains the checked problem needed for replay.

20.4 Positive-literal quotient and modulo without SMT `div` or `mod`

Natural quotient and remainder by a positive literal k are lowered to the unique Euclidean pair (q, r) satisfying

$$e = kq + r, \quad q \geq 0, \quad 0 \leq r < k.$$

The requested projection is q for quotient or r for modulo. The translator allocates deterministic private witness symbols in expression preorder. It deliberately does not emit SMT-LIB `div` or `mod`; this keeps the supported semantics and arithmetic fragment explicit.

For example, when $e \bmod 3$ is the first Euclidean operation met in expression preorder, the translator declares the pair `djex_length_modulo_quotient_0` and `djex_length_modulo_remainder_0` (a quotient operation uses the `djex_length_quotient_...` spellings; the ordinal counter is shared by both kinds) and, after the input-nonnegativity assertions, asserts

```
(assert (<= 0 djex_length_modulo_quotient_0))
(assert (<= 0 djex_length_modulo_remainder_0))
(assert (<= djex_length_modulo_remainder_0 2))
(assert (= E (+ (* 3 djex_length_modulo_quotient_0) djex_length_modulo_remainder_0)))
```

where E stands for the rendering of e . Note that $r < 3$ is spelled $r \leq 2$: the typed plan has no strict comparison, so the bound is the literal $k - 1$. Inside the bad-state formula the operation itself is replaced by the remainder symbol. Only original input symbols are requested from Z3. Private witnesses are part of the canonical encoding, not part of the counterexample exposed to the evaluator; the query fingerprint records which lowering schemas were used (`djex-length-z3-qf-lia-positive-literal-modulo-witness/v1` and its quotient sibling).

20.5 Input-only models

The candidate result is not requested from Z3. Djex asks only for the modeled source inputs. This design has several benefits:

- the model artifact is smaller and structurally predictable;
- no solver-chosen result value can masquerade as the candidate’s semantics;
- private lowering witnesses remain private;
- the independent evaluator must recompute the result from the exact candidate graph;
- replay tests the complete translation boundary rather than merely checking a copied Boolean flag.

The response decoder requires the exact expected symbol set. It rejects wrong arity, unknown symbols, duplicates, missing symbols, oversized names, negative values, oversized numerals, malformed S-expressions, and protocol-inconsistent responses.

20.6 Replay turns an assignment into a counterexample

Suppose Z3 reports `sat` and returns

$$n_0 = 1.$$

The current scalar replay path:

1. checks that the binding belongs to the exact sealed query;
2. restores source order from exact generated symbols;
3. converts nonnegative integers to naturals;
4. evaluates the exact retained candidate problem under $n_0 = 1$;
5. evaluates the precondition and postcondition independently;
6. requires the precondition to hold and the postcondition to fail;
7. constructs opaque domain evidence;
8. re-associates that evidence with the same behavioral-problem fingerprint.

If replay finds no violation, the model is stale, spurious, mistranslated, or otherwise unusable. It does not become a “weak counterexample.” The query transaction fails closed.

Trust boundary

The replayed fact is still relative to the finite-spine Length model and to the provider laws recorded by the problem. It is excellent evidence that the candidate violates that modeled contract. It is not a theorem about every aspect of the Lean function.

20.7 What unsat means in the current system

If Z3 reports `unsat`, then the emitted QF-LIA formula had no model according to that solver run. This is powerful information: no counterexample exists in the encoded arithmetic problem. Nevertheless, the public generic observation remains `HeuristicRankingOnly`. The system does not convert raw `unsat` into an opaque proof of the Lean behavioral theorem.

The distinction protects against:

- an unsound or stale translation;
- a contract or provider assumption attached to the wrong candidate;
- protocol desynchronization;
- a solver defect or unexpected executable;
- accidental reuse under a different inventory or encoding;
- the difference between model-relative correctness and a theorem in Lean.

A current policy may use `unsat` as a trigger for independent finite-domain validation, as a ranking signal, or as an indication that no live counterexample was found. Hard filtering does not reject on `unsat`, and it does not need to: filtering rejects violators, not candidates that survived a heuristic proof attempt.

20.8 Solver-independent checks before and after Z3

The Length stack has several useful operations that do not trust a solver status.

Counterexample-bank replay.

Re-evaluate stored input vectors against the new candidate. A hit produces fresh candidate-specific evidence without opening Z3.

Origin probe.

Test the all-zero input vector derived from the sealed problem arity. This cheaply catches many constant or malformed behaviors.

Finite input-box validation.

Exhaustively enumerate a caller-configured Cartesian box such as $0 \leq n_i \leq b_i$. Return either the first independently replayed counterexample or a bounded-positive receipt recording what was checked.

Applicable-domain validation.

Enumerate a bounded finite union describing the complete currently expressible applicable domain when that representation is admitted. Distinguish an inapplicable/vacuous contract from a nonvacuously established one.

Counterexample simplification.

Search a deterministic dominated box for a strictly smaller counterexample, preserving exact query association.

These operations explain why the live solver can be opened lazily. A batch may be fully assessed from existing samples and pure bounded evaluation.

20.9 Counterexample banks

A bank stores replay inputs, not verdicts. It is scoped by candidate-independent identities such as domain, exact inventory, contract, and interpretation/encoding policy. For each candidate, a stored sample must be replayed through the current query before it can reject that candidate.

This is an important CEGIS-ready distinction. The same input $n = 1$ may refute constant-empty and duplicate candidates but not identity. The bank says “this input has been useful in this exact problem family,” not “every future candidate is wrong at this input.”

The current Leant filtering context owns one nominal mutable bank for the dynamic lifetime of a command-local request. Progressive same-run batches can therefore learn from earlier rejected candidates. Ranking is different: the rank runners keep only a batch-local, newest-first raw store of at most four input vectors (the “MRU” described in `src/Leant/Synth/Length/Ranking.hs`), tried against later candidates of the same batch and discarded with it; the integration owner’s rank context carries no bank across batches.

20.10 Canonical query identities and limits

The scalar translator schema is currently

```
djex-length-z3-qf-lia-smtlib2/v2
```

and the product schema is

```
djex-length-spine-pair-z3-qf-lia-smtlib2/v1
```

The query fingerprint binds the exact checked problem, translator schema, typed plan, symbol order, helper and witness structure, command sequence, and rendered bytes. Execution identity is layered on top because the executable, observed digest, protocol profile, limits, and runtime options are not properties of the pure formula. Concretely, `buildQueryFingerprint` in the private `Internal.Semantic.Length.SMTLib` builds a version-1 fingerprint with role `finite-list-spine-length/z3-qf-lia-query` (the product translator uses `finite-binary-product-spine-lengths/z3-qf-lia-query`) whose tagged fields are, in order:

- `schema`: the translator schema tag above;
- `logic`: `QF_LIA`;
- `fixed-options`: `:produce-models=true` and `:random-seed=1`;
- `problem-domain`, `problem-inventory`, `problem-encoding`, `problem-candidate`, `complete-problem`: the domain tag and the four canonical fingerprints of the checked problem’s generic behavioral envelope;
- `typed-plan`: the input-symbol list, the translated bad-state condition, every check command, the value-request command or an explicit absent marker, and, when a quotient or modulo was lowered, an expression-lowering list naming the witness schemas used—all projected structurally from the typed AST, not from text;
- `check-command`: the rendered check bytes;
- `input-value-request`: the rendered value-request bytes, or absent.

The `problem-encoding` field is itself a fingerprint (role `finite-list-spine-length/concrete-encoding`) built one stage earlier from the session’s encoding policy, the contract fingerprint, a list of interpreter policy tags such as `lazy-symbolic-interpreter/v1` and `exact-zero-step-spine-case/v1`, the provider laws the candidate actually used, the normalized candidate result, and the normalized bad-state formula. Two candidates with the same rendered bytes but different used provider laws therefore have different query fingerprints; the same bytes translated for the scalar and the product domain do too.

The current default pure-query limits include:

- 65,536 bytes for each rendered command artifact;
- 262,144 bytes for the complete query fingerprint builder;
- 4,096 bits for a rendered natural numeral.

Other layers independently bound problem graphs, syntax, formula clauses, response bytes, nesting, nodes, tokens, integer widths, evaluation steps, finite-domain assignments, process queries, and wall/resource budgets.

20.11 An end-to-end example: one contract, four candidates

This section follows one small goal through the whole stage list of Chapter 13, with every emitted line derived from the translation code rather than sketched. Take the goal `List a -> List a`, the Exference engine (`:set synth-engine exference`), and the following contract-only document, which says “the result has the same length as the input” and admits two provider laws:

```

{
  "format": "leant-finite-list-spine-length-contract",
  "rankingDomain": "scalar",
  "contract": {
    "spine": {"family": "List", "zero": "List.nil", "step": "List.cons"},
    "targetArgumentRoles": ["observed-spine"],
    "candidateCasePolicy": "cases-rejected",
    "precondition": ["truth", true],
    "postcondition": ["equal", ["result"], ["input", 0]],
    "providerLaws": [
      {"name": "List.reverse", "argumentRoles": ["spine"],
        "transfer": ["argument", 0]},
      {"name": "List.append", "argumentRoles": ["spine", "spine"],
        "transfer": ["sum", [{"argument", 0}, {"argument", 1}]]}
    ]
  }
}

```

Listing 20.3: A scalar length-preservation contract with two provider laws.

Suppose the callback has verified four occurrences (their exact rendered spellings may differ from these Lean forms): `fun xs => xs`, `fun xs => List.reverse xs`, `fun _ => List.nil`, and `fun xs => List.append xs xs`.

Symbolic interpretation. With one observed spine of symbolic length n (`LengthInput 0`), the interpreter returns $R = n$ for the identity; $R = n$ for the reverse candidate through the admitted law (and records `List.reverse` as a used provider law); $R = 0$ for the zero constructor; and $R = n + n$ for the append candidate through its law. Nothing else about the candidates survives.

Contract substitution and normalization. The raw bad state is $\top \wedge \neg(R = n)$ for each candidate. Djex normalizes it before translation (`normalizeLengthFormula`): a true clause is dropped from a conjunction, an equation between syntactically equal sides folds to $\top$, and negation of a constant folds. So the identity and reverse candidates have the bad state $\perp$, the zero constructor has $\neg(n = 0)$, and the append candidate has $\neg(n = n + n)$; equations are oriented by a fixed term ordering (variables before literals before sums), which is why the input appears on the left.

Emitted queries. All four check programs share the six-line preamble, (`declare-const djex_length_input_0 Int`), and (`assert (<= 0 djex_length_input_0)`); they differ only in the bad-state assertion before (`check-sat`):

```

(assert false)                ; fun xs => xs
(assert false)                ; fun xs => List.reverse xs
(assert (not (= djex_length_input_0 0))) ; fun _ => List.nil
(assert (not (= djex_length_input_0
    (+ djex_length_input_0 djex_length_input_0)))) ; fun xs => List.append xs xs

```

Listing 20.4: The bad-state assertion emitted for each of the four candidates.

The two (`assert false`) programs are byte-identical, yet their query fingerprints differ, because the encoding fingerprint records that the second one used the `List.reverse` law. The doubled sum is not folded to $(* 2 \dots)$: the normalizer sorts and flattens sums but does not collect like terms.

Solver-independent stages first. Under the current startup policy each candidate is tried against the bank (empty at the start of the command), then against the applicable-domain traversal (a miss here, since $P = \top$ bounds nothing), then at the origin $n = 0$: all four satisfy $R(0) = 0$, so none is refuted there.

Live queries and replay. The first candidate that reaches this stage opens the one lexical worker. Z3 answers `unsat` for the two (`assert false`) programs; the assessment is `Heuristic SolverUnsatisfiable`, and because a post-`unsat` input box is configured (say `maxima [5]`), Djex then enumerates $n = 0, \dots, 5$ itself, finds no violation, and upgrades the assessment to

a nonvacuous BoundedPositive receipt (six checked, six applicable). For the zero constructor Z3 answers `sat`; `(get-value (djex_length_input_0))` returns, say, `((djex_length_input_0 1))`; the decoder checks the exact symbol set; the evaluator recomputes $R(1) = 0$, confirms $P(1)$ and $\neg Q(1, 0)$; and only then does the candidate carry a Counterexample receipt. In filter mode that vector `[1]` is recorded in the command's bank, so if the append candidate is assessed afterwards it is refuted by bank replay ($R(1) = 2 \neq 1$) without a second solver transaction; in rank mode the same effect comes from the batch-local four-entry store.

Outcomes. The two modes read the same assessments differently:

Candidate	Assessment	Rank	Filter
<code>fun xs => xs</code>	nonvacuous BoundedPositive	preferred tier	retained
<code>fun xs => List.reverse xs</code>	nonvacuous BoundedPositive (law assumed)	preferred tier	retained
<code>fun _ => List.nil</code>	Counterexample at <code>[1]</code>	demoted tier	rejected
<code>fun xs => List.append xs xs</code>	Counterexample at <code>[1]</code>	demoted tier	rejected

Ranking prints all four, the last two after the first two, each demoted one with a note of the form “replayed finite-list-spine Length counterexample (model-relative; ...): observed input spine lengths = `[1]` ...”. Filtering binds only the first two as `it1` and `it2` and prints the other two as `rejected` rows with the same note. Had the worker failed to open, or a transaction failed, both modes would have shown all four in callback order with a mode-neutral warning: no verdict is invented from an operational failure.

Source trail

Public query and replay API: [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib.hs](#). Canonical translation and model checking: [synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib.hs](#). Typed QF-LIA command AST and renderer: [synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/QFLIA.hs](#). Concrete evaluator and finite validations: [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Evaluate.hs](#) and its private implementation.

The live Z3 worker

21.1 Configuration is not execution

A `LengthSMTLibExecutionConfig` is a pure sealed policy. It describes such things as:

- the executable path and launch strategy;
- optional expected SHA-256 digest;
- direct argument prefix and policy-derived timeout/resource arguments;
- solver timeout and resource limit;
- host deadline;
- artifact policy (`LengthSMTLibArtifactPolicy`): either stop at the status, or request input values after sat; the current startup file always selects the latter (`"artifactPolicy": "input-values-after-satisfiable"`);
- bounded response-decoder policy.

Constructing this value does not prove that the executable exists, is accessible, matches the pin, is Z3, speaks the expected protocol, or can solve QF-LIA. Those are live observations made inside a scoped session.

21.2 No shell, ambient environment, or inherited working directory

The current execution profile launches Z3 directly with the fixed argument vector `-in -smt2 smtlib2_compliant=true timeout=<ms> rlimit=<n>`, where the last two values come from the sealed policy. It does not invoke a shell. The child receives an exact empty environment (`env = Just []`) rather than inherited variables, and it runs in a newly created empty working directory rather than the caller's directory. The first command written is `(set-option :print-success false)`, and every query is preceded by `(reset)` followed by that same option again.

These choices reduce ambiguity and attack surface:

- shell metacharacters cannot reinterpret a path or argument;
- environment variables cannot silently change library loading, locale, or solver behavior;
- project files cannot be discovered through the inherited current directory;
- command identity is easier to fingerprint and audit.

The three Linux descriptor-bound strategies bind execution to an already inspected and sealed image and perform progressively stronger executable-access checks; the current Leant startup decoder always selects the strongest one, `LengthSMTLibDescriptorBoundExecveCheckExecutableAccessLaunch`. On a build without the native launcher (any non-Linux host) the live opener fails closed with the sanitized class `LengthSMTLibLiveSessionLaunchFailed`, and Leant preserves the batch in callback order. The optional digest pin is checked against the staged image bytes. Even when no expected digest is configured, the live identity binds the observed digest.

Common mistake

A path string is not a stable executable identity. The file at that path can change between inspection and spawn. That time-of-check/time-of-use problem is why the implementation has descriptor-bound launch profiles rather than relying only on “hash path, then run path.”

21.3 Lexically scoped ownership

The public API lends a capability-probed worker only inside a rank-2 scoped callback. Callers cannot inspect or retain:

- a process handle;
- the workspace or executable path;
- the raw transcript;
- cancellation or cleanup internals;
- decoded valuations before replay;
- run ordinals or reversible process identities;
- causal framing markers.

The newer scoped usable-work token adds runtime owner-thread and dynamic-extent checks. A Haskell closure may retain the opaque token syntactically, but public operations reject it after the owning callback exits or from another thread.

21.4 Readiness probing

Opening a process is not enough. The session performs a readiness/capability probe for the exact operational profile needed by Length. It consists of four writes; each ends with an echo of a fresh 256-bit nonce, spelled (echo "djex-smtlib-frame/v1/<64 hex digits>"), and the phase machine advances only when that exact quoted string is the next complete frame:

```

; write 1                                ; expected: only the echo
(set-option :print-success false)
(echo "djex-smtlib-frame/v1/...")
; write 2                                ; expected: sat, then the echo
(reset)
(set-option :print-success false)
<the six-line canonical preamble>
(declare-const djex_capability_input Int)
(assert (= djex_capability_input 0))
(check-sat)
(echo "djex-smtlib-frame/v1/...")
; write 3                                ; expected: ((djex_capability_input 0)), then the echo
(get-value (djex_capability_input))
(echo "djex-smtlib-frame/v1/...")
; write 4                                ; expected: unsat, then the echo
(reset)
(set-option :print-success false)
<the six-line canonical preamble>
(declare-const djex_capability_input Int)
(assert (= djex_capability_input 0))
(assert (= djex_capability_input 1))
(check-sat)
(echo "djex-smtlib-frame/v1/...")

```

Listing 21.1: The four readiness writes and the responses they require (barrier nonces abbreviated).

Between phases only SMT-LIB whitespace may cross a write boundary, so a value or status emitted early can never satisfy a later phase. This checks QF-LIA acceptance, reset behavior, sat/unsat framing, the exact input-value response shape, causality under

chunked streams, bounded-parser compatibility, and transport liveness. The plan, its four writes, and the expected responses are sealed into a capability fingerprint that becomes part of the ready-worker identity.

This is a protocol capability, not a theorem-proving certificate. A malicious process could mimic the probe and later lie. Independent replay limits the damage of false satisfying models; raw unsat remains nonauthoritative.

21.5 One query transaction

A simplified successful scalar transaction is:

1. establish that the session is ready and has remaining query budget;
2. reset to the known base state;
3. send the canonical check bytes under causal framing;
4. decode exactly one status response;
5. if sat and the query has inputs, send the canonical input-value request;
6. decode the exact binding set under response limits;
7. replay the assignment against the retained checked problem;
8. return a query-associated observation and optional replayed evidence;
9. restore or close the worker according to transaction outcome.

A protocol error, malformed response, timeout, resource limit, stale marker, unexpected EOF, failed replay, or cleanup problem is not silently converted to unknown. It is represented by a sanitized failure class and the batch-level policy decides how conservatively to continue. On the wire, one scalar transaction is at most two writes. The first is the reset prefix, the sealed check bytes, and a fresh echo barrier; the second, sent only after sat when the query has inputs and the artifact policy asks for values, is the sealed value request and another barrier:

```
; write 1: (reset), print-success off, the sealed check bytes, barrier
(reset)
(set-option :print-success false)
<the sealed check program, ending in (check-sat)>
(echo "djex-smtlib-frame/v1/...")
; expected: exactly one status frame (sat | unsat | unknown), then the echo

; write 2 (only after sat, only if the query has inputs)
(get-value (djex_length_input_0 ...))
(echo "djex-smtlib-frame/v1/...")
; expected: exactly one valuation frame, then the echo
```

Listing 21.2: The two writes of one query transaction (barrier nonces abbreviated).

The 256-bit nonce in each barrier is fresh for that barrier (derived from a per-session seed, with the spent check and value roles recorded in the run identity), so a frame answering an earlier command cannot be mistaken for the answer to this one even if it was buffered late.

21.6 Budgets and cleanup

The execution layer distinguishes solver-local limits from host ownership limits. A solver timeout requests that Z3 stop work; a host deadline prevents the parent from waiting indefinitely if the child ignores it. Resource limits bound solver work. Response limits bound what the parent will parse. Shared usable-work budgets can cover opening and several candidate queries under one absolute deadline while reserving separate cleanup windows.

The current public default maximum is 64 scalar-plus-product transactions per live scope, not 64 for each domain. Cleanup remains owned even when the callback throws, receives an asynchronous exception, or exhausts its usable-work window. The result records whether cleanup was incomplete without exposing child-controlled diagnostic bytes at the public boundary.

21.7 Causal parsing

Operating-system reads do not preserve SMT-LIB response boundaries. One read may contain half a response, several responses, or a marker split across chunks. The private stream and causal driver therefore tokenize bounded S-expressions incrementally and associate responses with explicit transaction framing.

The parser must also tolerate legal SMT-LIB whitespace and comments without accepting arbitrary trailing payloads. A status response and a value response are different protocol states. The code does not call a generic “parse some S-expression” function and hope that the next object belongs to the current command.

21.8 Artifact policy

Raw child output is useful for debugging but can be large, malformed, sensitive, or attacker-controlled. The execution configuration therefore has an explicit artifact policy. Public errors are sanitized into stable byte-free classes. Internal bounded artifacts can be retained under policy for diagnostics and exact observation association, but they do not automatically cross every API layer.

Trust boundary

Process hardening and replay solve different problems. Hardening limits substitution, injection, desynchronization, denial of service, and ownership bugs. Replay limits semantic dependence on a returned satisfying model. Neither by itself proves that a raw unsat reply is correct.

Source trail

Pure execution policy: [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Execution.hs](#). Public scoped live facade: [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs](#). Private process/session/protocol stack: [synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session.hs](#), the sibling `Session/` modules, [synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Z3/Process.hs](#), and [synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Causal/Driver.hs](#).

Leant's rank and filter modes

22.1 Behavioral assessment is disabled by default

A normal Leant process starts without Length assessment authority. In that state:

- ordinary `:synth TYPE` behaves as before;
- the disabled ranking path is a lazy identity and performs no Z3-related IO;
- a request for hard filtering is refused before any contract file is admitted or read;
- a request-scoped Length contract cannot borrow a nonexistent execution policy.

This is not automatic solver discovery. Leant does not search `PATH`, infer a Z3 installation, normalize a caller path, or silently accept an unpinned executable. The complete policy enters through an explicit startup configuration.

22.2 Startup activation

The current command-line options are:

```
leant --length-ranking-config /absolute/path/config.json [other options]

# Optional: bounded file-load timeout in milliseconds
leant --length-ranking-config /absolute/path/config.json \
      --length-ranking-config-timeout 5000

# Explicitly permit a configuration without an executable SHA-256 pin
leant --length-ranking-config /absolute/path/config.json \
      --length-ranking-allow-unpinned
```

Listing 22.1: Current Leant Length-assessment startup options.

The default load timeout is 5000 milliseconds and the maximum is 60,000. The current acquisition entrance is POSIX-only (Windows fails closed before reading) and requires an admitted absolute path. Unless `--length-ranking-allow-unpinned` is present, activation rejects a policy with no expected executable digest.

The startup file is bounded JSON (at most 256 KiB) with one versionless current grammar. Its root has exactly ten required members, validated in this order: the fixed format literal `leant-live-length-ranking-configuration`; `rankingDomain`, selecting scalar or binary-product; and then

- `executionAdmission`: the executable-path character bound and the policy-fingerprint byte bound;
- `execution`: absolute executablePath, optional expectedExecutableSha256, solver timeout, solver resource limit, host deadline, artifactPolicy, and response byte/depth/node/token/integer bounds;
- `evaluation`: concrete-replay value-width limits;

- `inputBoxValidation`: the post-unsat finite input box (inclusive per-input maxima and an assignment cap);
- `applicableDomainValidation`: the caps of the guarded recursive applicable-domain traversal;
- `counterexampleSimplification`: input-count and assignment caps for counterexample simplification;
- `usableWorkBudget`: one shared usable-work duration in milliseconds (at most 65,000);
- `contract`: one fixed scalar or binary-product contract in the selected grammar.

There are no strategy switches. Every accepted file selects the same policy bundle: the descriptor-bound `execve-check` launch strategy, the all-zero origin probe, both non-vacuous positive-receipt preferences, deferred live-session opening, and the scoped (v2) usable-work owner. Decoding is pure and grants no execution authority. Activation separately decides whether an absent digest pin is acceptable. Loading and activation do not launch Z3; an enabled deferred policy may still complete an all-pure batch without starting a process.

Checkpoint

There are three distinct moments: *decode the file*, *authorize its executable policy*, and *open a live worker*. Keeping them separate makes it possible to reject malformed or insufficiently pinned configuration before any child process exists.

22.3 Request syntax

The current command parser recognizes two request-scoped options in a fixed order:

```
:synth --length-contract /absolute/path/contract.json -- TYPE

:synth --behavior-mode rank \
      --length-contract /absolute/path/contract.json -- TYPE

:synth --behavior-mode filter \
      --length-contract /absolute/path/contract.json -- TYPE
```

Listing 22.2: Current Length-aware `:synth` forms.

The `--behavior-mode` option, when present, must precede `--length-contract`. Once either option is recognized, the standalone `--` delimiter is mandatory. Without an explicit mode, a contract request uses `rank`. A configured startup contract can be used without naming another contract:

```
:synth --behavior-mode filter -- TYPE
```

A request-scoped contract is passive. It reuses the already activated startup execution/replay policy. It cannot introduce a second executable policy or activate a disabled process.

22.4 Ranking mode

Ranking is the conservative default. It retains every callback-verified occurrence and seals a permutation of the original batch. Candidate assessments include:

Unassessed.

No usable semantic assessment was obtained.

Heuristic.

Carries the raw `SolverStatus` (`SolverSatisfiable`, `SolverUnsatisfiable`, or `SolverUnknown`); no independently replayed counterexample or positive finite receipt was attached.

Counterexample.

An input vector independently reproduced the contract violation.

BoundedPositive.

Exhaustive finite-box validation completed without a counterexample; the receipt records the applicable-assignment count.

ApplicableDomainEstablished.

The current finite representation of the complete applicable domain was independently traversed and established.

The historical stable ranking moves replayed counterexamples after all other candidates, preserving source order within both partitions. Two additive preferences, both enabled by the current startup decoder, move nonvacuous applicable-domain receipts first and nonvacuous bounded-positive receipts next, while vacuous receipts (no assignment met the precondition) stay neutral. The resulting stable order is: nonvacuous applicable-domain receipts, then nonvacuous input-box receipts, then everything neutral (Unassessed, Heuristic, preparation refusals, vacuous receipts), then replayed counterexamples; original order is kept inside each partition. Raw status alone is neutral.

If any batch-wide operational or sealing failure occurs, the ranking result preserves callback order. Candidate-local preparation refusals are diagnostic and do not become negative behavioral evidence.

22.5 Filter mode

Filter mode is an explicit hard-selection boundary. It partitions the exact verified occurrences into selected and rejected collections, preserving occurrence association and original-order semantics. The decisive current rule is narrow:

A candidate may be rejected only when the Length adapter has independently replayed a counterexample against that exact candidate-specific checked problem.

The following do *not* reject a candidate:

- raw sat, unsat, or unknown status;
- a positive finite-box receipt;
- applicable-domain establishment or inapplicability;
- a handoff or query-preparation refusal;
- inability to open or use Z3;
- timeout or resource exhaustion;
- an unassessed candidate;
- a batch sealing failure.

Instead, these candidates are retained with an explicit retention class. Every operational or selection-seal failure returns the complete original callback batch. This makes hard filtering fail open with respect to availability while remaining fail closed with respect to evidence: it may miss a rejection, but it does not invent one.

Trust boundary

The selection seal proves only that the output is a total occurrence-respecting partition of the supplied verified batch. Domain-specific private builders decide which occurrences may enter the rejected side. The generic seal itself does not know what a counterexample means.

22.6 Same-run progressive filtering

A filter request introduces one nominal command-local counterexample-bank context. Leant can feed several synthesis batches through that same context. When an early candidate yields a replayed counterexample, its input vector is recorded and tried against later candidates before another live query.

This produces an elementary CEGIS-like feedback effect inside one command even though the candidate generator itself is not yet solver-guided:

1. Djex enumerates a bounded candidate prefix (Main consumes an opaque engine cursor in batches of at most 12 candidate groups, or 24 when both engines run);
2. Lean verifies the batch of occurrences;
3. Length filtering rejects replayed violators and records their input vectors in the bank;
4. only if the batch had no verified candidate at all, or every verified candidate was rejected, does Main request at most one same-width successor batch; a batch with even one survivor, or a preserve-all fallback, is terminal (this is not survivor-quota filling);
5. the successor batch first replays the retained inputs against each new candidate;
6. Z3 is opened only for candidates that survive pure replay and the other enabled solver-independent checks.

The bank does not outlive its nominal request context, and it stores only bounded input vectors. It does not cache verdicts, opaque evidence receipts, or raw solver observations.

22.7 A worked conceptual session

The following transcript is illustrative rather than a promise of exact current presentation text (Section 20.11 derives the exact queries and verdicts behind it). Assume startup configuration enables the scalar List spine, a pinned Z3 worker, and a length-preservation contract, and that `:set synth-engine exference` has been issued: under the default djinn engine no candidate carries a typed graph, so every one is retained with the refusal code `unsupported-route` and no worker is opened. In filter mode the omitted occurrences are printed as separate rejected rows carrying the counterexample note; only survivors are bound as `it1, it2, ...`.

```
-- Every shown term has already been accepted by Lean at List a -> List a.

:synth List a -> List a
fun xs => xs
fun xs => xs.reverse
fun _ => []
fun xs => xs ++ xs

-- Explicit Length assessment in ranking mode.
:synth --behavior-mode rank -- List a -> List a
fun xs => xs           -- no replayed length counterexample
fun xs => xs.reverse   -- no replayed length counterexample
fun _ => []            -- counterexample, e.g. input length 1
fun xs => xs ++ xs     -- counterexample, e.g. input length 1

-- Explicit hard selection.
:synth --behavior-mode filter -- List a -> List a
fun xs => xs
fun xs => xs.reverse
-- the two replayed violators are omitted and reported as rejections
```

Listing 22.3: Conceptual ranking and filtering outcomes.

Identity and reverse both survive because length is only one observation. That is correct behavior for this contract, not a solver limitation.

22.8 Failures are classified, not flattened

The public layers preserve distinctions such as:

- unsupported candidate route;
- typed authority unavailable;

- candidate or rendering association rejected;
- spine or provider binding unavailable;
- session, contract, candidate semantics, or query construction rejected;
- live session unavailable or capability rejected;
- query deadline/resource/transport/protocol failure;
- counterexample replay rejected;
- batch input limit exceeded;
- cleanup incomplete.

Stable presentation codes omit source snippets, private graph coordinates, executable paths, digests, and child-controlled output. The original verified occurrence remains attached internally so association is not reduced to matching a diagnostic string.

22.9 Current feature status

Capability	Status	Meaning
Lean callback verification	Implemented	Every displayed synthesis candidate is re-elaborated against the exact goal.
Scalar finite-spine Length interpretation	Implemented	Exact checked fragment with explicit roles, cases, and provider laws.
Binary-product spine lengths	Implemented	Canonical Lean Prod with two configured spine fields.
Canonical QF-LIA SMT-LIB	Implemented	Typed plan, canonical bytes, structural fingerprint, input-only model request.
Hardened scoped Z3 execution	Implemented	Bounded, capability-probed subprocess and protocol ownership.
Independent model replay	Implemented	Required before a satisfying assignment becomes counterexample evidence.
Post-verification ranking	Implemented	Stable permutation; no pruning.
Opt-in hard filtering	Implemented, narrow	Rejects only exact replayed Length counterexamples.
Same-request counterexample bank	Implemented	Progressive batches can reuse validated input vectors.
Solver-guided candidate generation	Not yet	Z3 does not currently choose Djex productions.
General CEGIS loop	Not yet	Enumeration is not yet driven by accumulated behavioral samples.
Richer semantic domains	Proposed	Sequence, bag, scalar, bit-vector, state, cost, and others.
Positive Lean proof artifacts	Proposed	Raw unsat is not yet upgraded to a checked contract theorem.
CHC/Spacer recursion path	Proposed	Intended for recursive relational semantics and invariant discovery.

Source trail

Request parser: [src/Leant/Synth/Length/Command.hs](#). Startup options: [src/Leant/Options.hs](#). Configuration decoding and activation: [src/Leant/Synth/Length/Configuration/File.hs](#) and [src/Leant/Synth/Length/Integration.hs](#). Ranking: [src/Leant/Synth/Length/Ranking.hs](#). Generic occurrence-sealed selection: [src/Leant/Synth/BehavioralSelection.hs](#). Scalar filter policy: [src/Leant/Synth/Length/Selection.hs](#); the binary-product sibling is under [SpinePair/Selection.hs](#). Main's progressive scheduler and presentation are in [src/Main.hs](#).

A maintainer's source map for one assessment

23.1 Trace from command line to verified batch

When debugging a Length-aware synthesis command, follow this order:

1. `src/Leant/Options.hs`: did startup parse and authorize a configuration source?
2. `src/Leant/Synth/Length/Configuration/File/Acquire.hs`: was the file admitted and acquired under its bounded path/timeout policy?
3. `src/Leant/Synth/Length/Configuration/File.hs`: did bounded JSON decoding and pure policy construction succeed?
4. `src/Leant/Synth/Length/Integration.hs`: did pin activation succeed, and what request/context was introduced?
5. `src/Leant/Synth/Length/Command.hs`: how were command-local mode, contract path, delimiter, and goal parsed?
6. `src/Main.hs`: which synthesis lane and batches were generated, verified, assessed, and displayed?
7. `src/Leant/Synth/Verification.hs`: which candidate variants did the callback actually attempt, and which occurrence received a Verified receipt?

Do not begin by inspecting Z3 output. Many candidates never become eligible for a Length query, and many eligible candidates can be classified from pure replay.

23.2 Trace from a verified occurrence to a query

For one eligible occurrence:

1. `src/Leant/Synth/Length/Handoff.hs` recovers and validates exact typed provenance.
2. `src/Leant/Synth/Length/Adapter.hs` seals the checked query wrapper and maps detailed domain/query refusals.
3. `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Problem.hs` exposes the opaque checked candidate problem.
4. `synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/Problem/Candidate.hs` performs graph interpretation and problem sealing.
5. `synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib.hs` translates the substituted bad state and seals canonical bytes.
6. `synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/QFLIA.hs` renders the restricted command AST.

A failure at any of these stages is a pure preparation refusal. No live worker is required to discover it.

23.3 Trace through live execution

For a live miss:

1. `src/Leant/Synth/Length/Ranking/Internal.hs` (or its nominal twin `src/Leant/Synth/Length/SpinePair/Ranking/Internal.hs`) runs the per-candidate source order—bank replay, applicable-domain traversal, origin probe, live query, post-unsat input box—and decides when to open the one lexical worker under the configured policy. The selection adapters in `src/Leant/Synth/Length/Selection/Internal.hs` run no queries; they consume the finished ranking assessment.
2. `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs` exposes the public scoped capability (with `lengthSMTLibLiveSession...`, `runLengthSMTLibLiveQuery`, `replayLengthSMTLibLiveQueryObservation`).
3. The private `synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session.hs` owns the ready worker: workspace, launch, the four-write readiness probe from `synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session/Capability.hs`, query leases, and run identities.
4. The private `synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Protocol.hs` is the pure per-query phase machine (reset + check bytes + barrier, one status frame, optional value request + barrier).
5. The private `synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Causal/Driver.hs` and the stream/response modules perform the writes and maintain framing over chunked stdout.
6. The private `synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Z3/Process.hs` owns process launch (including the descriptor-bound pipeline), pipes, cancellation, and cleanup.
7. The public replay gate returns a status plus optional independently validated evidence.

When diagnosing a reported unknown, distinguish a genuine solver unknown from host timeout, resource exhaustion, transport failure, malformed response, capability rejection, and an application-level refusal. They have different remediation and trust implications.

23.4 Stage by stage: owning module and failure class

The following table lists, for one Length-aware `:synth`, each stage in execution order, the module that owns it, the closed failure vocabulary it can raise, and what that failure does to the batch. “Candidate-local” means the occurrence stays in place with a payload-free refusal code and the rest of the batch proceeds; “batch” means every verified occurrence is preserved in callback order as `Unassessed` and `Main` prints a mode-neutral warning.

Stage	Module	Failure vocabulary	Effect
Option parsing	<code>src/Leant/Options.hs</code>	usage error (<code>--length-ranking-config-timeout</code> and <code>--length-ranking-allow-unpinned</code> require <code>--length-ranking-config</code>)	startup refused
File acquisition	<code>src/Leant/Synth/Length/Configuration/File/Acquire.hs</code> over the shared <code>src/Leant/Synth/Length/File/Acquire.hs</code>	<code>LengthFileAdmissionError</code> , sanitized load-error class; <code>LengthFilePlatformUnsupported</code> on Windows	startup refused
Decoding	<code>src/Leant/Synth/Length/Configuration/File.hs</code>	<code>LengthRankingConfigurationFileError</code> : bounded-JSON, format, rankingDomain, exact ten-member root, then per-section field errors	startup refused
Activation	<code>src/Leant/Synth/Length/Integration.hs</code>	<code>LengthAssessmentActivationRejected</code> (no digest pin without <code>--length-ranking-allow-unpinned</code>)	startup refused
Command syntax	<code>src/Leant/Synth/Length/Command.hs</code>	<code>LengthSynthCommandError</code> : mode missing/invalid, delimiter missing, contract path missing, mode after contract	command refused
Request admission	<code>src/Leant/Synth/Length/Integration.hs</code>	<code>LengthAssessmentRequestError</code> : an explicit contract path or a filter request without an activated startup policy	command refused before file IO
Batch admission	<code>src/Leant/Synth/Length/Ranking/Internal.hs</code>	<code>LengthRankingInputLimitExceeded</code> (more than 64 candidates)	batch

Stage	Module	Failure vocabulary	Effect
Handoff	src/Leant/Synth/Length/Handoff.hs	LengthHandoffRefusal, surfaced as the codes unsupported-route, typed-authority-unavailable, candidate-association-rejected, rendering-association-rejected, spine-binding-unavailable, provider-binding-unavailable, session-rejected, contract-rejected, candidate-semantics-rejected	candidate-local
Query sealing	src/Leant/Synth/Length/Adapter.hs calling Djex.sealLengthSMTLibQuery	LengthSMTLibQueryError (result variable leaked, input out of range, zero divisor, numeral-bit, command-byte, or fingerprint-byte limit), surfaced as query-construction-rejected	candidate-local
Pure validations	Djex.Semantic.Length.SMTLib/Evaluate	LengthRankingFailureClass constructors for origin-probe evaluation, applicable-domain validation, input-box validation, and counterexample-simplification failures (bounded admission misses are ordinary, not failures)	batch
Worker opening	Djex.synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs	LengthRankingLiveSessionFailed carrying LengthSMTLibLiveSessionFailure: deadline exceeded, workspace unavailable, executable unavailable/rejected, launch failed, capability rejected, resource limit, transport, cleanup, internal, usable-work scope unavailable	batch
Query transaction	same	LengthRankingLiveQueryFailed carrying LengthSMTLibLiveQueryFailure: session unavailable, query limit, deadline, configuration rejected, resource limit, transport, protocol rejected, counterexample rejected, internal	batch
Association and replay	Djex replay gates	LengthRankingQueryAssociationMismatch, LengthRankingEvidenceReplayMismatch	batch
Permutation seal	src/Leant/Synth/PostVerification.hs	PostVerificationError: collection limit, length mismatch, index out of range, duplicate index	batch
Selection seal	src/Leant/Synth/BehavioralSelection.hs via src/Leant/Synth/Length/Selection/Internal.hs	LengthSelectionFailure: post-verification failed, ranking failed/unavailable, index out of range, seal failed	batch

Solver status is deliberately absent from the failure column: sat, unsat, and unknown are assessments (Heuristic), not failures, and none of them changes the batch shape by itself.

23.5 Trace through ranking or selection

The post-verification layers protect occurrence association:

Ranking.

[src/Leant/Synth/PostVerification.hs](#) admits an exact batch, the Length ranker produces occurrence-associated decisions, and a permutation seal verifies that every original verified receipt appears exactly once.

Selection.

[src/Leant/Synth/BehavioralSelection.hs](#) introduces a fresh rank-2 epoch and seals a total selected/rejected partition. Domain-private builders can reject only under their evidence policy.

Presentation consumes the sealed result. It should never reconstruct selection by comparing rendered strings, because duplicate spellings and repeated semantic candidates are valid occurrences.

23.6 Public facade versus private implementation

Many Djex modules have a small public facade that reexports opaque types and projections from a large private implementation. This is deliberate, not merely directory organization. Public modules describe what callers may construct or observe. Private modules may retain raw bytes, graph coordinates, launch material, or validating constructors that would destroy the authority boundary if exported.

A useful reading pattern is:

1. read the public module header to understand the contract;
2. inspect exported constructors and projections;
3. only then enter the private implementation to study the algorithm;
4. check that no new public projection leaks the material the header promises to hide;
5. treat a convenience conversion as suspicious if it can reassociate a checked value without replay.

23.7 Changes between Leant’s pinned Djex and Djex head

Leant builds against the submodule revision recorded on the title page. The independent Djex repository head is slightly newer. This document uses the pinned revision when describing code that Leant can call and uses the head only to note additive current-tree developments. A maintainer should never assume that a symbol visible at Djex head is already available through Leant’s submodule.

The strongest rule is simple: inspect `lib/Djex` before changing Leant. Commit-pinned links in this document make that distinction visible.

Checkpoint

When a behavioral result looks wrong, identify the earliest boundary at which the unexpected fact appears: candidate generation, Lean callback, exact-origin handoff, symbolic interpretation, query translation, process protocol, response decoding, independent replay, ranking/selection sealing, or presentation. Jumping directly to “Z3 bug” skips most of the system.

Part V

How Z3 Is Planned to Be Used Next

From post-verification assessment to behavioral synthesis

24.1 The sequential architecture today

The implemented flow is mostly sequential:

type-directed generation $\longrightarrow$ Lean callback verification $\longrightarrow$ Length assessment.

This already gives useful ranking and safe opt-in rejection (`--behavior-mode filter`). In filter mode the disposition of one assessed batch even decides whether Main forces a second ordered batch out of the same lazy engine outcome: a successor is requested only after no verification or complete all-rejection, and never a third time. What the tree does not yet do is use behavioral information to decide *which* candidates Djex generates next. The counterexamples learned so far never reach the enumerator, so if every type-correct candidate in a batch violates the same simple property, the generator still produces the whole batch, Lean still checks each candidate, and only then does the assessor say the same thing repeatedly.

The planned architecture adds feedback:

typed grammar/search $\rightleftharpoons$ samples and semantic summaries
 $\rightleftharpoons$ solver constraints $\longrightarrow$ Lean-verified candidate and checked evidence.

Z3 is valuable in three distinct loops:

1. **Verification loop:** find a counterexample to one candidate.
2. **Synthesis loop:** choose among a finite set of typed productions so that all current samples are satisfied.
3. **Recursive-proof loop:** search for a bad trace or an inductive invariant in a CHC model, then turn the result into replayable evidence or a Lean proof obligation.

24.2 Governing invariants

Every extension should preserve the following principles.

Djex owns syntax and typing.

Z3 may select among alternatives prepared by Djex. It does not invent arbitrary Lean text, resolve binders, choose implicit arguments, or reconstruct type-class evidence.

Lean remains the exact source-language checker.

Every materialized candidate is re-elaborated against the exact goal. A solver model does not bypass callback verification.

Behavior is domain-scoped.

A Length result is about spine lengths; a bit-vector result is about fixed-width arithmetic; a state/provenance result is about its relational model. No observation silently expands its semantic claim.

Counterexamples require replay.

A model is decoded under limits and independently re-evaluated against the exact candidate-specific problem before it can guide search or reject a candidate.

Positive claims need explicit authority.

Finite enumeration can produce a bounded-positive receipt. A complete exact finite-domain traversal can produce stronger model-relative evidence. General source-level correctness requires a checked Lean proof or an explicitly chosen external-solver trust policy.

Learned state is fingerprint-scoped.

Samples, semantic signatures, cores, nogoods, objectives, and caches are reusable only under the exact grammar, inventory, contract, domain, encoding, and policy identities that justify them.

Unknown is not false.

Timeout, resource exhaustion, unsupported semantics, and solver unknown are separate outcomes. Search policy may postpone or preserve a candidate, but it must not rewrite uncertainty into violation.

24.3 Five increasingly strong integration levels

Level	Main idea	What Z3 does	Status at the inspected tree
1	Post-verification behavioral assessment	Search for a counterexample to each checked candidate	Ranking and replay-authorized hard filtering implemented for Length
2	Counterexample-guided enumeration	Supply separating samples and equivalence classes to Djex's enumerator	Bounded slice implemented: one command-local nominal counterexample bank (src/Leant/Synth/Length/CounterexampleBank/Internal.hs) is replayed across at most two ordered same-run filter batches taken from one lazy engine outcome through the Engine cursor (src/Leant/Synth/Engine.hs); counterexample-directed generation and semantic signatures are planned
3	Typed finite sketches	Solve selector, wiring, literal, and finite-table variables in a Djex-prepared DAG	Planned
4	Semantic prefix pruning	Refute incomplete search states using sound over-approximations	Planned
5	Recursive relational synthesis/proof	Solve CHCs for traces or invariants; support richer domains and proof artifacts	Planned research path

The levels are additive. Level 3 does not eliminate Level 1: a selector model still materializes a candidate, Lean still checks it, and the exact behavioral verifier still tries to refute it. Level 5 does not eliminate replay or proof checking. The numbering compresses the proposal's ladder. The proposal [10] distinguishes six levels: its Level 5 is the recursive CHC path alone, and its Level 6 is an explicitly experimental *specification and law inference* mode in which Z3 proposes contract parameters or provider summaries from examples, proposals that never become assumed laws without user acceptance or a Lean theorem. This guide folds richer domains and proof artifacts into its Level 5 and does not plan law inference. When the two documents are read side by side, compare mechanisms rather than level numbers.

24.4 A generic behavioral domain interface

The Length implementation suggests a reusable conceptual interface for future domains:

```
class BehavioralDomain d where
  type ContractSource d
  type CheckedContract d
  type CheckedProblem d
  type Query d
  type ModelPayload d
  type Counterexample d
  type PositiveReceipt d

  sealContract  :: Session d -> ContractSource d
                -> Either (ContractError d) (CheckedContract d)
  sealProblem   :: CheckedContract d -> VerifiedCandidate
                -> Either (ProblemError d) (CheckedProblem d)
  encodeBadState :: CheckedProblem d
                -> Either (EncodingError d) (Query d)
  decodeModel   :: Query d -> BoundedResponse
                -> Either (ModelError d) (ModelPayload d)
  replayModel   :: CheckedProblem d -> ModelPayload d
                -> Either (ReplayError d) (Maybe (Counterexample d))
  validateFinite :: CheckedProblem d -> FiniteScope d
                -> Either (ValidationError d)
                    (Either (Counterexample d) (PositiveReceipt d))
```

Listing 24.1: Conceptual domain responsibilities, not a current public API.

A sketch-capable domain adds sample encoding and partial-summary operations. A CHC-capable domain adds relational rules, trace decoding, and invariant translation. The types should remain nominally distinct across domains even when they share an SMT theory.

24.5 Exactness is multidimensional

Future results need more than a Boolean “proved/not proved” flag. Relevant dimensions include:

- exactness of the source fragment interpreted;
- exactness of provider laws;
- finite versus unbounded input scope;
- exact versus over- or under-approximate value abstraction;
- solver status versus independently replayed evidence;
- model-relative result versus checked Lean theorem;
- assumptions about purity, totality, extensionality, or termination;
- whether recursive semantics is bounded unfolding or inductive.

For example, a candidate may be interpreted exactly for list length, conditional on an assumed law for `List.reverse`, validated for every input in a finite box, and still have no Lean proof of the universal theorem. A sequence model may be exact for bounded finite atoms but under-approximate arbitrary element equality. A CHC relation may over-approximate execution and therefore justify safety pruning, while its concrete trace decoder can still produce exact counterexamples.

Mental model

An “exactness descriptor” says which bridge was crossed and under which assumptions. It prevents a useful bounded fact from being presented as a universal source theorem.

24.6 Planned mechanisms, Z3 features, and status at a glance

The chapters of this Part discuss one mechanism at a time. The table below lists them together with the Z3 feature each relies on and its status at the inspected tree, so that a reader can tell at a glance what is already running from what is only argued for. Every Z3 feature named here was exercised against Z3 5.0.0 while writing this Part; the exact scripts and answers appear in the sections cited.

Planned mechanism	Z3 feature relied on	Status at the inspected tree
Post-verification counterexample search (Level 1)	QF_LIA check-sat plus get-value, one reset/check/value transaction per query	Implemented for Length: ranking by default, hard filtering under <code>--behavior-mode filter</code> with the occurrence-sealed partition of src/Leant/Synth/BehavioralSelection.hs
Counterexample bank replay	none (pure Djex replay of stored input vectors)	Implemented: four-entry batch-local MRU on the rank path; command-local nominal bank on the filter path (src/Leant/Synth/Length/CounterexampleBank/Internal.hs)
Progressive same-run batching	none	Implemented: at most two ordered batches from one lazy engine outcome through the Engine cursor, a successor only after no verification or complete all-rejection
Guarded-conditional finite domains	none (pure applicable-domain validation)	Implemented: guarded recursive piecewise-affine validation over <code>LengthIf</code> summaries
Semantic signatures and scheduling	none at first; later check-sat equivalence queries (Section 25.5)	Planned, Milestone 1
Candidate-level CEGIS with generator feedback	incremental assertion of samples in one solver session (Section 26.9)	Planned, Milestone 2
Typed sketches with selector variables	integer or one-hot selectors, <code>ite</code> , <code>check-sat</code> , <code>get-value</code> ; optional <code>pbeq/at-most</code> cardinality atoms	Planned, Milestone 3
Contract diagnostics and grammar nogoods	<code>check-sat-assuming</code> , <code>:produce-unsat-cores</code> , <code>get-unsat-core</code> , optional <code>smt.core.minimize</code> (Section 27.7)	Planned, Milestone 4
Optimization and canonical models	<code>minimize/maximize</code> , <code>assert-soft</code> , <code>get-objectives</code> ; or iterative bounds with <code>push/pop</code> (Section 29.6)	Planned, Milestone 4
Sound prefix pruning	<code>unsat</code> of an over-approximate summary against samples; cores for nogoods	Planned, staged; only scheduling hints at first
Richer behavioral domains	algebraic datatypes, uninterpreted sorts and functions (EUF), arrays, <code>Seq</code> , bit-vectors	Planned, Milestone 5
Lean proof artifacts	none from Z3; Lean tactics such as <code>simp</code> , <code>omega</code> , <code>bv_decide</code>	Planned, Milestone 6
CHCs for recursive candidates	Spacer through <code>declare-rel/rule/query</code> or <code>(set-logic HORN)</code> with <code>check-sat</code> ; <code>:fp.print_certificate</code> , <code>get-model</code>	Planned, Milestone 7

Counterexample-guided enumeration

25.1 Turning the current bank into generator feedback

The existing filter can learn an input from candidate C_1 and replay it against later candidates $C_2, C_3, \dots$: in `--behavior-mode` filter one nominal bank of source-ordered input vectors is opened per `:synth` command and reused across the run's at most two ordered batches and its later provider and classical lanes (the rank path keeps its older four-entry batch-local MRU). Every retained vector still has to be independently replayed against the later candidate's own checked problem before it counts; the bank stores no verdicts. The next step is to expose that sample set to Djex's search scheduler before candidates reach Lean.

For a sample bank

$$S = \{x_1, \dots, x_k\},$$

a domain can compute a semantic signature

$$\sigma_S(C) = (\text{obs}(C, x_1), \dots, \text{obs}(C, x_k)).$$

Candidates with identical signatures are indistinguishable under the current observations. The scheduler can:

- retain only one representative of each signature in the active frontier;
- postpone duplicates until the representative is refuted;
- prefer signatures satisfying more sample obligations;
- partition a candidate stream without changing the underlying type-search completeness claim;
- ask Z3 for a new input that separates two important signature classes.

When a new counterexample arrives, the signature partition is refined. Previously equivalent candidates may split.

25.2 A candidate-level CEGIS loop

A conservative first CEGIS loop can continue to use ordinary Djex enumeration:

1. enumerate the next bounded type-correct candidate;
2. interpret it on every retained sample before rendering or callback verification when the exact typed graph is available;
3. discard or postpone it only under an explicit sample-consistency policy;
4. render and ask Lean to verify survivors;
5. run the exact behavioral verifier;
6. if replay yields a new counterexample, insert its input into the scoped bank and refine signatures;
7. stop after finding enough survivors or exhausting explicit search, candidate, solver, and wall budgets.

There are two policy choices for sample inconsistency:

Hard sample pruning.

A candidate that independently evaluates to a violation on a replay-authorized sample is omitted before Lean. This is safe relative to the exact domain problem but changes where rejection occurs; the occurrence and reason still need an auditable trace.

Scheduling only.

Inconsistent candidates are moved behind consistent classes but remain enumerable. This preserves the present callback frontier more closely and is useful while semantic applicability is incomplete.

The initial implementation should support both, with scheduling as the compatibility default and hard pruning requiring an explicit behavioral mode; the existing request-scoped `--behavior-mode filter` is the natural switch, since it already carries the authority to omit candidates and already owns the command-local bank.

25.3 Counterexample lifecycle

A counterexample should move through named states:

1. raw solver bindings;
2. bounded, structurally decoded model payload;
3. candidate-specific replayed evidence;
4. source-ordered input vector detached for bank storage;
5. fresh replay against a later candidate;
6. optional deterministic simplification;
7. optional user-visible rendering or generated regression test.

Only the input vector is generally reusable across candidates. Evidence remains tied to the candidate that was replayed. A bank origin label may say “live solver,” “origin probe,” “finite validation,” or “user example,” but the label itself grants no authority.

25.4 Sample selection and minimization

Unboundedly retaining every counterexample can make sample evaluation dominate search. A bank needs a deterministic bounded policy. Useful strategies include:

- newest-first retention with duplicate promotion, as the current Length layer uses (a four-entry batch-local MRU on the rank path; on the filter path a command-local bank whose default limits are four entries, eight naturals per sample, 256 bits per natural, 4096 encoded bytes, and 256 replay attempts, per [synthesis/src/Language/Haskell/Synthesis/Semantic/Length/CounterexampleBank.hs](#));
- smallest-first under a domain-specific lexicographic measure;
- coverage-based retention: keep samples that separate the most current signature classes;
- subsumption: remove a sample whose rejection set is contained in another’s;
- stratification by contract branch or constructor tag;
- protected user examples plus replaceable solver-generated samples.

Every strategy must be deterministic under the same fingerprints and budget. Removing a sample can invalidate a nogood derived from that sample set unless the nogood has an independent proof of monotonicity.

25.5 Distinguishing candidates with Z3

Suppose two candidates C_1 and C_2 both satisfy the current sample bank. A domain can ask for a separating input:

$$P(x) \wedge \text{obs}(C_1, x) \neq \text{obs}(C_2, x).$$

If sat, replay validates the decoded x for both candidates and refines the partition. This is not necessarily a contract counterexample; it is an *equivalence query*. The evidence type and bank origin should therefore be distinct from violation evidence.

For the Length domain, identity and reverse cannot be separated because both have result length n . This correctly tells the scheduler that a richer observation domain is needed, not that the functions are extensionally equal.

Worked equivalence query

Two candidates *can* be separated when their length behavior differs. Suppose the bank holds only $n = 0$, and both an identity-like candidate C_1 (result length n) and a tail-like candidate C_2 (result length $\text{ite}(n = 0, 0, n - 1)$) agree with it. The separating query is:

```
(set-logic QF_LIA)
(declare-const n Int)
(assert (<= 0 n))
; C1: result length n          (identity-like)
(define-fun out1 () Int n)
; C2: result length ite(n=0,0,n-1) (tail-like)
(define-fun out2 () Int (ite (= n 0) 0 (- n 1)))
; both agree on the current bank {n = 0}; ask for a separating input
(assert (not (= out1 out2)))
(check-sat)
(get-value (n))
```

Listing 25.1: An equivalence query separating two candidates that agree on the current bank.

Z3 5.0.0 answers sat with $(n\ 1)$. Replaying $n = 1$ against both candidates gives result lengths 1 and 0, so the two signature classes split. Nothing here says which candidate is right; that is the contract's job, and the input enters the bank under a distinct “equivalence probe” origin rather than as violation evidence.

25.6 Interaction with completeness claims

Djinn may produce a genuine uninhabitation result for a complete logical translation. Exference typically produces bounded search results. Behavioral pruning must not contaminate these claims.

A safe report distinguishes:

- no inhabitant exists in a complete type fragment;
- no candidate was generated within a bounded grammar/search budget;
- candidates existed but every semantically applicable one was refuted;
- some candidates could not be interpreted;
- the solver or validator returned unknown/incomplete results;
- a finite sketch was proved infeasible for the current sample bank;
- no universally correct candidate was established.

“No result” is not one proposition.

25.7 A practical first milestone

The lowest-risk generator feedback is:

1. keep the current verified-candidate filter unchanged as the final authority;
2. add a pure domain evaluation hook over exact typed candidates and existing bank inputs;

3. compute deterministic signatures;
4. schedule one representative per signature before duplicates;
5. record how many candidates were delayed, not silently erased;
6. compare callback count, live-query count, and first-survivor latency on the existing Length corpus.

This delivers measurable CEGIS benefit without yet introducing selector encodings or a new incremental solver protocol.

Typed sketches and solver-selected productions

26.1 What a sketch is

A *sketch* is a type-correct partial program whose holes have finite, explicitly enumerated alternatives. Djex prepares the graph; Z3 chooses alternatives.

A simple hole might be

$$h \in \{x, y, f(x), g(y)\}.$$

A structured sketch can have many nodes, wiring choices, operator selectors, finite literal choices, and optional branches. Every alternative is already known to fit the expected Djex type and binder scope.

In Leant/Djex

This is not “ask Z3 to synthesize Lean.” It is “ask Z3 to solve finite mathematical variables that identify one member of a Djex-checked family of candidates.” Materialization then follows the ordinary exact-origin and Lean callback paths.

26.2 Why selector variables are the first design

Z3 supports algebraic datatypes and recursive syntax encodings, but re-encoding the entire Djex type system inside SMT would duplicate a trusted boundary and handle rank-N terms poorly. The first sketch design should use selectors over a Djex-owned DAG:

- one finite-domain selector s_i per choice node;
- one alternative table owned by Djex;
- optional Boolean reachability variables for dormant subgraphs;
- arithmetic variables only for explicitly admitted literals or coefficients;
- semantic equations generated from the selected alternatives;
- exact fingerprints of grammar, alternative order, types, inventory, and samples.

Typing is true by construction because every alternative was admitted at the node’s expected type. Z3 solves consistency and behavior, not typing.

26.3 Exactly-one encodings

For a node with alternatives $0, \dots, m - 1$, an integer selector can use

$$0 \leq s_i < m.$$

A Boolean one-hot encoding uses $b_{i,0}, \dots, b_{i,m-1}$ with exactly one true. Integer selectors are compact and work naturally with `ite`; one-hot selectors make assumption cores and cardinality constraints transparent. The representation is part of the sketch schema.

For three alternatives:

```
; Integer selector
(declare-const s Int)
(assert (and (<= 0 s) (< s 3)))

; One-hot selector
(declare-const choose.0 Bool)
(declare-const choose.1 Bool)
(declare-const choose.2 Bool)
(assert (or choose.0 choose.1 choose.2))
(assert (not (and choose.0 choose.1)))
(assert (not (and choose.0 choose.2)))
(assert (not (and choose.1 choose.2)))
```

Listing 26.1: Two ways to encode one finite production choice.

Run as written, Z3 5.0.0 answers `sat` and `(get-value (s choose.0 choose.1 choose.2))` prints `((s 0) (choose.0 true) (choose.1 false) (choose.2 false))`: with no semantic constraint the solver simply picks the first alternative in both encodings, which is one reason later sections insist on canonical tie-breaking. Z3 also offers native cardinality atoms for the one-hot form, for example `(assert ((_ pbeq 1 1 1 1) choose.0 choose.1 choose.2))` (“exactly one of the three, each with weight one”) or `((_ at-most 1) ...)` together with the disjunction; adding `(assert (not choose.0))` to the `pbeq` form yields `((choose.0 false) (choose.1 false) (choose.2 true))`.

A production assumption literal can imply the corresponding choice. During a candidate query, `check-sat-assuming` can activate a partial assignment and expose an unsat core over those assumptions, provided `(set-option :produce-unsat-cores true)` was issued first; without it Z3 5.0.0 answers the `get-unsat-core` with the error “unsat core construction is not enabled”. Section 27.7 runs such a query.

26.4 Semantic equations over samples

For each retained sample x_j , the domain creates semantic variables for reachable sketch nodes. If node i has alternatives with sample semantics $e_{i,0,j}, \dots, e_{i,m-1,j}$, then

$$v_{i,j} = \text{ite}(s_i = 0, e_{i,0,j}, \text{ite}(s_i = 1, e_{i,1,j}, \dots)).$$

The root value must satisfy the contract on every sample:

$$\bigwedge_j (P(x_j) \Rightarrow Q(x_j, v_{root,j})).$$

The same selector is shared across samples; only semantic values vary. Thus Z3 searches one program consistent with all examples.

26.5 Worked sketch: `Option.bind`

Consider the target

$$\text{Option } \alpha \rightarrow (\alpha \rightarrow \text{Option } \beta) \rightarrow \text{Option } \beta.$$

A tiny Djex-prepared sketch for the some branch might choose among:

1. return `none`;
2. return the callback result `k x`;

3. return a separately available opaque Option beta provider value, if the inventory contains one.

The none branch may choose between none and a provider value. Two finite provenance samples suffice to force the ordinary bind wiring:

- input tag none; expected output tag none;
- input tag some with atom a_0 ; callback table maps a_0 to a distinctive token t_0 ; expected output is t_0 .

A schematic SMT encoding is:

```
(set-logic QF_LIA)

; 0 = none, 1 = provider value
(declare-const none.branch Int)
(assert (and (<= 0 none.branch) (< none.branch 2)))

; 0 = none, 1 = callback result, 2 = provider value
(declare-const some.branch Int)
(assert (and (<= 0 some.branch) (< some.branch 3)))

; Finite symbolic tokens: 0 = none, 1 = callback token, 2 = provider token
(define-fun out.none.sample () Int
  (ite (= none.branch 0) 0 2))
(define-fun out.some.sample () Int
  (ite (= some.branch 0) 0
    (ite (= some.branch 1) 1 2)))

(assert (= out.none.sample 0))
(assert (= out.some.sample 1))
(check-sat)
(get-value (none.branch some.branch))
```

Listing 26.2: Schematic selector problem for an Option-bind sketch.

Run through Z3 5.0.0 the script answers sat and prints ((none.branch 0) (some.branch 1)): the model chooses (0, 1), which is the only assignment consistent with both samples (the none sample rules out the provider token in the none branch, and the some sample rules out both none and the provider token in the some branch). Djex decodes those selector indices against the exact alternative tables, materializes the case expression, and asks Lean to elaborate it. The behavioral verifier then checks the materialized exact candidate, rather than trusting the sample equations alone.

26.6 Worked sketch: state threading

For

$$(S \rightarrow A \times S) \rightarrow (A \rightarrow S \rightarrow B \times S) \rightarrow S \rightarrow B \times S,$$

a critical choice is whether the second callback receives the original state s_0 or the intermediate state s_1 . Djex can prepare a choice node

$$s_{arg} \in \{s_0, s_1\}.$$

A relational domain models F and G as uninterpreted total functions and the candidate as

$$G(\pi_1(F(s_0)), s_{arg}).$$

The contract requires

$$G(\pi_1(F(s_0)), \pi_2(F(s_0))).$$

A separating model can make $s_0 \neq s_1$ and assign different outputs to the two G applications. The selector consistent with all such counterexamples is $s_{arg} = s_1$. Concretely, the proposal’s verification query for the wrong wiring ([10], appendix “Concrete SMT-LIB encodings”) answers sat with s_0 and s_1 distinct. Turning that observation into a sample constraint and adding the selector gives the following script, which Z3 5.0.0 answers with sat and ((use.s1 true)):

```

(set-logic ALL)
(declare-sort S 0)
(declare-sort A 0)
(declare-sort B 0)
(declare-datatypes ((Pair 2))
  ((par (X Y) ((mk-pair (fst X) (snd Y))))))
(declare-fun f (S) (Pair A S))
(declare-fun g (A S) (Pair B S))
(declare-const s0 S)
(define-fun a0 () A (fst (f s0)))
(define-fun s1 () S (snd (f s0)))
(define-fun expected () (Pair B S) (g a0 s1))

; selector: false = pass s0 to g, true = pass s1
(declare-const use.s1 Bool)
(define-fun actual () (Pair B S) (g a0 (ite use.s1 s1 s0)))

; sample learned from the wrong wiring's counterexample model:
; there, the two applications of g differ
(assert (not (= (g a0 s0) (g a0 s1))))
(assert (= actual expected))
(check-sat)
(get-value (use.s1))

```

Listing 26.3: Selector version of the state-threading sketch over uninterpreted sorts and a tuple datatype.

The disequality is the sample: it records the finite interpretation in which the two G applications differ, and it forces the only wiring that agrees with the expected result. Djex materializes the alternative selected by `use.s1`; Lean elaborates it; the domain's ordinary verification query then confirms that no interpretation separates `actual` from `expected`.

This example shows why future domains need uninterpreted sorts/functions and algebraic tuples even when the synthesis selector itself is finite.

26.7 Model decoding and materialization

A sketch model is decoded under a stricter policy than an arbitrary Z3 model:

- every expected selector must be present exactly once;
- every value must be within the declared alternative range;
- inactive-node selectors must satisfy the chosen canonicalization policy;
- integer literals must satisfy width/range constraints;
- finite tables must contain exactly the expected keys;
- no unknown model declaration is accepted as syntax;
- the selector vector is associated with the exact sketch fingerprint.

Djex then walks the prepared DAG and replaces each hole with the selected alternative. The materialized graph receives a fresh exact typed origin. Lean callback verification and full behavioral checking run as usual.

26.8 Why samples do not finish verification

A sketch model proves only that the selected candidate satisfies the current finite sample constraints under the sketch encoding. It may fail on another input. Therefore a CEGIS iteration is:

1. solve the sketch under samples;

2. materialize and Lean-check the candidate;
3. ask the verification encoding for a counterexample over the full modeled domain;
4. if sat and replayed, add the input and solve again;
5. if no counterexample is established, either emit the best available status, run a complete domain validator, or generate a Lean proof obligation;
6. stop under explicit iteration, model, solver, and wall budgets.

26.9 Worked example: two rounds of sketch CEGIS on lengths

The loop is easiest to see on the smallest possible sketch. Take the target `List a → List a`, the contract “result length equals input length”, and one Djex-prepared choice node with three alternatives: `return []` (result length 0), `cons one element onto the input` (result length $n + 1$), or `return the input` (result length n). Every alternative already has the right type; Z3 only chooses. All four scripts below were run as written against Z3 5.0.0, each in a fresh process, and the answers are quoted after them.

Round 1, sketch query. The bank is empty, so only the range constraint applies:

```
(set-logic QF_LIA)
; one choice node; alternatives prepared by Djex at type List a
; 0: []          result length 0
; 1: x :: xs     result length n + 1
; 2: xs         result length n
(declare-const s Int)
(assert (and (<= 0 s) (< s 3)))
(define-fun out ((n Int)) Int
  (ite (= s 0) 0
    (ite (= s 1) (+ n 1)
      n)))
; round 1: the bank is empty, so only the range constraint applies
(check-sat)
(get-value (s))
```

Listing 26.4: Round 1: choose an alternative with no samples.

Answer: `sat, ((s 0))`. Z3 picks the constant-empty alternative. Djex materializes `fun xs => []`, Lean accepts it, and the domain now asks whether it violates the contract.

Round 1, verification query. The materialized candidate has result length 0, so the bad state is $n \geq 0 \wedge 0 \neq n$:

```
(set-logic QF_LIA)
(declare-const n Int)
(assert (<= 0 n))
; alternative 0 materialized: result length 0
(define-fun out () Int 0)
; bad state: the contract "result length = input length" fails
(assert (not (= out n)))
(check-sat)
(get-value (n))
```

Listing 26.5: Round 1: refute the materialized candidate.

Answer: `sat, ((n 1))`. Independent replay evaluates the exact candidate at input length one, obtains result length zero, confirms $0 \neq 1$, and inserts the input vector (1) into the bank.

Round 2, sketch query. The same declarations are kept and the sample is asserted as one more equation over the shared selector:

```
(set-logic QF_LIA)
(declare-const s Int)
(assert (and (<= 0 s) (< s 3)))
(define-fun out ((n Int)) Int
  (ite (= s 0) 0
```

```
(ite (= s 1) (+ n 1)
      n)))
; round 2: the replayed sample n = 1 now constrains every alternative
(assert (= (out 1) 1))
(check-sat)
(get-value (s))
```

Listing 26.6: Round 2: the replayed sample constrains every alternative at once.

Answer: sat, ((s 2)). One sample eliminated two of the three alternatives ($0 \neq 1$ and $2 \neq 1$), and Z3 selects the identity alternative. In an incremental session the second script is just the extra `assert` and `check-sat` appended to the first one; we confirmed that the same session answers ((s 0)) and then ((s 2)), and still ((s 2)) after a further sample (`assert (= (out 0) 0)`).

Round 2, verification query. The materialized candidate is `fun xs => xs` with result length `n`:

```
(set-logic QF_LIA)
(declare-const n Int)
(assert (<= 0 n))
; alternative 2 materialized: result length n
(define-fun out () Int n)
(assert (not (= out n)))
(check-sat)
```

Listing 26.7: Round 2: the verification query for the identity alternative.

Answer: unsat. The loop stops. Note what has and has not been established: the selector model proved that alternative 2 agrees with the one retained sample under the sketch encoding; Lean established that the materialized term is well typed at the goal; the final `unsat` is a heuristic solver observation that no length-model counterexample exists for that exact candidate. In the current authority vocabulary the candidate is “solver-indisproved in model”, not proved; a complete finite validator or a Lean theorem is needed for more. Two rounds sufficed here because the sketch was tiny; the same protocol scales to many nodes, but the three kinds of answer keep their distinct meanings.

26.10 Unsat means different things in synthesis and verification

If the sketch problem is `unsat` for the current sample bank, no selector assignment in that exact finite grammar satisfies those samples according to the encoding. This can justify enlarging the grammar, increasing the size bound, or reporting sample-level infeasibility. It does not prove that no Lean program of the target type satisfies the contract.

If the verification counterexample query is `unsat`, the candidate has no bad input in the encoded model according to that solver run. This is a different formula, different query schema, and different result claim. The two `unsat` statuses must never share one generic “proved” flag.

Source trail

The detailed design motivating this chapter is the existing [docs/Z3_Behavioral_Synthesis_Proposal/Z3_Behavioral_Synthesis_Proposal.tex](#). Current Djex typed candidates, inventories, and fingerprints provide the raw material for sketches; the new sketch/query/session APIs should be additive rather than overloading the Length live protocol.

Sound pruning of incomplete search states

27.1 Why pruning is harder than checking a complete candidate

A complete candidate has one exact modeled behavior in a domain. An incomplete search state denotes a set of possible completions. To prune that state soundly, the system must establish that *every* completion violates the contract or cannot satisfy the current samples.

Testing one convenient completion is not enough. Nor is observing that every completion generated so far failed. The summary used for pruning must have a formal relationship to the completion set.

27.2 Over- and under-approximation

Let $C(p)$ be the set of completions of prefix p , and let $B(C, x)$ be the observable behavior of candidate C at input x .

An *over-approximation* $O(p, x)$ contains every behavior of every completion:

$$\{B(C, x) \mid C \in C(p)\} \subseteq O(p, x).$$

If even the over-approximation cannot intersect the contract's allowed outputs, then no completion can work and the prefix may be pruned.

An *under-approximation* $U(p, x)$ contains only behaviors known to be realizable:

$$U(p, x) \subseteq \{B(C, x) \mid C \in C(p)\}.$$

It can produce witnesses or promising directions, but its failure to satisfy a contract says nothing about other completions.

Mental model

Over-approximation is safe for proving impossibility; under-approximation is safe for finding examples. Reversing those roles is a classic synthesis bug.

27.3 A Length interval summary

Suppose a partial Length expression contains a hole whose allowed completions, for one sample input, can return any length in interval $[L, U]$. If the contract requires exactly n and $n \notin [L, U]$, the prefix is impossible for that sample.

For symbolic inputs, summaries can be affine bounds or finite unions of polyhedra. For example, a hole built only from:

- the input length n ;
- zero and successor;
- append-like addition;

- at most two constructor allocations;

may have an over-approximation

$$0 \leq r \leq 2n + 2.$$

This is too broad to prove length preservation impossible. If a prefix has already committed to a constant-empty root and no reachable hole can alter the result, the exact summary $r = 0$ refutes preservation at $n = 1$. The proposal’s worked prefix ([10], Level 4) is the smallest honest example. Suppose Exference is at the partial state `fun xs => List.cons ?h1 ?h2` with `?h2 : List a`, and the bank holds the empty list, $n = 0$. The Length domain knows nothing about the eventual completion of `?h2` except its type, so its sound summary is “some length $t \geq 0$ ”, and the prefix summary is $r = 1 + t$. The pruning query on the bank is:

```
(set-logic QF_LIA)
; hole ?h2 : List a may complete to any length t >= 0
(declare-const t Int)
(assert (>= t 0))
; prefix fun xs => List.cons ?h1 ?h2 has result length 1 + t;
; the retained sample n = 0 requires result length 0
(assert (= (+ 1 t) 0))
(check-sat)
```

Listing 27.1: An over-approximate prefix summary refuted by one sample.

Z3 5.0.0 answers `unsat`: every completion returns a nonempty list on the empty input, so the whole subtree can be pruned, and the receipt records the summary $t \geq 0$, the exact bank identity, and the grammar fingerprint. Had the domain summarized `?h2` by the completions it happened to enumerate first (say “one of the locals”, giving $r = 1 + n$), the query for a length-one sample would also be `unsat`, yet the completion `?h2 := []` satisfies that sample; that is the polarity error the mental model above warns about.

The useful summary is often structural rather than purely numeric: which holes can still influence the root, which inputs can still flow to them, and which provider transfers remain selectable.

27.4 Abstract interpretation of a typed grammar

A domain-specific abstract interpreter can compute summaries bottom-up over a partial Djex graph. The abstract domain might contain:

- intervals or affine forms for scalar/length values;
- finite sets of constructor tags;
- provenance sets for opaque atoms;
- may-call/must-call callback relations;
- bag-count lower and upper bounds;
- possible sequence index mappings;
- bit masks of definitely-zero/definitely-one bits;
- cost lower and upper bounds;
- finite transition relations for state machines.

Every abstract transfer requires a soundness argument relative to the concrete domain semantics. Provider summaries are explicit contracts, just as current Length transfers are. A missing summary yields top, not an invented precise value.

27.5 Encoding a pruning query

Let $A_p(\vec{x}, \vec{y})$ describe an over-approximation of possible outputs $\vec{y}$ for prefix p . Let the behavioral contract be $P(\vec{x}) \Rightarrow Q(\vec{x}, \vec{y})$. To show that no completion can satisfy the contract on a retained sample $\vec{a}$, ask whether

$$A_p(\vec{a}, \vec{y}) \wedge Q(\vec{a}, \vec{y})$$

is satisfiable. If `unsat` and the abstraction relation itself is trusted, the prefix has no completion satisfying that sample.

For universal pruning over the whole modeled input domain, the formula is subtler. One seeks

$$\exists \text{completion behavior satisfying the contract for every input,}$$

which may introduce quantifiers or require a template. Early implementations should prune only against replay-authorized finite samples or domain-specific decidable summaries.

27.6 Pruning receipts

A prefix-pruning decision needs its own opaque receipt binding:

- the exact partial graph and hole set;
- the completion grammar fingerprint;
- the abstract-domain schema and transfer table;
- every provider summary used;
- the sample or finite scope;
- the exact query and solver execution identity;
- any independent validation of the abstraction result.

Unlike a candidate counterexample, the evidence may be positive infeasibility of an abstract intersection. If raw `unsat` remains heuristic, the first production mode can use pruning only as a search-order hint. Hard pruning should require either an independently checkable finite abstraction or a generated Lean proof of the abstract exclusion.

27.7 Nogoods over production choices

When a partial selector assignment cannot satisfy the current samples, Z3 can return an `unsat` core over assumption literals representing those choices. If the core is

$$\{a_{i_1}, \dots, a_{i_k}\},$$

the learned nogood is

$$\neg(a_{i_1} \wedge \dots \wedge a_{i_k}).$$

This forbids every future model containing the same conflicting choice combination.

A core is not necessarily minimal. A deterministic deletion pass can attempt to remove assumptions while preserving unsatisfiability under a separate budget. The resulting nogood remains scoped to the exact grammar and sample set.

Worked nogood extraction

The following two-node sketch shows both the mechanism and the non-minimality. Node A chooses the base value (constant zero or the input length n), node B chooses an increment ($+0$, $+1$, or $+2$), the root is their sum, and two retained samples require lengths 1 and 3 to be preserved. Each production choice gets an assumption literal, and three partial assignments are probed with `check-sat-assuming`:

```
(set-option :produce-unsat-cores true)
(set-logic QF_LIA)
; node A, base value:  0 = constant zero, 1 = input length n
(declare-const sA Int)
(assert (and (<= 0 sA) (< sA 2)))
; node B, increment:  0 = +0, 1 = +1, 2 = +2
(declare-const sB Int)
(assert (and (<= 0 sB) (< sB 3)))
(define-fun out ((n Int)) Int
```

```

(+ (ite (= sA 0) 0 n)
  (ite (= sB 0) 0 (ite (= sB 1) 1 2)))
; retained samples: lengths 1 and 3 must be preserved
(assert (= (out 1) 1))
(assert (= (out 3) 3))
; one assumption literal per production choice
(declare-const pA0 Bool) (declare-const pA1 Bool)
(declare-const pB0 Bool) (declare-const pB1 Bool) (declare-const pB2 Bool)
(assert (=> pA0 (= sA 0))) (assert (=> pA1 (= sA 1)))
(assert (=> pB0 (= sB 0))) (assert (=> pB1 (= sB 1))) (assert (=> pB2 (= sB 2)))
; partial assignment: constant-zero base, increment +1
(check-sat-assuming (pA0 pB1))
(get-unsat-core)
; partial assignment: input-length base, increment +1
(check-sat-assuming (pA1 pB1))
(get-unsat-core)
; partial assignment: input-length base, increment +0
(check-sat-assuming (pA1 pB0))
(get-value (sA sB))

```

Listing 27.2: Assumption literals per production choice, and cores over them.

Z3 5.0.0 answers:

```

unsat
(pA0 pB1)
unsat
(pA1 pB1)
sat
((sA 1)
 (sB 0))

```

The first core is not minimal: a constant-zero base can never reach length 3 with an increment of at most 2, so `pA0` alone is already infeasible, yet Z3 blamed both literals. Inserting `(set-option :smt.core.minimize true)` before the logic declaration and rerunning in a fresh process gives the cores `(pA0)` and `(pB1)` instead: the first now says “no constant-zero base, whatever the increment”, and the second says “no +1 increment, whatever the base” (with base 0 it fails length 3; with base n it fails length 1). Both smaller nogoods are correct and prune more of the remaining grammar, which is why a deterministic minimization pass, or the solver’s own minimizer under a fingerprinted option, is worth its budget. Every one of these nogoods is valid only for the exact grammar and the exact two-sample bank that produced it.

27.8 Monotonicity rules for reuse

A nogood derived solely from hard grammar constraints and retained samples remains valid as more samples are added, because the formula only becomes stronger. It may become invalid if:

- a sample used in the derivation is removed;
- the grammar gains a new alternative that changes a node’s semantics;
- the size bound changes the meaning of reachability variables;
- provider laws or exactness profiles change;
- a finite-domain encoding changes atom or index identities;
- an objective was accidentally mixed into a hard infeasibility claim.

The safest cache key includes the complete sketch fingerprint plus a monotone sample-bank lineage, not merely a textual list of core literals.

27.9 A staged implementation

Sound prefix pruning should arrive in stages:

1. sample-only exact evaluation of fully determined subgraphs;
2. exact finite choice propagation within a small sketch;
3. domain-specific abstract summaries used only for scheduling;
4. finite-sample hard pruning with independently validated summaries;
5. core-derived nogoods under an incremental profile;
6. unbounded pruning only where the abstraction has a mechanized or Lean-checked soundness theorem.

This order avoids making search completeness depend early on an unverified abstract interpreter.

Richer behavioral domains and the Z3 theories they need

28.1 Domain design comes before solver selection

A common mistake is to begin with a Z3 theory and ask what source behavior can be squeezed into it. Leant/Djex should instead define a precise observation domain:

1. which Lean values and candidate constructs are modeled;
2. what information is observed and forgotten;
3. which provider laws are assumed or proved;
4. how a model is decoded;
5. how independent replay works;
6. what exactness claim a positive or negative result carries;
7. only then, which SMT theory and protocol profile implement the queries.

The following domains are natural extensions of the Length architecture.

28.2 Generalized size and constructor tags

A first extension observes several recursive datatypes through:

- size, height, or constructor count;
- root constructor tag;
- a finite vector of field sizes;
- selected branch reachability facts;
- product/sum component observations.

This reuses QF-LIA for numeric measures and finite integers or algebraic datatypes for tags. It can distinguish functions that preserve list length but change an option/tree tag, and it supports tuples of observations without immediately modeling elements.

Examples include:

$$\begin{aligned}
 \text{size}(\text{map}(f, t)) &= \text{size}(t), \\
 \text{height}(\text{mirror}(t)) &= \text{height}(t), \\
 \text{tag}(\text{Option.bind}(x, k)) &= \begin{cases} \text{none}, & x = \text{none}, \\ \text{tag}(k(a)), & x = \text{some}(a). \end{cases}
 \end{aligned}$$

The main new work is a generic structural-model declaration rather than assuming one zero/step spine.

28.3 Sequences, indices, and order

To distinguish identity, reverse, rotation, map, filter, and arbitrary permutations, the model must observe element positions. Two useful encodings are:

Native sequences.

Use Z3's sequence sort, `seq.len`, concatenation, extraction, and guarded `seq.nth`. Every index read must preserve an explicit bounds guard because out-of-range `seq.nth` is not a useful source-level value.

Array plus length.

Represent a sequence by (n, a) with $n \geq 0$ and $a : \mathbb{Z} \rightarrow E$. Every read is guarded by $0 \leq i < n$. This makes a symbolic counterexample index natural and combines well with bags and finite atoms.

A reverse contract can search for one violating index:

$$0 \leq i < n \wedge r[i] \neq x[n-1-i].$$

This replaces a universal property by an existential bad index, making counterexample search quantifier-free when candidate semantics and array theory permit it. As a first taste of the sequence theory, here is the reverse contract checked against the identity candidate with native sequences; the candidate's output is the input itself, so r is just x :

```
(declare-const x (Seq Int))
(declare-const i Int)
(define-fun n () Int (seq.len x))
; candidate: identity, so r = x; contract: r[i] = x[n-1-i]
(assert (and (<= 0 i) (< i n)))
(assert (not (= (seq.nth x i) (seq.nth x (- (- n 1) i)))))
(check-sat)
(get-value (x i))
```

Listing 28.1: One bad index refutes identity against the reverse contract.

Z3 5.0.0 answers sat with `((x (seq.++ (seq.unit 4) (seq.unit 7))) (i 1))`: the two-element sequence `[4, 7]` and index 1, where the identity returns 7 but reversal would return 4. The element values 4 and 7 are arbitrary; a domain decoder would map them to two distinct finite atoms and replay the index comparison, never trusting the literal integers as Lean elements. The bounds guard on `i` is essential: without it Z3 could satisfy the disequality with an out-of-range `seq.nth`, whose value is unconstrained.

Replay can use finite symbolic atoms rather than arbitrary Lean elements. The receipt must state whether the atom model is exact for the claimed property. Parametricity can sometimes justify a small-model theorem, but that theorem belongs in Lean or a separately checked domain argument.

28.4 Bags and permutation

Bag preservation can be expressed with a symbolic element e :

$$\text{count}(r, e) = \text{count}(x, e).$$

A counterexample query existentially chooses both input structure and e . For bounded sequences, finite atom counts can be exact. For unbounded lists, one can combine:

- finite maps from atoms to multiplicities;
- arrays of integer counts;
- algebraic datatypes and recursive count functions for bounded unfolding;

- CHCs for recursive count relations;
- provider-level permutation laws linked to Lean theorems.

Length plus bag preservation characterizes permutation in the mathematical list model, but order still requires a sequence relation. The domain should not call bag preservation “same list.”

28.5 Scalar arithmetic

A scalar domain can model natural, integer, rational, or bounded real-valued behavior. Early exact fragments include:

- QF-LIA affine and piecewise-linear integer functions;
- QF-LRA linear real/rational templates;
- finite ranges and branch thresholds;
- coefficient synthesis in affine forms;
- min/max, saturation, clipping, and monotonic piecewise templates;
- relational properties using two symbolic calls.

For an affine sketch $f(x) = ax + b$, samples constrain a, b , and a verification query asks

$$P(x) \wedge ax + b \neq T(x).$$

If a, b are bounded integers, the synthesis problem remains decidable and predictable. Nonlinear integer arithmetic and polynomial templates can be useful but require separate profiles because completeness and performance degrade sharply.

Relational properties such as monotonicity duplicate symbolic evaluation:

$$x_1 \leq x_2 \wedge f(x_1) > f(x_2).$$

Providers must be pure/extensional in the model before their calls can be shared or duplicated this way.

28.6 Bit-vectors and machine arithmetic

Z3 bit-vectors model fixed-width wraparound exactly. A domain can synthesize:

- masks and constants;
- shifts, rotates, packing, and extraction;
- branchless min/max/absolute-value idioms;
- saturating addition/subtraction;
- overflow predicates;
- parity, population-count, and byte-level templates.

The checked contract records width, signedness, comparison operators, shift policy, division-by-zero semantics, and Lean coercions. A selector model chooses operators and constants in `QF_BV`. The materialized Lean theorem can often be checked by `bv_decide` or a bit-vector normalization tactic, making this a strong candidate for early proof-producing synthesis.

28.7 State and provenance

A state/provenance domain treats arbitrary source types as uninterpreted sorts and callbacks as uninterpreted functions or explicit call events. It can verify wiring properties without knowing concrete values:

- pass the intermediate state rather than the initial state;

- preserve a reader environment;
- append writer logs in the correct order;
- satisfy lens get-put and put-get equations;
- thread parser position or transaction state;
- connect outputs to the correct callback arguments.

The free-term or EUF model distinguishes provenance. If two symbolic terms are syntactically different, Z3 can often choose an interpretation making them unequal. Replay reconstructs the finite uninterpreted model or evaluates the same provenance algebra; it does not need concrete Lean inhabitants of every quantified type.

The exactness claim includes assumptions of purity, totality, and extensionality for callback symbols. Effects, divergence, and exceptions are not silently erased.

28.8 Finite callbacks and higher-order observations

Arbitrary higher-order extensional equivalence is out of scope, but several finite models are useful:

- an uninterpreted function over first-order modeled sorts;
- a finite table over sample atoms;
- a trace of call arguments and returned tokens;
- an explicit algebraic law supplied with a provider;
- a first-order relation in a CHC system.

For `List.map`, a finite atom model can require that output index i contains $G(x[i])$. A candidate applying G to the wrong element or reversing the result is refutable. A function-valued output remains harder; initial domains should observe it only at finitely many test inputs.

28.9 Cost and resource models

A cost domain associates symbolic nonnegative cost with term-graph nodes and provider calls. It can optimize or constrain:

- AST size and weighted production cost;
- number of provider calls, branches, or constructor allocations;
- maximum recursion depth under a recurrence;
- affine time/memory estimates;
- peak intermediate size;
- duplicated versus shared graph nodes.

This is a modeled cost, not automatically Lean runtime. The contract must state evaluation strategy and sharing assumptions. A proof-grade cost theorem requires a Lean formalization of the cost semantics or instrumentation connecting source execution to the model.

28.10 Finite-state protocols

A candidate may implement a transition function or protocol controller. Z3 arrays, datatypes, and bit-vectors can encode finite states, events, and transition tables. Contracts can require:

- forbidden states are unreachable within a bounded horizon;
- every request eventually receives one of a finite responses under a bounded fairness model;

- transition tables preserve an invariant;
- a synthesized guard or action selector agrees with examples;
- two finite implementations are observationally equivalent.

Bounded model checking is a natural first profile. Unbounded safety can later move to CHCs.

28.11 Domain-priority order

A technically efficient order is:

1. generalized size/tag observations;
2. exact state/provenance wiring;
3. sequence/index observations over bounded atoms;
4. scalar linear arithmetic and bit-vectors;
5. bag/permutation reasoning;
6. finite-state protocols and modeled cost;
7. recursive CHC domains;
8. broader higher-order observations.

The first four maximize reuse of current typed graph interpretation, bounded model decoding, and solver protocol work while adding behaviors that types alone commonly leave ambiguous.

Incremental solving, cores, optimization, and explanations

29.1 Why the current Length protocol should not be stretched

The Length worker has a deliberately narrow reset/check/value transaction. A sketch/CEGIS session needs persistent declarations, growing sample constraints, temporary choice assumptions, optional cores, and objectives. Adding these informally to the current protocol would weaken its state machine and fingerprints.

The planned design should introduce a separate capability profile with a typed incremental state. Its base might contain:

- sketch selector declarations;
- grammar well-formedness constraints;
- stable semantic equations independent of samples;
- provider/domain axioms;
- objective definitions;
- deterministic response options.

Each sample extends the persistent base. Each candidate or partial choice enters through assumptions or a push/pop scope.

29.2 Assumptions versus push/pop

Both mechanisms support temporary constraints.

Push/pop.

Natural for adding a block of temporary formulas. The state machine must prove balanced scopes even on cancellation and protocol failure.

Check with assumptions.

Keeps declarations and clauses stable while activating Boolean literals. It supports unsat cores directly and avoids deep nested scopes. Assumption order and literal identity become part of the query fingerprint.

A likely design uses assumptions for production choices and tracked contract clauses, with push/pop reserved for larger ephemeral query families.

29.3 Contract diagnostics with unsat cores

Before searching candidates, the system can check whether hard contract clauses and finite-domain axioms are mutually consistent. Each clause is guarded by a stable assumption literal:

```
(set-option :produce-unsat-cores true)
(set-logic QF_LIA)
(declare-const n Int)
(declare-const r Int)
(declare-const c.pre Bool)
(declare-const c.length Bool)
(declare-const c.pointwise Bool)

; each contract clause is guarded by a stable assumption literal
(assert (=> c.pre      (<= 0 n)))
(assert (=> c.length   (= r n)))
(assert (=> c.pointwise (= r (+ n 1))))

(check-sat-assuming (c.pre c.length c.pointwise))
(get-unsat-core)
; drop one blamed clause and the remaining contract is satisfiable
(check-sat-assuming (c.pre c.length))
(get-value (n r))
```

Listing 29.1: Tracked contract clauses. A contract author asked for length preservation and, by mistake, also for a pointwise clause that forces one extra element.

Z3 5.0.0 answers:

```
unsat
(c.length c.pointwise)
sat
((n 0)
 (r 0))
```

The core names exactly the two clauses that contradict each other and leaves the harmless precondition literal `c.pre` out, so Leant can point at the two contract spans instead of saying “the contract is inconsistent”. A returned core identifies a conflicting subset, not necessarily a minimal one, and the order in which Z3 prints the literals is not stable (`smt.core.minimize` changed it in our runs), so the decoder must canonicalize the set. Leant can map stable literals back to source spans and run a deterministic deletion minimizer under a separate budget. The diagnostic should say “inconsistent in this checked finite/model profile,” not “logically inconsistent in Lean” unless Lean checks the contradiction.

29.4 Grammar cores and learned nogoods

Production choices can also be assumptions. If one partial sketch is infeasible, a core yields a nogood that blocks all supersets of the conflicting choices. A learned clause database should retain:

- exact core literals in canonical order;
- whether minimization was attempted and completed;
- grammar and sample-bank fingerprints;
- solver execution identity;
- derivation status: raw core, minimized core, or independently validated finite conflict;
- hit count and eviction metadata for bounded caching.

Nogoods are search accelerators. Materialized candidates still cross every ordinary verification boundary.

29.5 Optimization

After hard sample constraints hold, Z3 can choose a preferred sketch. Potential objectives include:

- minimize reachable AST nodes;
- minimize weighted provider cost;
- minimize branches, duplicate calls, or recursion depth;
- minimize literal magnitude;
- maximize satisfied weighted soft examples;
- lexicographically prefer exact/proved provider laws over assumed ones;
- minimize a symbolic resource measure.

Two strategies are useful.

29.5.1 Iterative satisfiability

Add a bound $cost \leq k$, solve, and tighten. Binary search or monotone linear tightening can find the optimum. This reuses ordinary SAT/SMT status and assumptions, produces clear progress, and often interacts better with cores.

29.5.2 Native Optimize

Use minimize, maximize, and soft assertions. This is concise and supports lexicographic, Pareto, or boxed priorities. The exact objective order, weights, priority mode, and model extraction rules must be fingerprinted and tested across the pinned Z3 build.

Common mistake

An optimal model is optimal only within the encoded finite grammar and objective semantics. It says nothing about programs outside the grammar or costs omitted from the model. Optimization never replaces behavioral verification.

29.6 Canonical models

Even with a fixed seed, Z3 may choose different satisfying selector assignments after a solver upgrade. To make materialization stable, the system can impose a canonical lexicographic choice:

- minimize total cost;
- then minimize selector s_0 ;
- then s_1 , and so on;
- constrain inactive selectors to a fixed default;
- canonicalize finite tables and uninterpreted representatives where possible.

Alternatively, after finding a cost optimum, ordinary SAT queries can pin each selector greedily to the smallest feasible value. This is slower but gives an explicit deterministic algorithm independent of Optimize's model preferences.

Worked optimization

A root with three alternatives, the input, a provider identity applied to the input, and the constant [], has two selector values consistent with the sample $n = 1 \mapsto 1$. A modeled cost of one node for the input, two for the provider call, and one for [] breaks the tie, and a second objective on the selector itself fixes the canonical choice:

```
(set-logic QF_LIA)
; root: 0 = the input, 1 = a provider identity applied to the input, 2 = []
(declare-const s Int)
(assert (and (<= 0 s) (< s 3)))
(define-fun out ((n Int)) Int
  (ite (= s 0) n (ite (= s 1) n 0)))
```

```
(assert (= (out 1) 1))
; modeled cost: one node for the input, two for the provider call, one for []
(define-fun cost () Int
  (ite (= s 0) 1 (ite (= s 1) 2 1)))
(minimize cost)
(minimize s)
(check-sat)
(get-value (s cost))
(get-objectives)
```

Listing 29.2: Native Optimize with two lexicographic objectives.

Z3 5.0.0 answers:

```
sat
((s 0)
 (cost 1))
(objectives
 ((ite (= s 0) 1 (ite (= s 1) 2 1)) 1)
 (s 0)
 )
```

Note that `get-objectives` prints the expanded body of `cost`, not its name, and the objective values in declaration order; a decoder must key on position, not text. The iterative alternative uses only ordinary satisfiability: `assert (<= cost 1)` inside a `push/pop` scope and `check (sat, ((s 0) (cost 1)))`, then `(<= cost 0)` in a fresh scope (`unsat`), so `cost 1` is optimal and the smallest selector value with that cost is then pinned by one more query. Both procedures were run against Z3 5.0.0 and agree; the iterative one is slower per optimum but reuses the plain state machine, assumptions, and cores.

29.7 Session rebuild and recovery

An incremental worker can become unusable after malformed output, deadline, cancellation, or a failed `pop/reset`. The robust response is:

1. close and clean the worker;
2. open and capability-probe a fresh worker;
3. replay the canonical base declarations;
4. replay retained samples in canonical order;
5. replay still-valid learned hard clauses;
6. resume from the last externally committed state.

The authoritative session state lives in the parent as typed persistent data. Z3's internal learned clauses are disposable performance state.

29.8 Protocol identity

A future incremental observation should bind at least:

- solver executable digest and observed version/capabilities;
- launch, environment, directory, timeout, and resource policies;
- grammar and domain schemas;
- base command bytes and fingerprint;
- sample lineage and assertion order;
- push/pop depth or assumption set;

- core and optimization settings;
- transaction ordinal and rebuild generation;
- requested symbol schema and response limits.

A status detached from that identity is not cacheable evidence.

Recursive programs, CHCs, and Lean-checked proofs

30.1 Why recursive functions are a different problem

For a nonrecursive candidate in a first-order domain, symbolic evaluation produces a finite expression. A recursive candidate denotes an unbounded computation tree. Simply expanding recursive calls to depth k gives a bounded model:

- excellent for finding small counterexamples;
- useful for CEGIS samples and regression tests;
- incapable by itself of proving a property for arbitrary depth;
- sensitive to the chosen unfolding bound.

Z3 supports recursive function definitions (`define-fun-rec`), but iterative unfolding is not mathematical induction. A three-line experiment shows the difference. Define `len` recursively as $\text{len}(n) = \text{ite}(n \leq 0, 0, 1 + \text{len}(n - 1))$ and ask whether some $n \geq 0$ has $\text{len}(n) \neq n$:

```
(set-option :timeout 3000)
(define-fun-rec len ((n Int)) Int
  (ite (<= n 0) 0 (+ 1 (len (- n 1)))))
(declare-const n Int)
(assert (>= n 0))
; is there an n at which len n differs from n?
(assert (not (= (len n) n)))
(check-sat)
(get-info :reason-unknown)
```

Listing 30.1: Bounded unfolding cannot prove a universal fact about a recursive definition.

Z3 5.0.0 answers unknown with `(:reason-unknown "timeout")`: it keeps unfolding `len` without ever finding a counterexample and without being able to conclude that none exists. Adding the bound `(assert (<= n 3))` turns the answer into `unsat` within milliseconds, and replacing the definition by a faulty one that never increments `((ite (<= n 0) 0 (bad (- n 1))))` yields `sat` with `((n 1))` at once. So unfolding is excellent at finding small counterexamples and useless as a proof of the unbounded property. The planned positive path therefore uses constrained Horn clauses for relational semantics and Lean for final proof authority.

30.2 Relational semantics

Instead of defining a recursive result as one term, introduce a relation

$$\text{Eval}_f(x, y)$$

meaning that candidate f can evaluate input x to output y in the modeled semantics. Base and step cases become Horn rules. For a schematic list-length transformer:

$$\begin{aligned} \text{Eval}(0, r_0) &\leftarrow \text{BaseResult}(r_0), \\ \text{Eval}(n + 1, r) &\leftarrow n \geq 0 \wedge \text{Eval}(n, r') \wedge \text{StepResult}(n, r', r). \end{aligned}$$

The bad relation is

$$\text{Bad} \leftarrow \text{Eval}(n, r) \wedge P(n) \wedge \neg Q(n, r).$$

A fixedpoint query asks whether Bad is reachable.

30.3 SMT-LIB fixedpoint shape

A small illustrative CHC system for the identity length recursion is:

```
(set-logic HORN)

(declare-rel Eval (Int Int))
(declare-rel Bad ())
(declare-var n Int)
(declare-var r Int)

; Base: input length 0 produces result length 0.
(rule (Eval 0 0))

; Step: if n produces r, n+1 produces r+1.
(rule (=> (and (<= 0 n) (Eval n r))
         (Eval (+ n 1) (+ r 1))))

; Contract violation: result length differs from input length.
(rule (=> (and (<= 0 n) (Eval n r) (not (= n r)))
         Bad))

(query Bad)
```

Listing 30.2: Illustrative Horn-clause safety query.

Run through Z3 5.0.0, (query Bad) answers unsat: under the declare-rel/rule/query front end this means that Bad is *not derivable*, that is, the candidate is safe. Spacer establishes it by inferring an invariant equivalent to $r = n$; adding (set-option :fp.print_certificate true) before the query makes Z3 print that invariant after the status:

```
unsat
(forall ((A Int) (B Int))
  (! (let ((a!1 (not (<= (+ A (* (- 1) B)) (- 1))))
      (a!2 (not (>= (+ A (* (- 1) B)) 1))))
    (= (Eval A B) (and a!1 a!2)))
  :weight 0))
```

Read the answer as $\text{Eval}(A, B) \iff \neg(A - B \leq -1) \wedge \neg(A - B \geq 1)$, which is $A = B$: exactly the invariant a domain would normalize to $r = n$ before handing it to Lean. A faulty step rule makes Bad reachable and produces a derivation. Replacing the step head by (Eval (+ n 1) r), so that the result length is never incremented, gives sat and, with the same option, the hyper-resolution derivation

```
sat
(let ((a!1 (forall ((A Int) (B Int))
  (! (=> (and (Eval A B) (<= 0 A) (not (= A B))) query!0) :weight 0)))
  (a!2 (forall ((A Int) (B Int) (C Int))
```

```

      (! (=> (and (Eval B C) (<= 0 B) (= A (+ 1 B))) (Eval A C))
        :weight 0))))
(1et ((a!3 ((_ hyper-res 0 0 0 1)
  (asserted a!2)
  ((_ hyper-res 0 0) (asserted (Eval 0 0)) (Eval 0 0))
  (Eval 1 0))))
  (mp ((_ hyper-res 0 0 0 1) (asserted a!1) a!3 query!0)
    (asserted (=> query!0 false))
    false)))

```

whose interesting fact is the derived atom (Eval 1 0): input length one evaluates to result length zero, the concrete counterexample the domain would replay and store in the bank.

Common mistake

The two Horn front ends report the same fact with opposite words. With `declare-rel/rule/query`, as above, (query Bad) answers `unsat` when the bad state is unreachable (safe) and `sat` when it is reachable. With the pure SMT-LIB encoding of Chapter 12, where the clauses are asserted under (set-logic HORN) and the query is a clause with head `false` checked by `check-sat`, Z3 answers `sat` when an inductive model of all clauses exists (safe; `get-model` then prints the invariant, for this system (define-fun Eval ((x!0 Int) (x!1 Int)) Bool ...) with the same $x_0 = x_1$ body) and `unsat` when false is derivable (bad state reachable; `get-model` then fails with “model is not available”). Both were checked against Z3 5.0.0. A fixedpoint protocol must commit to one front end and name outcomes by meaning, “safe with invariant artifact” or “bad state reachable with derivation”, never by the raw word.

The fixedpoint command language and result interpretation are a separate protocol profile from ordinary `check-sat`. Query status, answer expressions, trace extraction, and statistics need their own bounded decoder and fingerprints.

30.4 Counterexample traces

When a bad state is reachable, the useful artifact is a finite derivation:

- initial symbolic or concrete input;
- sequence of rule applications;
- values assigned at each relation instance;
- final violated contract clause.

The domain independently replays the trace against the exact candidate’s relational semantics. If the trace corresponds to a concrete finite input, it can enter the ordinary counterexample bank. If it is a symbolic derivation using uninterpreted relations, the evidence type records that model.

A trace should be simplified deterministically where possible: shortest derivation, smallest input, or smallest lexicographic model. Solver trace output is untrusted and bounded just like ordinary models.

30.5 Invariant candidates

When Bad is unreachable, Spacer may expose an inductive interpretation for relations: under the query front end through (set-option :fp.print_certificate true), under the check-sat front end through `get-model`, as shown above. An invariant candidate $I(x, y)$ is useful only after checking:

$$\begin{aligned}
 \text{Base} &\Rightarrow I, \\
 I \wedge \text{Step} &\Rightarrow I', \\
 I &\Rightarrow Q.
 \end{aligned}$$

There are three authority levels:

1. raw fixedpoint “not derivable” status (unsat under the query front end, sat under the check-sat front end) and its answer expression: heuristic solver observation;
2. independently rechecked invariant obligations in a restricted decidable domain: model-relative positive evidence;
3. translated invariant and obligations proved in Lean: source-level checked theorem under explicit provider assumptions.

The translator should accept only a restricted invariant grammar that can be represented faithfully in Lean: linear inequalities, constructor/tag predicates, finite conjunction/disjunction, selected array facts, or other domain-owned forms. Unsupported answer expressions are diagnostic, not proof material.

30.6 Lean theorem generation

A proof-producing behavioral request should generate two artifacts:

1. the synthesized definition or exact term;
2. a theorem stating the checked contract for that exact definition.

Examples include:

```
def synthesizedReverse {a : Type u} : List a -> List a :=
  List.reverse

theorem synthesizedReverse_length {a : Type u} (xs : List a) :
  (synthesizedReverse xs).length = xs.length := by
  simp [synthesizedReverse]

def synthesizedOptionBind {a b : Type u} :
  Option a -> (a -> Option b) -> Option b :=
  fun x k => match x with
  | none   => none
  | some v => k v

theorem synthesizedOptionBind_spec {a b : Type u}
  (x : Option a) (k : a -> Option b) :
  synthesizedOptionBind x k = x.bind k := by
  cases x <;> rfl
```

Listing 30.3: Proof-artifact shapes.

For arithmetic domains, the proof may use `omega`, `linarith`, `normalization`, or `bv_decide`. For recursive domains, it may use induction with a solver-proposed invariant or strengthening lemma.

30.7 Provider-law proof linkage

Current Length provider laws are explicit assumptions. A proof-producing system should allow a law to carry one of several statuses:

- assumed uniformly by the user;
- assumed after exact ground class discharge;
- linked to a named Lean theorem with checked instantiation;
- generated and proved for the exact provider definition;
- imported from a domain package whose soundness theorem is checked.

The generated contract theorem should expose assumptions that are not discharged. It must not hide an assumed `reversePreservesLength` law behind a theorem whose statement appears unconditional.

30.8 Termination remains a Lean concern

A CHC model can describe recursive relations without proving that a proposed Lean definition passes the termination checker. Djex may generate structural recursive schemes or explicit well-founded templates, but Lean must accept the resulting definition. A solver-selected decrease measure can be proposed; the final termination proof is checked by Lean.

Likewise, a relational semantics may over-approximate nontermination. Its exactness descriptor states whether Eval is total, partial, may-diverge, or bounded. Safety of terminating results and total correctness are different claims.

30.9 Recursive synthesis

Selectors can occur inside CHC rules. For example, a recursive step may choose:

- which recursive argument decreases;
- which fields are passed through;
- which constructor wraps the recursive result;
- which branch guard or arithmetic update is used;
- which invariant template clauses are active.

A synthesis loop can alternate between selector solving, CHC safety checking, and counterexample traces. This is powerful but substantially more complex than finite sample semantics. It should follow mature typed sketches and exact nonrecursive domains.

30.10 Synthesiz3 as an adjacent experiment

The Z3 ecosystem has explored synthesis-oriented extensions such as Synthesiz3 (Hozzová and Bjørner, FMCAD 2025; the proposal [10] gives the citation and artifact location), which encodes restricted synthesis specifications over Z3 through an extended assert-synth command and an :uncomputable option naming symbols that may appear in the specification but not in the synthesized term. It is relevant as a comparative backend, particularly for first-order grammars and specifications with symbols allowed in reasoning but forbidden in generated terms. It is not part of the pinned Z3 build's ordinary SMT-LIB surface and nothing in the current tree depends on it.

For Leant/Djex, the initial production path should remain selector-based because:

- Djex already owns the checked typed grammar;
- the current process protocol can be extended incrementally without requiring a synthesis-specific fork first;
- selector models decode to exact known alternatives;
- Lean-specific materialization and occurrence certificates remain central;
- benchmarks can compare an experimental synthesis backend against the same domain and grammar fingerprints later.

30.11 What a positive result should say

User-visible wording should distinguish:

No replayed counterexample found.

Search ended without evidence of violation; no universal claim.

Validated within scope.

Every admitted assignment in a finite scope was checked independently.

Established in exact finite model.

The complete represented applicable domain was traversed and the encoding is exact for that model.

Solver-indisproved in model.

Z3 returned unsat for the exact encoding, but no proof artifact was checked.

Invariant independently checked.

A restricted invariant satisfied all domain-owned obligations.

Lean theorem checked.

Lean accepted a theorem for the exact synthesized definition and stated assumptions.

This vocabulary is not bureaucratic decoration. It is the difference between a synthesis assistant and a source of overclaimed proofs.

A staged implementation and evaluation plan

In Leant/Djex

The milestones below are this guide's compressed ordering; they do not correspond one-to-one to the proposal's Milestones 0–10 [10]. In particular the proposal places constructor-behavior and state-provenance domains (its Milestone 4) before nonrecursive typed sketches (its Milestone 5), and sound prefix pruning at its Milestone 9, whereas here the first sketch slice comes before new domains and prefix pruning is staged across chapters. Its Milestone 1, hard filtering over the current Length domain, is the part already implemented at the inspected tree.

31.1 Milestone 0: preserve and benchmark the current baseline

Before adding new search feedback, establish a reproducible corpus for the current Length system:

- scalar preservation, growth, truncation, affine, and conditional contracts;
- binary-product sum and component constraints;
- exact-case candidates;
- providers with unconditional and ground-conditional laws;
- handoff refusals and duplicate spellings;
- sat, unsat, unknown, timeout, malformed-child, and cleanup paths;
- rank/filter equivalence when no replayed counterexample exists;
- progressive filtering with bank hits across batches.

Record candidate counts, callback attempts, pure replay attempts, live queries, response bytes, elapsed usable work, and retained/rejected occurrence identities.

31.2 Milestone 1: semantic signatures

Add exact sample evaluation over typed candidate graphs and deterministic signature partitioning. Initially use it only to schedule candidates. Success criteria:

- no change to final result set under ranking mode;
- fewer Lean callback attempts before the first behaviorally surviving candidate;
- fewer live Z3 queries through earlier bank hits;
- deterministic partitions under repeated runs;
- explicit diagnostics for candidates outside the domain.

31.3 Milestone 2: candidate-level CEGIS

Permit explicit hard sample pruning and feed each replayed counterexample back into the active enumeration context. Add bounded sample retention and lineage fingerprints. Verify that:

- every pruned candidate has an exact sample-replay reason;
- search completeness labels are adjusted correctly;
- removing or replacing samples invalidates dependent caches;
- the final post-verification filter remains the authority of last resort.

31.4 Milestone 3: one typed-sketch vertical slice

Choose a small first-order domain such as `Option.bind` or state-threading provenance. Implement:

- Djex-prepared finite choice DAG;
- canonical selector encoding;
- bounded model decoding;
- exact materialization and new typed origin;
- Lean callback verification;
- full CEGIS loop;
- deterministic cost and selector tie-breaking;
- differential comparison against ordinary enumeration.

The goal is not maximal grammar size. It is an end-to-end authority-preserving slice.

31.5 Milestone 4: incremental protocol, cores, and nogoods

Introduce a separate scoped Z3 profile with persistent base state, assumption checks, core decoding, rebuild, and canonical sample replay. Begin with sample-level grammar infeasibility and small minimized nogoods. Native Optimize can remain experimental until iterative cost tightening is stable.

31.6 Milestone 5: generalized observation domains

Implement size/tag, state/provenance, bounded sequence/index, and bit-vector domains in that order. Each domain ships with:

- solver-neutral contract AST;
- exact source-fragment declaration;
- checked provider-summary vocabulary;
- model decoder and independent replay;
- finite validator where practical;
- query and execution schemas;
- user-visible exactness vocabulary;
- benchmark and malformed-response corpus.

31.7 Milestone 6: proof artifacts

For domains aligned with existing Lean automation, generate and check contract theorems. Bit-vectors and simple structural laws are attractive first targets. Store the theorem statement, candidate identity, provider assumptions, and proof script/result beside the synthesis outcome.

A failed proof does not discard a candidate unless proof mode explicitly requires it. It can fall back to the strongest available model-relative status.

31.8 Milestone 7: CHC experiments

Add a separate fixedpoint capability profile and begin with small relational recursions whose invariants are linear arithmetic. Evaluate:

- trace quality and replay;
- stability of answer expressions;
- invariant translation coverage;
- proof-generation success in Lean;
- behavior under time/resource limits;
- benefit over bounded unfolding.

Only after these experiments should CHC results influence hard prefix pruning or recursive selector synthesis.

31.9 Benchmark scenarios

A compact but expressive benchmark suite should include:

List identity versus reverse versus constants.

Shows observation granularity and Length's inability to distinguish permutations.

Drop/take/append compositions.

Exercises monus, min/max, conditionals, provider laws, and small counterexamples.

Split or partition pair.

Exercises binary-product lengths and later bag/order domains.

Option bind.

Exercises tags, callback provenance, finite sketches, and simple Lean proof generation.

State bind.

Exercises uninterpreted sorts/functions, argument wiring, and relational replay.

List map.

Exercises length plus sequence/index semantics and callback functions.

Saturating fixed-width addition.

Exercises bit-vector selectors, constant synthesis, optimization, and `bv_decide` proof.

Small recursive accumulator.

Exercises bounded unfolding, CHC invariant inference, termination, and induction proof generation.

31.10 Metrics

Measure more than wall time:

- generated typed candidates and unique semantic signatures;
- rendered variants and Lean callback attempts;
- callback-accepted occurrences;
- pure bank hits, origin hits, and finite-validation hits;
- live worker openings, queries, resets, and rebuilds;
- sat/unsat/unknown distribution by query schema;
- replay success and rejection rates;
- samples retained, promoted, simplified, or evicted;
- core and nogood sizes before/after minimization;
- sketch variables, clauses, and model-decode sizes;
- proof obligations generated and Lean-checked;
- survivor quality under a stable human-readable ordering;
- exact failure class and cleanup completeness.

A faster run that silently weakens exactness is not an improvement. Every performance comparison must hold the grammar, domain, policy, and result-strength criteria fixed.

31.11 Testing strategy

The test matrix should combine:

- pure unit tests for every sealer, encoder, decoder, and fingerprint;
- golden canonical SMT-LIB bytes;
- property tests for encode/decode and evaluator/translator agreement;
- metamorphic tests under alpha-renaming and stable provider reorderings;
- differential checks against a small exhaustive interpreter;
- real Z3 integration tests pinned to an observed build;
- fake-child protocol tests for chunking, malformed responses, stalls, and marker injection;
- cancellation and cleanup stress tests;
- Lean project tests for elaboration, duplicate spellings, universes, and type classes;
- proof-artifact recompilation tests;
- mutation tests that deliberately break one translation rule and require replay to catch it.

31.12 Non-goals

The proposed direction does not require:

- encoding all of Lean's dependent type theory in Z3;
- trusting Z3 as Lean's kernel;
- synthesizing arbitrary source strings from an unrestricted solver model;
- deciding arbitrary higher-order extensional equivalence;
- proving termination from an ordinary SMT model;
- treating an unsat core as minimal;
- promising stable model choices across solver versions;
- making every provider transparent to symbolic execution;
- replacing Djinn or Exference with one monolithic SMT query.

The architecture gains power by keeping boundaries narrow and compositional.

Checkpoint

The shortest useful roadmap is: replayed samples first, semantic signatures second, one finite typed sketch third, incremental cores and optimization fourth, richer exact domains fifth, Lean proof artifacts sixth, and CHCs last. Every stage should remain independently useful if later stages take longer.

Part VI

Practical Reading and Debugging Guide

How to read one Z3 query end to end

32.1 Start with the semantic question

Before reading SMT-LIB, write the intended query in ordinary mathematics:

1. What are the modeled inputs?
2. What domain restrictions apply?
3. What does the candidate denote in this observation domain?
4. What is the contract precondition?
5. What is the contract postcondition?
6. Is the query looking for a violation, a completion, an equivalence witness, an optimum, or a reachable bad state?

For current Length verification, the answer should reduce to

$$\exists \vec{n} \in \mathbb{N}^k. P(\vec{n}) \wedge \neg Q(\vec{n}, R_C(\vec{n})).$$

If the SMT-LIB seems to answer a different question, stop at the translator rather than debugging Z3 heuristics.

32.2 Identify the logic and theory profile

Find set-logic. In the current Length query it must be QF_LIA. Then verify that every emitted operation belongs to the intended profile:

- integer constants and variables;
- addition, subtraction, and literal scaling;
- linear comparisons and equality;
- Boolean connectives;
- ite;
- helper functions that expand to the same fragment.

Unexpected variable-by-variable multiplication, quantifiers, arrays, sequences, bit-vectors, recursive functions, or uninterpreted sorts indicate either a different domain/profile or a translation bug.

32.3 List declarations in order

Classify every declared symbol:

Source input.

Should have a stable generated position and appear in the exact input-value request. At the pinned revision the scalar translator names them `djex_length_input_0`, `djex_length_input_1`, ... in source-argument order (Section 32.11).

Selector.

Future sketch choice; must have a checked finite range.

Private witness.

Quotient/remainder, abstraction, or auxiliary value; should not be exposed as a source counterexample without a decoder. The pinned translator allocates `djex_length_modulo_quotient_N/djex_length_modulo_remainder_N` and `djex_length_quotient_quotient_N/djex_length_quotient_remainder_N` in expression preorder; they are declared but never requested by `get-value`.

Semantic value.

Sample-specific node output in a sketch.

Assumption literal.

Tracks a clause or production choice for cores.

Relation.

Fixedpoint/CHC predicate with a declared argument signature.

A source-level result variable should normally have been substituted away in current Length queries. If it appears, determine whether it is an intentionally constrained semantic variable or an accidental unconstrained degree of freedom.

32.4 Separate domain axioms from the bad state

Mark assertions into groups:

1. sort/domain constraints such as $n \geq 0$;
2. helper/witness equations;
3. provider or datatype axioms;
4. candidate semantic equations;
5. contract precondition;
6. negated postcondition;
7. finite-sample or objective constraints.

Then check polarity. The most common verification mistake is to assert the postcondition instead of its negation, which asks Z3 to find a good input rather than a counterexample.

32.5 Predict trivial models by hand

Try 0, 1, and small boundary values. For a constant-empty candidate under preservation:

$$0 \neq n$$

is false at $n = 0$ and true at $n = 1$. A sat model should therefore exist immediately. If the model is huge, simplify it during replay or add a deterministic minimization layer; do not mistake model size for semantic necessity.

For identity:

$$n \neq n$$

is syntactically impossible. If Z3 reports sat, the query or protocol is wrong.

32.6 Read status before values

A `get-value` request is meaningful only after `sat`. After `unsat`, there is no model. After `unknown`, Z3 may expose a partial model in some APIs, but the current Length profile does not treat it as a counterexample source.

The protocol state should therefore enforce:

- exactly one status for the check;
- a value request only on the admitted `sat` path;
- exact expected binding count and names;
- no stale values from an earlier transaction;
- complete causal framing before the next reset.

32.7 Replay the model manually

Given decoded inputs, ignore the SMT result expression and evaluate the candidate semantics yourself. For input length 1:

- constant empty yields 0;
- duplicate yields 2;
- tail yields 0;
- identity yields 1;
- reverse with the admitted law yields 1.

Check P and Q . This manual exercise mirrors the independent evaluator and often reveals whether the problem lies in symbolic interpretation, SMT translation, model decoding, or result presentation.

32.8 Run a standalone SMT-LIB file

For debugging only, save the exact canonical bytes to a file under an explicit artifact policy and run the pinned executable directly:

```
z3 -smt2 query.smt2
```

Listing 32.1: Standalone Z3 invocation for an SMT-LIB file.

For an interactive stream:

```
z3 -in -smt2
```

Standalone success does not prove the embedded protocol is correct. The application may use different arguments, environment, timeout, resource limit, working directory, executable bytes, framing, or response parser. Section 32.8.1 reproduces the exact embedded argument vector and stream so that a shell test can at least speak the same protocol.

32.8.1 Reproducing the embedded launch by hand

The pinned execution profile ([synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Z3/Execution.hs](#)) launches the executable directly, without a shell, with an empty environment, in a fresh empty working directory, and with exactly this argument vector:

```
z3 -in -smt2 smtlib2_compliant=true timeout=<solverTimeoutMilliseconds> rlimit=<solverResourceLimit>
```

Listing 32.2: Argument vector of the pinned profile; the two numbers come from the startup file.

From a shell, `env -i and cd "$(mktemp -d)"` approximate the environment and directory policy (the real launcher additionally binds a descriptor and, on Linux, stages a sealed copy of the executable). On standard input the session first writes the startup command and a barrier, then for every query the reset prefix, the canonical check bytes, and a barrier, and, only after sat, the value request and a barrier:

```
(set-option :print-success false)
(echo "djex-smtlib-frame/v1/<64 hex digits>")
(reset)
(set-option :print-success false)
(set-option :produce-models true)
(set-option :random-seed 1)
(set-logic QF_LIA)
(define-fun djex_nat_monus ((x Int) (y Int)) Int (ite (<= y x) (- x y) 0))
(define-fun djex_nat_min ((x Int) (y Int)) Int (ite (<= x y) x y))
(define-fun djex_nat_max ((x Int) (y Int)) Int (ite (<= x y) y x))
(declare-const djex_length_input_0 Int)
(assert (<= 0 djex_length_input_0))
(assert (not (= djex_length_input_0 0)))
(check-sat)
(echo "djex-smtlib-frame/v1/<64 hex digits>")
(get-value (djex_length_input_0))
(echo "djex-smtlib-frame/v1/<64 hex digits>")
```

Listing 32.3: Exact stream shape for one scalar transaction; each barrier carries a fresh 32-byte nonce in hex.

Z3 5.0.0 answers, in order, the quoted startup barrier, sat, the quoted check barrier, `((djex_length_input_0 1))`, and the quoted value barrier (reproduced through the Python bindings with the same three global parameters). Before the first query the session runs a four-write readiness handshake against the same preamble ([synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session/Capability.hs](#)): a declaration `djex_capability_input` asserted equal to 0 must answer sat, its `get-value` must answer exactly `((djex_capability_input 0))`, and the same declaration asserted equal to both 0 and 1 must answer unsat, each followed by its own barrier echo. A worker that prints success lines, drops the quotes around echoed strings, or reorders frames is `CapabilityRejected` before any `Length` query is sent. Nothing in the reproduction grants evidence: the point is only to make a shell test byte-compatible with what the worker actually receives.

32.9 Minimize without changing the claim

When reducing a failing query:

1. retain `set-logic` and relevant options;
2. retain all declarations used by the bad state;
3. remove unrelated helper definitions one at a time;
4. inline candidate semantics only if the inlining is mechanically checked;
5. preserve source-input nonnegativity and all witness constraints;
6. keep symbol names or maintain a renaming map;
7. record the original query fingerprint and reduction steps;
8. replay any model against the original checked problem.

A hand-minimized formula is a diagnostic artifact, not the original query's authority.

32.10 Compare two queries structurally

A byte diff can be noisy when one private witness shifts every later name. Compare typed plans first:

- same problem and contract fingerprints?

- same input arity and role vector?
- same symbolic candidate result?
- same helper/witness preorder?
- same assertion tree modulo generated names?
- same command and value-request bytes?
- same schema and execution identity?

If plans differ, a byte diff is merely a symptom. If plans match but bytes differ, the canonical renderer or symbol allocator is suspect.

32.11 Reading a real query and its fingerprint

The pedagogical listings in this document use friendly names such as `input.0`. This section shows what the pinned translator actually emits, so that a maintainer reading a retained artifact recognizes it. The bytes below were produced by sealing a checked problem through the pinned Djex tree (`sealLengthSMTLibQuery` with `defaultLengthSMTLibLimits`, using the Length test harness’s constant-empty candidate `fun _ => []` under the contract $P \equiv \top$, $Q \equiv r = n_0$) and are the exact `lengthSMTLibQueryCheckBytes`:

```
(set-option :produce-models true)
(set-option :random-seed 1)
(set-logic QF_LIA)
(define-fun djex_nat_monus ((x Int) (y Int)) Int (ite (<= y x) (- x y) 0))
(define-fun djex_nat_min ((x Int) (y Int)) Int (ite (<= x y) x y))
(define-fun djex_nat_max ((x Int) (y Int)) Int (ite (<= x y) y x))
(declare-const djex_length_input_0 Int)
(assert (<= 0 djex_length_input_0))
(assert (not (= djex_length_input_0 0)))
(check-sat)
```

Listing 32.4: Exact check bytes sealed by the pinned Djex translator for a constant-empty candidate under length preservation.

The separate value request is `(get-value (djex_length_input_0))` followed by one line feed, and `lengthSMTLibQueryInputSymbols` is exactly `["djex_length_input_0"]`. Z3 5.0.0 answers `sat` and `((djex_length_input_0 1))`. Points to notice:

- One command per line, single spaces, no comments: the renderer in [synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/QFLIA.hs](#) has exactly one spelling for each construct, which is what makes byte identity meaningful.
- The precondition `true` has disappeared and the equality reads `(= djex_length_input_0 0)` although the contract wrote “result equals input.” The Length normalizer canonicalizes conjunct and operand order before translation, so source order cannot be read back from the bytes; a contract with precondition $n_0 > 0$ and postcondition $r = n_0 \div 1$ came out, for the same candidate, as `(assert (and (not (= 0 (djex_nat_monus djex_length_input_0 1))) (not (<= djex_length_input_0 0))))`, negated postcondition first.
- A modulo in the contract adds declarations and witness constraints between the input declaration and the bad state; for $Q \equiv r = n_0 \bmod 3$ and the same candidate the sealed bytes declare `djex_length_modulo_quotient_0` and `djex_length_modulo_remainder_0`, `assert` both nonnegative, `assert (<= djex_length_modulo_remainder_0 2)` and `(= djex_length_input_0 (+ (* 3 djex_length_modulo_quotient_0) djex_length_modulo_remainder_0))`, and only then `(assert (not (= 0 djex_length_modulo_remainder_0)))`; Z3 answers `sat` with `djex_length_input_0 = 1`, and the witnesses are never requested.
- Replay is a separate step with its own closed errors. Calling `validateLengthSMTLibCounterexample` on this query with the binding `djex_length_input_0 = 1` yields evidence; `0` yields `Nothing` (not a counterexample, so the transaction would fail closed); `-1` yields `LengthSMTLibNegativeInputValue`; the symbol `input.0` yields `LengthSMTLibUnknownInputSymbol`; and an empty binding list yields `LengthSMTLibBindingArityMismatch 1 0`. All five were reproduced in the pinned tree.

The query fingerprint is not a digest of those bytes but a complete canonical key. Its layout ([synthesis/internal/Language/Haskell/Synthesis/Internal/Fingerprint.hs](#)) is the six magic bytes DJEXFP and an encoding-version byte, then a natural (the builder version, here 1), then the role bytes `finite-list-spine-length/z3-qf-lia-query`, then one sequence of tagged fields, each field written as a one-byte kind (`0x01` natural, `0x02` bytes, `0x03` sequence, `0x04` tag, `0x05` name), a variable-length byte

count, and the payload. For the query above the tags are, in order: schema (djex-length-z3-qf-lia-smtlib2/v2), logic (QF_LIA), fixed-options (:produce-models=true, :random-seed=1), problem-domain (finite-list-spine-length/v1), problem-inventory, problem-encoding, problem-candidate, and complete-problem (each the complete nested key of that fingerprint, which is why the key for even this trivial query is about 8.9 KB and mostly consists of the inventory, session policy, contract, and typed-graph identities), then typed-plan (input symbols, the condition tree, the command tree, and the value request), check-command (the exact bytes above), and input-value-request (present plus the exact request bytes, or absent for a nullary problem). Because every payload is length-prefixed the key is decodable and two different plans cannot share a key; the Show instance prints it as lowercase hex (fingerprintCode), and any SHA-256 you see in tests is computed over this key for a snapshot assertion, not stored as identity. When two retained queries “look the same,” diff the tagged fields in this order before diffing bytes: a difference in problem-inventory or problem-candidate with identical check-command is the normal case of two candidates that happen to have the same length semantics, and it is exactly why identical bytes never make one candidate’s evidence valid for another.

Troubleshooting by symptom

The table below is the short form of this chapter: what you see, what most often causes it, where the deciding code lives, and what to change. Every message is quoted from the pinned sources; the failure constructors appear verbatim in Leant's warning:/error: lines because they are printed with show. Long identifiers are set in a line-breakable monospace so that they fit the columns. The sections that follow give the longer reasoning.

Symptom	Likely cause	Where to look	What to change
Leant exits before the prompt with error: finite-spine Length counterexample ranking setup failed: LengthAssessmentFileAdmissionRejected LengthRankingConfigurationFilePathNotAbsolute	Relative --length-ranking -config path	src/Leant/Synth/Length/File/Acquire.hs , mkLengthFileRequest	Pass an absolute path (also required for --length-contract).
... LengthAssessmentFileLoadRejected (LengthRankingConfigurationFileLoadError LengthRankingConfigurationFilePlatformUnsupported False)	Windows build: acquisition is POSIX-only and fails closed	same file, the mingw32_HOST_OS branch	Run Leant's Length assessment on Linux.
... LengthRankingConfigurationFileDecodeRejected (LengthRankingConfigurationUnexpectedField LengthRankingConfigurationRootObject)	A historical version or strategy member in the startup file	src/Leant/Synth/Length/Configuration/File.hs , rootFields	Keep exactly the ten root members; delete removed ones.
... LengthRankingConfigurationFieldValueRejected LengthRankingConfigurationExpectedExecutableSha256Field	Digest is not 64 lowercase hex characters (uppercase hex is rejected)	same file, decodeExpectedDigest	Use sha256sum output verbatim, or null plus --length-ranking-allow-unpinned.
... LengthAssessmentActivationRejected LengthRankingConfigurationExecutablePinRequired	expectedExecutableSha256 is null without the allow-unpinned flag	src/Leant/Synth/Length/Integration.hs	Supply the digest or add the flag deliberately.
:synth answers usage: :synth TYPE :synth [--behavior-mode rank filter] [--length-contract ABSOLUTE-PATH] -- TYPE	Mode word missing or not rank/filter; -- delimiter missing; empty path; mode after contract	src/Leant/Synth/Length/Command.hs	Order is mode, then contract, then a standalone --.
finite-spine Length request rejected before IO: LengthAssessmentFilteringRequiresActivatedPolicy or LengthAssessmentExplicitContractRequiresActivatedPolicy	Filter mode or a contract file requested while assessment is disabled	src/Leant/Synth/Length/Integration.hs , startupLengthAssessmentRequest	Start Leant with --length-ranking-config; a contract file cannot activate Z3.

Symptom	Likely cause	Where to look	What to change
one-shot finite-spine Length contract admission failed: or ... load failed:	Contract path not absolute, or the document is not the exact three-member lean t-finite-list-spine-length-contract root with a six-member contract	src/Leant/Synth/Length/Contract/File.hs	Follow the schema in docs/length-ranking.md; rankingDomain must match the result references the expressions use.
Every candidate is shown, no notes, no warning	Default djinn engine (no typed origin), or every candidate refused preparation	src/Main.hs (rsSynthEngine), src/Leant/Synth/Length/Handoff.hs	:set synth-engine exference (or both); confirm with LEANT_SYNTH_DEBUG=1.
warning: finite-spine Length behavioral assessment preserved all verified candidates: ... LengthRankingLiveSessionFailed LengthSMTLibLiveSessionLaunchFailed ...	Non-Linux build, or a Linux kernel without AT_EXECVE_CHECK; the fixed launcher has no fallback	synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs , sessionProcessFailure	Use Linux with a kernel providing AT_EXECVE_CHECK (upstream 6.14 or later).
... LengthSMTLibLiveSessionExecutableRejected	Digest mismatch, executable not a regular/executable file, metadata changed between inspection and launch, or exec-check denied	same mapping	Recompute the SHA-256 of the exact file named by executablePath; fix mode bits.
... LengthSMTLibLiveSessionExecutableUnavailable	executablePath does not exist	same	Fix the path (absolute, no NUL, no surrogates).
... LengthSMTLibLiveSessionCapabilityRejected	The file is not Z3, or does not answer the readiness handshake byte-exactly	synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session/Capability.hs	Replay the handshake by hand (Section 32.8.1).
... LengthSMTLibLiveSessionDeadlineExceeded or LengthRankingLiveQueryFailed LengthSMTLibLiveQueryDeadlineExceeded	Host deadline or the shared usable-work budget too small for opening plus queries	startup execution.host DeadlineMilliseconds, usableWorkBudget.milliseconds	Raise them (host deadline $\geq$ solver timeout +100 ms, cap 65 000; budget cap 65 000).
... LengthSMTLibLiveQueryLimitExceeded 64 65	More than 64 scalar-plus-product transactions in one live scope	synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs	Only library callers can hit this; Main's batches are at most 24.
... LengthSMTLibLiveQueryProtocolRejected	Response exceeded responseLimits or was malformed	startup execution.responseLimits	Raise bytes, nodes, tokenBytes, or integerBits (caps 65 536, 4 096, 4 096, 4 096).
... LengthSMTLibLiveQueryCounterexampleRejected	Z3's model failed Djex's independent replay, or its bindings were malformed	synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib.hs , validateLengthSMTLibCounterexample	Treat as a correctness bug: rerun the exact bytes standalone and compare.
Z3 says sat standalone but the candidate carries no note	"artifactPolicy": "status-only": no get-value is ever sent	synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session.hs	Use "input-values-after-satisfiable".

Survivor note ends in vacuous within this box (no assignment met the precondition)	The precondition excludes every assignment of the configured box	startup inputBoxValidation.inclusiveInputMaximums	Widen the box or check the precondition; a vacuous receipt is neutral, not positive.
--	--	---	--

33.1 Z3 was not launched

Possible explanations include:

- assessment is disabled (no `--length-ranking-config`), so ranking is the lazy identity;
- the startup file was refused, in which case Leant printed error: `finite-spine Length counterexample ranking setup failed: ...` and exited with status 1 before the prompt appeared (Section 33.12);
- the synthesis engine is the default djinn, which supplies no typed candidate graph: only `:set synth-engine exference` or both produce candidates with the exact typed authority the handoff requires (`src/Main.hs` starts with `EngineDjinn`; the enabled-mode banner ends with “Select synth-engine exference or both to produce graph-eligible candidates”);
- the candidate route otherwise lacks exact typed authority (text-only or duplicate-recovered without a sidecar);
- handoff, contract sealing, candidate interpretation, or query construction refused (a candidate-local preparation refusal; Section 33.5);
- a retained bank sample already produced a counterexample;
- the guarded applicable-domain validation or the all-zero origin probe completed the assessment before IO (input-box validation cannot: it runs only after a live unsat);
- deferred opening is the fixed startup policy and no candidate reached a live miss;
- the verified batch was empty;
- the platform cannot launch the fixed descriptor-bound launcher at all: on Windows the configuration file itself is refused (`LengthRankingConfigurationFilePlatformUnsupported`), and on a non-Linux build or a Linux kernel without `AT_EXECVE_CHECK` (upstream 6.14) the first live miss ends the batch with `LengthSMTLibLiveSessionLaunchFailed` (Section 33.13).

Inspect preparation and request-context observations before process logs. Two cheap probes: the enabled-mode startup banner (“Finite-spine Length counterexample ranking enabled for eligible typed Exference origins ...”) confirms activation, and `LEANT_SYNTH_DEBUG=1` makes each lane print debug metric: `typed-candidate-rendered=N`; with `N` absent or zero no candidate could have reached the Length handoff.

33.2 Z3 reported sat, but the candidate was retained

This can be correct. Possibilities:

- model decoding failed;
- requested bindings were missing, duplicate, negative, oversized, or unknown;
- independent replay did not reproduce a violation;
- the result was attached only as heuristic status;
- ranking mode demoted but did not remove the candidate;
- the startup file says `"artifactPolicy": "status-only"`, under which a live sat is deliberately never followed by `get-value`: it stays a heuristic status and no candidate can ever be rejected from a live query (only `"input-values-after-satisfiable"` requests inputs; `synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session.hs`, `replayLengthSMTLibQuery`);
- a later candidate in the same batch hit an operational failure, and the fail-closed ranking then reset *every* candidate of that batch to Unassessed (Section 33.13);
- filter mode encountered a batch-level failure and preserved the original batch;
- the candidate belonged to a different nominal domain than the observation;

- a displayed sat came from a synthesis or equivalence query rather than the verification bad state.

Only an adapter-authorized replayed counterexample rejects in the current filter. Note that a Z3 model that fails independent replay does not merely leave the candidate retained: it fails the query transaction (`LengthSMTLibLiveQueryCounterexampleRejected`) and, through the batch-wide fallback, unassesses the whole batch. A live sat that “vanishes” should therefore be looked for first in the batch warning, not in the candidate note.

33.3 Z3 reported unsat, but Leant did not call the candidate proved

That is the intended trust policy. Raw unsat is a solver observation. The system may rank it neutrally, trigger a finite validator, or retain the candidate. A universal Lean theorem requires a separate proof-producing path.

In the current versionless startup grammar these are not switches. Every accepted configuration file enables the post-unsat input-box validation, the guarded recursive applicable-domain validation, both non-vacuous preferences, the all-zero origin probe, counterexample simplification, and deferred opening (`src/Leant/Synth/Length/Configuration/File.hs`, `decodeLengthAssessmentConfigurationFile`); the file chooses only their limits, chiefly `inputBoxValidation.inclusiveInputMaximums` and `maximumAssignments` and the seven `applicableDomainValidation` caps. A candidate that survives after a raw unsat is therefore shown either with no note at all (heuristic) or with an “independently checked ... Length input box” or “complete ... domain” note (Section 33.14). Those receipts are still scoped to their validated model and never become a proof.

33.4 The same candidate receives different models

SMT models are generally noncanonical. Differences may come from:

- a different Z3 build;
- changed random seed or options;
- changed assertion order or private symbols;
- added unconstrained auxiliaries;
- a different timeout/resource path;
- multiple equally small counterexamples;
- incremental solver history.

Replay should accept every genuine counterexample. For reproducible presentation, run deterministic counterexample simplification or add an explicit optimization/minimization profile.

33.5 A candidate is unassessed

Distinguish candidate-local preparation refusal from operational fallback. Preparation refusal says the exact modeled problem could not be constructed. Operational fallback says a prepared batch could not complete assessment. Neither adds a note in the REPL: an unassessed candidate is displayed exactly like an unranked one, and only operational fallback prints a batch-wide warning (Section 33.13). The candidate-local class is available programmatically as one of ten payload-free codes (`src/Leant/Synth/Length/Ranking/Internal.hs`, `lengthPreparationRefusalClassCode`):

- | | |
|----------------------------------|--------------------------------|
| • unsupported-route | • provider-binding-unavailable |
| • typed-authority-unavailable | • session-rejected |
| • candidate-association-rejected | • contract-rejected |
| • rendering-association-rejected | • candidate-semantics-rejected |
| • spine-binding-unavailable | • query-construction-rejected |

The first two are by far the most common and both mean the same thing for a user: the candidate did not come from a typed Exference route.

Common preparation causes:

- text-only or unsupported rendering route;
- missing exact semantic sidecar;
- changed source/search/request goal association;
- unsupported premises or request contexts;
- renderer mismatch;
- spine/provider name unavailable or ambiguous;
- unsupported candidate graph operation;
- resource limit during problem or query construction.

Common operational causes include executable admission, capability probe, timeout, resource, transport, parser, cancellation, replay, or cleanup failures.

33.6 Filtering changed between batches

In the current progressive filter, later batches may benefit from counterexamples learned earlier. Therefore a candidate assessed in isolation can require a live query while the same candidate later in a command is rejected by bank replay.

Verify:

- the same nominal request context owns both batches;
- the bank scope matches the exact contract/inventory/domain;
- the sample was freshly replayed against the later query;
- no stale evidence receipt was reused;
- MRU promotion/eviction follows deterministic policy.

33.7 A valid provider is reported unavailable

A provider law is resolved against the exact goal-relevant checked inventory and retained translation binding, not every declaration in the Lean environment. Causes include:

- the provider was not included in the synthesis inventory;
- the source name is ambiguous in retained bindings;
- the closed scheme differs from the law's expected role structure;
- required ground constraints were not discharged;
- the candidate occurrence came from a different provider binding;
- the contract names a declaration spelling rather than the exact retained source name.

Do not “fix” this by matching rendered names more loosely; that would weaken occurrence authority.

33.8 Standalone Z3 works, embedded Z3 fails

Compare:

- exact executable digest;

- direct arguments;
- empty versus inherited environment;
- empty temporary versus current working directory;
- solver timeout/resource settings;
- host deadline;
- full canonical bytes, including reset/framing commands;
- interactive stream buffering;
- response chunking and parser limits;
- cleanup and EOF behavior.

A shell test is not equivalent to descriptor-bound execution under an empty environment. Concretely, the pinned profile runs `z3 -in -smt2 smtlib2_compliant=true timeout=<ms> rlimit=<n>` with an empty environment in a fresh empty directory, sends `(set-option :print-success false)` first, prefixes every query with `(reset)` and that option again, and brackets every write with an echo barrier whose quoted reply it must see before it reads the next frame; Section 32.8.1 gives the exact stream. The two differences that most often turn a working shell test into an embedded failure are `smtlib2_compliant=true` (it changes response formatting, for example echo replies keep their double quotes) and the missing `print-success` suppression (a worker that answers success to every command fails the readiness handshake).

33.9 The query unexpectedly returns unknown

Ask Z3 for a reason in a standalone diagnostic context if the profile supports it, but preserve the exact embedded failure category. Potential causes:

- theory/profile outside a complete fragment;
- solver timeout or resource limit;
- nonlinear arithmetic or difficult quantifiers;
- fixedpoint engine limitation;
- unsupported combination of theories;
- cancellation or host deadline reported separately by the application.

Current Length is QF-LIA and should ordinarily be decidable; repeated solver unknown on small canonical queries is therefore a strong capability/configuration signal. Standalone, `(get-info :reason-unknown)` after the check names the cause; because the pinned profile passes the solver limits as `timeout=<ms>` and `rlimit=<n>` on the command line, the two answers to expect are `(:reason-unknown "timeout")` and `(:reason-unknown "max. resource limit exceeded")` (both reproduced with Z3 5.0.0). Raise `execution.solverTimeoutMilliseconds` (cap 60 000) or `execution.solverResourceLimit` (cap 10 000 000) in the startup file, remembering that `hostDeadlineMilliseconds` must exceed the solver timeout by at least 100 ms and is capped at 65 000. The embedded worker never asks for the reason: an unknown stays a byte-free heuristic status.

33.10 The candidate is rejected in filter mode but seems correct

Because rejection requires replayed evidence, inspect the sanitized rejection presentation and, under an authorized diagnostic path, the counterexample input. Then:

1. evaluate the exact accepted candidate at that modeled input;
2. verify the contract precondition is applicable;
3. verify the postcondition really fails in the Length model;
4. inspect provider laws used by the candidate;
5. confirm the configured argument roles and spine constructors;
6. distinguish source-level intent from the actual contract text.

The most likely explanation may be a contract/law mistake rather than Z3. If independent replay is wrong, the defect is in the domain interpreter or evidence association and should be treated as a high-priority correctness bug.

33.11 The candidate seems behaviorally wrong but survives

Possible explanations include:

- the chosen observation is too weak, as with reverse under length preservation;
- the violating source feature is outside the modeled fragment;
- handoff refused, so no sound problem was constructed;
- no counterexample was found within operational limits;
- Z3 returned unknown or failed;
- the contract precondition excludes the suspected input;
- an assumed provider law masks the behavior;
- filter mode conservatively retains uncertain candidates.

A survivor means “not rejected under this exact configured evidence policy,” not “universally correct.”

33.12 Leant refuses at startup or at the prompt

Every refusal before a worker exists is a pure decision printed with its constructor name; there is no hidden state to inspect. Three places can refuse.

Startup.

`src/Leant/Options.hs` accepts `--length-ranking-configPATH`, `--length-ranking-config-timeoutMS` (1...60 000, default 5 000; refused without the config option), and `--length-ranking-allow-unpinned` (likewise). `src/Main.hs` then calls `loadLengthAssessmentMode`; any `Left` prints error: `finite-spine Length counterexample ranking setup failed:` followed by one of `LengthAssessmentFileAdmissionRejected` (path length, empty path, embedded NUL, relative path, timeout out of range: no IO has happened), `LengthAssessmentFileLoadRejected` (platform unsupported, open/inspect/read failure, not a regular file, more than 256 KiB, deadline, cleanup, or the decoder’s `LengthRankingConfigurationFileError`), or `LengthAssessmentActivationRejected` `LengthRankingConfigurationExecutablePinRequired`, and exits with status 1. The decoder validates in a fixed order: bounded JSON, format (`leant-live-length-ranking-configuration`), rankingDomain (scalar or binary-product), the exact ten-member root, then `executionAdmission`, `execution`, `evaluation`, `inputBoxValidation`, `applicableDomainValidation`, `counterexampleSimplification`, `usableWorkBudget`, and finally the domain-selected contract. An unexpected member is reported before a missing one, so a leftover historical version key surfaces as `LengthRankingConfigurationUnexpectedField LengthRankingConfigurationRootObject`. Success prints the enabled-mode banner, the launch-strategy line, and the engine hint, and nothing else; no process has been started.

The :synth line.

`src/Leant/Synth/Length/Command.hs` parses only two exact tokens. `--behavior-mode` without a following word is `LengthSynthCommandBehaviorModeMissing`; a word other than `rank/filter` is `...BehaviorModeInvalid`; either option without a later standalone `--` is `...DelimiterMissing`; `--length-contract` directly followed by `--` is `...ContractPathMissing`; and a mode token after the contract option is `...BehaviorModeMustPrecedeContract`. `Main` prints the single usage: line for all five. Longer look-alikes such as `--behavior-model` are ordinary goal text and are handed to Lean.

Request authorization.

Before any contract path is admitted, `src/Leant/Synth/Length/Integration.hs` asks the activated mode for permission. With assessment disabled, `--behavior-modefilter` yields `LengthAssessmentFilteringRequiresActivatedPolicy` and `--length-contract` yields `LengthAssessmentExplicitContractRequiresActivatedPolicy`; both are printed after `finite-spine Length` request rejected before `IO`. Only then is a named contract file admitted (one-shot `finite-spine Length` contract admission failed:) and read (... load failed:); its decoder mirrors the startup one but expects the three-member `leant-finite-list-spine-length-contract` root and can never supply execution authority.

33.13 A batch-wide warning says assessment preserved all verified candidates

This warning is the only place operational failure becomes visible. `src/Main.hs` prints warning: finite-spine Length behavioral assessment preserved all verified candidates: followed by the `LengthAssessmentFailure` constructor, and then displays the callback batch in original order with no notes; in filter mode nothing is rejected. The wrapper says which layer failed (`LengthAssessmentPostVerificationFailed`, `LengthAssessmentRankingFailed`, `LengthAssessmentSelectionFailed`, or their `SpinePair` siblings), and the payload is a `LengthRankingFailure` whose three positional fields are the class, whether Djex reported incomplete cleanup, and the zero-based index of the candidate whose transaction failed (or `Nothing` for opener/finalizer failures). For example:

```
warning: finite-spine Length behavioral assessment preserved all verified candidates: LengthAssessmentRankingFailed (
  LengthRankingFailure (LengthRankingLiveSessionFailed LengthSMTLibLiveSessionLaunchFailed) False Nothing)
```

Listing 33.1: Shape of a batch-wide warning after a live-session refusal.

The class vocabulary is closed (`src/Leant/Synth/Length/Ranking/Internal.hs`, `LengthRankingFailureClass`):

LengthRankingLiveSessionFailed

wraps a `LengthSMTLibLiveSessionFailure` from opening, probing, or closing the scope: `DeadlineExceeded`, `WorkspaceUnavailable` (the fresh empty directory could not be created), `ExecutableUnavailable`, `ExecutableRejected` (not regular, not executable, metadata changed between inspection and spawn, digest mismatch, or an executable-access/exec check denied), `LaunchFailed` (descriptor-bound launch unavailable on this build or kernel, staging or exec failure, spawn or pipe failure), `CapabilityRejected` (readiness handshake failed), `ResourceLimitExceeded`, `TransportFailed`, `CleanupFailed`, `InternalFailure`, and `UsableWorkScopeUnavailable`. The mapping from Djex's fine-grained process errors to these classes is `sessionProcessFailure` in `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs`.

LengthRankingLiveQueryFailed

wraps a `LengthSMTLibLiveQueryFailure` for one transaction: `SessionUnavailable`, `LimitExceeded` 64 65 (the shared per-scope query budget), `DeadlineExceeded`, `ConfigurationRejected`, `ResourceLimitExceeded`, `TransportFailed`, `ProtocolRejected` (framing, response limits, unexpected reply), `CounterexampleRejected` (the model failed replay or its bindings were malformed), and `InternalFailure`.

LengthRankingQueryAssociationMismatch, LengthRankingEvidenceReplayMismatch

an observation or receipt did not belong to the query or problem it was presented for; both indicate a Djex/Leant defect rather than a configuration problem.

LengthRankingOriginProbeEvaluationFailed, ...InputBoxValidationFailed, ...ApplicableDomainValidationFailed, ...CounterexampleSimplificationFailed

a pure bounded evaluation exceeded its configured evaluation, box, domain, or simplification limits.

Two properties of the fallback are worth internalizing. First, it is atomic per batch: one failing transaction discards every earlier pure or live assessment of that batch, so a sat you saw standalone can vanish because a *later* candidate's query timed out. Second, it is not a transaction over the filter bank: replays, promotions, and recordings that completed before the failure stay committed, so the next batch of the same command may still reject a duplicate from the bank even though this batch printed the warning. A `LengthPostVerificationFailed` wrapper with `LengthRankingInputLimitExceeded` 64 65 means a library caller submitted more than 64 verified candidates in one batch; Main never does.

33.14 Reading a real assessment note

Every semantic note in the REPL is produced by one of eight renderers in `src/Leant/Synth/Length/Presentation.hs`, so its shape tells you which evidence class produced it. Main prints a survivor as an `itN` row with the note indented beneath it, and an omitted occurrence as a rejected row whose note is always present. Illustratively, for a length-preservation contract:

```
it1 fun xs => xs
it2 fun xs => xs.reverse
rejected fun _ => []
      replayed finite-list-spine Length counterexample (model-relative; provider-independent): observed input spine
      lengths = [1]; result spine length = 0
```

```
rejected fun xs => xs ++ xs
    replayed finite-list-spine Length counterexample (model-relative; conditional on 1 assumed provider law used by
    this candidate): observed input spine lengths = [1]; result spine length = 2
```

Listing 33.2: Note shapes as rendered by the pinned Presentation module (candidate texts illustrative).

How to read each shape:

- `replayed finite-list-spine Length counterexample (model-relative; ...)`: an independently replayed Counterexample receipt. The parenthesis names the basis: provider-independent when the candidate used no provider law, otherwise conditional on N assumed provider law(s) used by this candidate; provider names are deliberately reduced to a count. Then the source-ordered input lengths (at most eight, then ...) and the result length; the binary-product family prints first result spine length and second result spine length. In rank mode this note sits under a still-visible, still-bound demoted candidate; in filter mode under a rejected row with no itN.
- `replayed bounded query-owned componentwise-lexicographic finite-list-spine Length counterexample (...)`: inspected lower-box assignments = N; input spine lengths reduced from [...] to [...]; result spine length = R; the simplifier found a strictly smaller replayed counterexample; the final vector is the one that entered the bank.
- `independently checked finite-list-spine Length input box (bounded/model-relative; ...)`: inclusive input maxima = [...]; checked assignments = N; applicable assignments = M: a BoundedPositive receipt after a live unsat; if M is zero the suffix ; vacuous within this box (no assignment met the precondition) is added and the receipt is neutral rather than preferred.
- `complete finite-spine Length Boolean finite-union atomic-branching recursive piecewise-affine domain ... (model/provider-relative; ...; no global proof or solver authority)`: boxes = ...; visits = ...; unique = ...; applicable = ...; maxima = [...]: an ApplicableDomainEstablished receipt from the pure validator, with the same vacuity suffix rule.
- No note: heuristic status, unassessed, preparation refused, disabled assessment, or preserve-all fallback. The renderer never distinguishes these for the user.

Every note is cut at 384 characters, and a natural above 24 decimal digits is shown as `<N-bit natural>`. Nothing in a note is authority: rejection came from the selection adapter's replay-only rule and the sealed occurrence association, and the note is projected from the same opaque wrapper, never matched back by text.

A concept crosswalk for Lean, Djex, and Z3

34.1 Types and sorts

A Lean type and an SMT sort play analogous organizational roles but are not interchangeable.

Concept	Lean/Djex side	Z3/SMT side
Type/sort	Dependent types, universes, inductives, propositions, functions; Djex schemes and private nominal identities	First-order sorts such as Bool, Int, bit-vectors, arrays, sequences, datatypes, uninterpreted sorts
Term	Lean expression or checked Djex candidate graph	First-order SMT expression
Binder	Dependent/ordinary lambda, forall, implicit binder	First-order constant declaration or quantifier variable
Function	May be higher-order, dependent, recursive, effectful through explicit structures	Usually total first-order function symbol or interpreted theory operation
Equality	Typed propositional equality in Lean	Built-in equality over one SMT sort
Proof	Lean term checked by kernel	Solver result, optional proof object, model, core, or fixedpoint answer; not automatically a Lean proof
Type class	Elaborator search and instance terms	No corresponding automatic authority; can be encoded only as explicit data/constraints
Inductive data	Lean inductive declaration with eliminators and recursion	SMT algebraic datatype or a domain abstraction
Natural number	Lean Nat with source semantics	Commonly SMT Int plus nonnegativity, or a custom datatype/bit-vector depending on domain
Candidate identity	Exact typed graph occurrence plus inventory/provenance	Selector vector or formula symbols only after explicit association
Truncated subtraction	Lean <code>Nat.sub</code> ($n - 1$ at type Nat is 0 when $n = 0$); Djex <code>monus</code>	No primitive: SMT <code>-</code> on Int goes negative, so the translator defines <code>djex_nat_monus</code> as <code>(ite (<= y x) (- x y) 0)</code>
Assumption	A hypothesis or <code>axiom</code> ; a Djex provider law is an explicit assumed length transfer	An assert, or an assumption literal in <code>check-sat-assuming</code> ; the solver never validates it
Decision procedure	<code>omega</code> (linear arithmetic over Nat/Int), <code>decide</code> (kernel evaluation of a Decidable instance)	<code>check-sat</code> under <code>QF_LIA</code> ; the closest Djex analogue of <code>decide</code> is exhaustive input-box validation, not Z3
Counterexample	A concrete Nat vector plus a candidate that Djex re-evaluates; Lean itself has no “model” notion	A satisfying assignment read back with <code>get-value</code> , evidence only after Djex replay

34.2 Elaboration versus solving

Lean elaboration fills implicit arguments, resolves overloaded names, synthesizes type-class instances, infers universes, inserts coercions, and checks expected types. Z3 solving assigns values to already declared first-order symbols so that asserted formulas hold.

Trying to make Z3 elaborate Lean would require encoding the environment and elaborator. The architecture instead lets each tool do what it is designed for: Djex prepares typed alternatives, Lean elaborates materialized source, and Z3 solves the bounded semantic choices.

34.3 Theorem proving versus counterexample search

Lean constructs a proof term for a proposition. Z3's ordinary solver establishes satisfiability or unsatisfiability in an SMT theory. A common bridge is:

$$\text{prove } \forall x. \varphi(x) \quad \leadsto \quad \text{check whether } \exists x. \neg \varphi(x) \text{ is satisfiable.}$$

If sat, the model suggests a refutation witness and replay can confirm it. If unsat, the encoded negation has no model, but transporting that fact into Lean requires a trusted checker, generated proof, or explicit external-solver axiom.

34.4 Model versus value

A Z3 model is an interpretation for every relevant symbol, often with arbitrary values for unconstrained parts. A Lean value is a term inhabiting a type, with semantics defined by Lean reduction and declarations. Decoding a model into a domain value is a checked partial function, not a cast.

For current Length, decoding is simple: exact named SMT integers become source-ordered naturals. For future sequences, bit-vectors, datatypes, or uninterpreted functions, decoding becomes domain-specific and may produce a finite symbolic object rather than a concrete Lean term.

34.5 Unsat core versus explanation

An unsat core is a subset of tracked assumptions sufficient for unsatisfiability. It is not guaranteed minimal, and its symbols are meaningful only through the application's source map. A good Leant explanation adds:

- stable contract/production labels;
- source spans or exact provider occurrences;
- a deterministic minimization attempt;
- the finite/model scope;
- a plain-language statement of the conflict;
- optionally a small witness showing why two desired properties cannot coexist.

34.6 Optimization versus synthesis quality

Z3 can minimize a numeric objective over the encoded grammar. Human notions such as readability, idiomatic Lean, abstraction quality, or maintainability are only captured if represented by stable costs or post-ranking. A lexicographic objective should be explicit rather than compressed into one arbitrary weighted sum.

34.7 Fixedpoint safety versus induction theorem

Spacer can infer a relation invariant and show a bad predicate unreachable in the CHC model. Lean induction proves a proposition about an exact recursive definition. The intended bridge is to translate the invariant and rule obligations into Lean, not to identify the two judgments by terminology alone.

34.8 Lean automation versus Z3

Three Lean tactics are the natural points of comparison, and confusing them with the solver is a common source of miscommunication.

omega

is a decision procedure for linear arithmetic over `Nat` and `Int`: literal coefficients, truncated `Nat` subtraction, `min/max`, and division and remainder by literals, with case splits performed internally. That is essentially the fragment the `Length` translator emits, so the arithmetic obligation of a `Length` contract, once the candidate's length behavior has been stated as a Lean lemma, is a natural `omega` target. The difference is authority, not power: `omega` produces a kernel-checked proof term, while `Z3` returns `unsat` for the negation and no artifact Lean can check.

decide

evaluates a `Decidable` instance by kernel reduction and therefore proves only what a finite computation can settle. Its Djex analogue is exhaustive input-box or applicable-domain validation, which is why those receipts are called “bounded-positive”: the analogue of `decide` over an infinite domain does not exist on either side.

simp

rewrites with an equational theory. Its solver counterpart is `Z3`'s preprocessing and the `define-fun` helpers, not the decision procedure; a `Length` provider law is closer to a `simp` lemma that Lean has *not* proved than to anything `Z3` does.

The proposed positive path of Chapter 30 can be summarized in this vocabulary: `Z3` finds the invariant or shows the bad state unsatisfiable, and Lean's `omega/decide/simp` (or an induction the invariant guides) discharges the translated obligations. Neither replaces the other.

Checkpoint

When moving a fact across tools, always name the conversion: `render`, `elaborate`, `seal`, `encode`, `decode`, `replay`, `materialize`, or `prove`. “`Z3` knows the Lean term is correct” skips every boundary that makes the system trustworthy.

SMT-LIB quick reference

A.1 Lexical structure

SMT-LIB is based on S-expressions.

```
(command argument1 argument2)
(operator term1 term2)
```

Comments begin with a semicolon and continue to the end of the line.

```
; this is a comment
(assert (= x 3)) ; this is also a comment
```

Common atoms:

Kind	Example	Notes
Simple symbol	x, input.0, seq.len	Must obey symbol syntax; punctuation is allowed in many positions.
Quoted symbol	a symbol with spaces	Vertical bars quote an identifier; escapes follow SMT-LIB rules.
Keyword	:produce-models	Begins with a colon and names an option or attribute.
Numeral	0, 42	Nonnegative decimal numeral; negative integer terms are written (- 42).
Decimal/rational	1.5, (/ 1 3)	Exact arithmetic depends on the target sort.
String	"hello"	Used by commands such as echo; escaping follows the standard.
Bit-vector	##b1010, ##x0A	Binary or hexadecimal fixed-width literal.

A.2 Core commands

Command	Purpose
(set-logic QF_LIA)	Select a declared logic/profile.
(set-option :produce-models true)	Request model production after satisfiable checks.
(declare-const x Int)	Declare a constant, shorthand for a zero-argument function.
(declare-fun f (Int Int) Int)	Declare a total function symbol.
(define-fun f ((x Int)) Int ...)	Define a nonrecursive function by an expression.
(assert phi)	Add a hard formula to the current scope.
(push 1) / (pop 1)	Create/remove assertion scopes.

Command	Purpose
(check-sat)	Ask whether current assertions are satisfiable.
(check-sat-assuming (a b))	Check with temporary Boolean assumption literals.
(get-value (x y))	Request values of exact terms after sat.
(get-model)	Request a model, often broader and less stable than exact get-value.
(get-unsat-core)	Request a core after an unsatisfiable tracked/assumption check.
(reset)	Reset declarations, assertions, and options to a fresh solver state; the pinned Djex profile sends it before every query.
(reset-assertions)	Remove assertions; the standard also drops declarations unless :global-declarations is true, whereas Z3 keeps them (verified: a constant declared before reset-assertions is still usable afterwards).
(set-option :print-success false)	Suppress the success reply Z3 otherwise prints after each non-query command in compliant mode; the pinned profile sends it at startup and after every reset.
(echo "text")	Print the string; in smtlib2_compliant=true mode the reply keeps its double quotes. Djex uses (echo "djex-smtlib-frame/v1/<64 hex>") as a causal barrier after every write.
(get-info :reason-unknown)	After unknown, name the cause, e.g. (:reason-unknown "timeout").
(get-option :produce-models)	Read an option back; useful when checking which options a script actually set.
(minimize cost)	Add an optimization objective.
(assert-soft phi :weight 3)	Add a weighted soft constraint in Optimize.
(declare-rel R (Int Int))	Declare a fixedpoint relation.
(rule phi)	Add a Horn/fixedpoint rule.
(query Bad)	Ask whether a fixedpoint relation is reachable.
(exit)	Request orderly solver termination.

A.3 Boolean terms

```
(and p q r)
(or p q)
(not p)
(=> p q)
(= x y)
(distinct x y z)
(ite p then-term else-term)
```

$$p \wedge q \leftrightarrow (\text{and } p \ q),$$

$$p \vee q \leftrightarrow (\text{or } p \ q),$$

$$\neg p \leftrightarrow (\text{not } p),$$

$$p \Rightarrow q \leftrightarrow (=> \ p \ q).$$

A.4 Integer and real arithmetic

```
(+ x y z)
(- x)      ; unary negation
(- x y)    ; subtraction
(* 3 x)    ; linear literal scaling
(div x 3)
(mod x 3)
(<= x y)
(< x y)
(>= x y)
(> x y)
```

In QF_LIA, multiplication must remain linear: at most one operand contains variables. Z3 does not police this for you: (set-logic QF_LIA) followed by (assert (= (* x y) 6)) is accepted and answered sat (verified with Z3 5.0.0), so a translator must enforce its own profile. The current Length translator lowers its admitted natural quotient/modulo operations to Euclidean witnesses rather than emitting div/mod.

A.5 Arrays

```
(declare-const a (Array Int Int))
(select a i)
(store a i v)
((as const (Array Int Int)) 0)
```

Arrays are total mathematical maps. The constant-array form `((as const (Array Int Int)) 0)` is a Z3 extension rather than part of the SMT-LIB standard. A sequence represented by array plus length needs explicit bounds guards around every semantically meaningful read.

A.6 Bit-vectors

```
(declare-const x (_ BitVec 32))
(bvadd x #x00000001)
(bvand x #x000000ff)
(bvshl x #x00000003)
(bvult x y) ; unsigned comparison
(bvslt x y) ; signed comparison
((_ extract 7 0) x)
((_ zero_extend 8) x)
```

Width is part of the sort. Signedness is chosen by operators, not stored in the bit-vector value.

A.7 Algebraic datatypes

```
(declare-datatypes ((Option 1))
  ((par (T) ((none) (some (value T))))))

(declare-const x (Option Int))
((_ is some) x)
(value x)
```

A selector such as `value` is meaningful only under the corresponding constructor guard in source-oriented semantics.

A.8 Quantifiers

```
(forall ((x Int)) (>= (* x x) 0))
(exists ((x Int)) (= (+ x x) 10))
```

Quantifiers can move a problem outside a complete or predictable fragment. Prefer existential bad witnesses, finite expansion, domain-specific elimination, or CHCs when those transformations are exact.

A.9 Typical responses

```
sat
unsat
unknown

((x 1) (y (- 2)))
```

```
(error "diagnostic text")
```

Three error responses are worth recognizing on sight (all reproduced with Z3 5.0.0): (error "line 7 column 14: model is not available") after get-value follows an unsat check; (error "line 2 column 11: unknown constant m") when a term names an undeclared symbol; and (error "line 4 column 11: the logic has already been set") when set-logic is repeated without a reset. Note that Z3 answers get-value after sat even when :produce-models was never set, so a script that “works” may still be relying on a default the pinned Djex preamble sets explicitly.

An application parser should not assume that one operating-system read equals one response. It must bound and frame the stream.

A.10 Logic-name orientation

Logic	Main theories	Typical Leant/Djex use
QF_LIA	Quantifier-free linear integer arithmetic	Current Length verification and many finite selector/cost constraints
QF_LRA	Quantifier-free linear real arithmetic	Rational/real affine templates and bounds
QF_UF	Quantifier-free uninterpreted functions	Pure provenance and argument-wiring relations
QF_AUFLIA or nearby combinations	Arrays, uninterpreted functions, linear integers	Array-plus-length sequence models and finite tables
QF_BV	Quantifier-free bit-vectors	Exact machine arithmetic, masks, shifts, overflow
QF_ABV	Arrays plus bit-vectors	Memory/table and low-level state models
HORN	Constrained Horn clauses/fixedpoint	Recursive relational safety and invariant inference
ALL	No restrictive declared logic	Convenient interactively, undesirable for a tightly finger-printed production profile

Common mistake

Logic names describe a syntactic/theory fragment, not a performance guarantee. Z3 may support formulas outside the declared standard logic, but a production translator should emit only its checked profile and fail when it cannot.

Complete small scripts

The scripts below are self-contained learning examples. They illustrate the mathematical protocol, not the exact private symbol names or framing used by Djex. Expected results follow from the formulas; model values can vary where several models exist.

B.1 A satisfiable arithmetic puzzle

```
(set-option :produce-models true)
(set-logic QF_LIA)

(declare-const x Int)
(declare-const y Int)
(assert (>= x 0))
(assert (>= y 0))
(assert (= (+ x y) 10))
(assert (> x y))

(check-sat)
(get-value (x y))
```

Listing B.1: Find two nonnegative integers with a given sum.

Expected status: sat. One possible model is $x = 6$, $y = 4$; many others satisfy the constraints, and Z3 5.0.0 actually answered $((x\ 10)\ (y\ 0))$, which is why a production decoder must not hard-code the model it “expects”.

B.2 Proving a linear implication by refuting its negation

Claim:

$$x \geq 0 \wedge y \geq 0 \Rightarrow x + y \geq 0.$$

Search for a counterexample:

```
(set-logic QF_LIA)
(declare-const x Int)
(declare-const y Int)
(assert (>= x 0))
(assert (>= y 0))
(assert (not (>= (+ x y) 0)))
(check-sat)
```

Expected status: unsat. In an application, this supports the encoded implication; it is not automatically a Lean proof term.

B.3 Length preservation: constant empty candidate

```
(set-option :produce-models true)
(set-logic QF_LIA)

(declare-const input.0 Int)
(assert (>= input.0 0))

; Candidate result length is 0.
; Bad state for required result = input.0:
(assert (not (= 0 input.0)))

(check-sat)
(get-value (input.0))
```

Expected status: sat. A smallest replayable counterexample is $\text{input.0} = 1$.

B.4 Length preservation: identity candidate

```
(set-logic QF_LIA)

(declare-const input.0 Int)
(assert (>= input.0 0))

; Candidate result length is input.0.
(assert (not (= input.0 input.0)))

(check-sat)
```

Expected status: unsat.

B.5 Length preservation: exact tail behavior

A finite-spine tail returns length 0 at input 0 and $n - 1$ otherwise.

```
(set-option :produce-models true)
(set-logic QF_LIA)

(declare-const n Int)
(assert (>= n 0))

(define-fun monus ((x Int) (y Int)) Int
  (ite (<= y x) (- x y) 0))

(define-fun tailLength ((x Int)) Int
  (ite (= x 0) 0 (monus x 1)))

(assert (not (= (tailLength n) n)))
(check-sat)
(get-value (n))
```

Expected status: sat. $n = 1$ is a counterexample.

B.6 Euclidean quotient/remainder witnesses

```
(set-option :produce-models true)
(set-logic QF_LIA)

(declare-const q Int)
(declare-const r Int)
(assert (>= q 0))
(assert (>= r 0))
(assert (< r 3))
(assert (= 8 (+ (* 3 q) r)))

(check-sat)
(get-value (q r))
```

Expected status: sat with $q = 2, r = 2$. The bounds make the pair unique.

B.7 Assumption literals and an unsat core

```
(set-option :produce-unsat-cores true)
(set-logic QF_UF)

(declare-const p Bool)
(declare-const irrelevant Bool)
(declare-const c.positive Bool)
(declare-const c.negative Bool)
(declare-const c.extra Bool)

(assert (=> c.positive p))
(assert (=> c.negative (not p)))
(assert (=> c.extra irrelevant))

(check-sat-assuming (c.positive c.negative c.extra))
(get-unsat-core)
```

Expected status: unsat. A sufficient core contains `c.positive` and `c.negative`; a solver is not required to return a minimal core.

B.8 A tiny optimization problem

```
(set-option :produce-models true)
(set-logic QF_LIA)

(declare-const selector Int)
(assert (and (<= 0 selector) (< selector 3)))

; Alternative costs: 2, 0, and 5.
(define-fun cost () Int
  (ite (= selector 0) 2
    (ite (= selector 1) 0 5)))

(minimize cost)
(check-sat)
(get-value (selector cost))
```

Expected optimum: `selector = 1, cost = 0`. A production system must fingerprint objective order and tie-breaking.

B.9 A bit-vector overflow witness

For 8-bit unsigned addition, find x, y such that the wrapped sum is smaller than x .

```
(set-option :produce-models true)
(set-logic QF_BV)

(declare-const x (_ BitVec 8))
(declare-const y (_ BitVec 8))
(assert (bvult (bvadd x y) x))

(check-sat)
(get-value (x y (bvadd x y)))
```

Expected status: sat. Many overflow pairs exist, such as $x = 255, y = 1$ producing 0; Z3 5.0.0 returned the mirror image ((x ##x01) (y ##xff) ((bvadd x y) ##x00)).

B.10 A fixedpoint safety query

```
(set-logic HORN)

(declare-rel Eval (Int Int))
(declare-rel Bad ())
(declare-var n Int)
(declare-var r Int)

(rule (Eval 0 0))
(rule (=> (and (>= n 0) (Eval n r))
        (Eval (+ n 1) (+ r 1))))
(rule (=> (and (>= n 0) (Eval n r) (not (= n r)))
        Bad))

(query Bad)
```

Expected fixedpoint result: unsat for reachability of Bad, because the inductive relation satisfies $r = n$.

An end-to-end Length counterexample walk-through

C.1 Target and contract

Take the target type

$$\text{List } \alpha \rightarrow \text{List } \alpha$$

and contract

$$\forall xs. \quad \text{length}(C(xs)) = \text{length}(xs).$$

The modeled input is

$$n = \text{length}(xs) \in \mathbb{N}.$$

The precondition is true and the postcondition is $r = n$.

C.2 Candidate

Consider the exact candidate

```
fun xs => xs.tail
```

The precise available Lean operation may differ by API and proof of nonemptiness; this listing names the intended finite-spine behavior. The Length interpreter does not execute arbitrary Lean source. It derives the modeled behavior from an exact case graph or an admitted provider transfer.

The modeled result is

$$R(n) = \begin{cases} 0, & n = 0, \\ n - 1, & n > 0. \end{cases}$$

Using monus:

$$R(n) = \text{ite}(n = 0, 0, n \dot{-} 1).$$

C.3 Bad state

Substitute $R(n)$ into the negated postcondition:

$$n \geq 0 \wedge \text{ite}(n = 0, 0, n \dot{-} 1) \neq n.$$

At $n = 0$, both sides are 0. At $n = 1$, the left side is 0 and the right side is 1. Therefore the formula is satisfiable.

C.4 Canonical translation stages

The real translator conceptually performs:

1. validate the exact input arity;
2. map the input to a canonical generated symbol;
3. translate monus through the fixed helper definition;
4. translate the conditional and equality;
5. prepend input nonnegativity;
6. append check-sat;
7. render under command-byte and numeral-bit limits;
8. construct the separate input-value request;
9. fingerprint the typed plan and rendered bytes with the exact problem.

Section 32.11 shows the bytes this produces for the simpler constant-empty candidate under the same contract; the tail candidate differs only in its bad-state assertion, which uses the `djex_nat_monus` helper from the preamble.

C.5 Live transaction

The scoped worker:

1. has already passed executable admission and capability probing;
2. resets to a known state;
3. receives the check program;
4. returns sat under causal framing;
5. receives get-value for the exact input symbol;
6. returns a binding, for example $n = 1$;
7. does not expose the binding as evidence directly.

C.6 Response decoding

The decoder checks:

- one binding was expected and one was present;
- its symbol is exactly the generated input symbol;
- the symbol is neither duplicate nor unknown;
- the value is an admitted integer numeral within bounds;
- the value is nonnegative and therefore converts to `Natural`.

C.7 Independent replay

The evaluator receives $n = 1$ and recomputes:

$$R(1) = 0, \quad P(1) = \top, \quad Q(1, 0) \equiv 0 = 1 = \perp.$$

It constructs candidate-specific behavioral evidence and associates it with the exact problem/query identities.

C.8 Ranking result

In rank mode, the candidate remains present but is stably placed after candidates without replayed counterexamples. Presentation may display a bounded/sanitized note identifying the Length violation class and input.

C.9 Filter result

In filter mode, the private Length selection builder converts the exact replayed evidence into a rejection decision. The generic selection seal verifies that this verified occurrence appears exactly once in the selected/rejected partition. The candidate is omitted from the selected output and reported among rejections.

C.10 Bank reuse

The input vector [1] may be stored in the command-local bank. For a later duplicate candidate D :

1. [1] is replayed against D 's fresh checked query;
2. if it violates D , fresh evidence is produced without a live query;
3. if it does not, the bank sample carries no verdict and assessment proceeds;
4. a hit may promote the sample in the bounded MRU order.

C.11 What was and was not established

Established:

- Lean accepted the exact candidate at the target type;
- the exact candidate was soundly interpreted in the configured Length domain;
- input length 1 independently reproduces a length-preservation violation;
- rank/filter association refers to the exact verified occurrence.

Not established:

- that a particular concrete Lean list value was executed;
- that every possible behavioral contract fails;
- that payload elements have any property;
- that a different provider law or spine model would yield the same result;
- that Z3's raw model alone was trusted.

Glossary

Term	Meaning in this document
Assertion	A formula added to an SMT solver state by <code>assert</code> . It constrains every satisfying model of the current scope.
Assumption literal	A temporary Boolean literal passed to <code>check-sat-assuming</code> , often guarding a production choice or contract clause so it can appear in an unsat core.
Behavioral contract	An explicit precondition/postcondition or related specification interpreted in a named observation domain.
Behavioral domain	A checked abstraction defining modeled source values, candidate semantics, contract vocabulary, query encoding, model decoder, replay, and evidence strength.
Bad state	The existential counterexample condition $P \wedge \neg Q$ after candidate semantics is substituted.
Bounded-positive receipt	Evidence that an explicitly finite admitted scope was exhaustively checked with no counterexample; not a universal proof outside that scope.
Callback verification	Leant's re-elaboration of a rendered candidate against the exact Lean target.
Candidate occurrence	One position/instance in a verified batch, even if another occurrence has the same text or semantic graph.
Canonical	Produced by one deterministic representation/order policy so structural identity and reproducibility can be checked.
CEGIS	Counterexample-guided inductive synthesis: solve for a candidate satisfying known samples, search for a new counterexample, add it, and repeat.
CHC	Constrained Horn clause, a rule form used by fixedpoint engines for recursive relational reasoning.
Complete search/result	A claim that all members of a precisely stated fragment/domain were covered. Completeness never follows merely from exhausting a practical budget.
Constraint	A logical condition that candidate values, model variables, or selectors must satisfy.
Counterexample	A decoded input or trace that independent domain replay confirms satisfies the precondition and violates the postcondition for the exact candidate problem.
Counterexample bank	A bounded fingerprint-scoped collection of input vectors for fresh replay against later candidates. It stores no reusable verdict authority.
Decoder	A bounded parser/converter from solver response syntax to a domain-owned payload.
Exact typed origin	Leant's opaque sidecar tying one accepted spelling to the exact Djex candidate graph, inventory, goals, provider bindings, and renderer identity.
Exactness descriptor	Structured metadata stating source-fragment coverage, abstraction direction, finite scope, provider assumptions, and evidence/proof level.
Fingerprint	Structural identity of a checked artifact, including its schema and all semantically relevant components. In Djex it is the complete versioned canonical encoding of those components (magic DJEXFP, tag-length-payload fields), so it is collision-free by construction and decodable; it is not a lossy hash, and its <code>Show</code> form is the lowercase hex of the whole key.
Fixedpoint	A solver mode for relations and Horn rules; Spacer is Z3's principal property-directed engine in this area.

Term	Meaning in this document
Grammar	The finite or recursively bounded family of candidate constructions made available to search.
Heuristic status	A solver status associated with an exact query but not upgraded to independent behavioral evidence.
Interpretation	An assignment of meanings to symbols. A satisfying interpretation is a model.
Invariant	A property preserved by every modeled transition/recursive step and strong enough to imply safety.
Materialization	Conversion of a checked selector/model assignment into one exact Djex candidate graph and then a Lean spelling.
Model	A satisfying interpretation returned or represented by Z3. It may assign arbitrary values to unconstrained symbols.
Model-relative	True in the explicitly selected abstraction/theory and assumptions, not automatically a theorem about all source behavior.
Monus	Truncated natural subtraction $a \dot{-} b = \max(a - b, 0)$.
Nogood	A learned clause forbidding a combination of production choices already shown infeasible under a scoped grammar/sample state.
Observation	Information retained by a domain, such as list length, root tag, array index value, provenance, or cost.
Opaque evidence	A value whose constructor is private and can be obtained only through an authorized replay or proof gate.
Over-approximation	A summary containing every concrete behavior and possibly extra behaviors. Safe for impossibility/pruning when even the summary cannot satisfy the goal.
Postcondition	The contract condition required of a candidate result when the precondition holds.
Precondition	The contract condition defining the applicable input domain.
Provider	A declaration/operation from the synthesis inventory available for candidate construction.
Provider law	An explicit assumed or proved semantic transfer for one exact provider occurrence/scheme.
QF_LIA	Quantifier-free linear integer arithmetic, the exact current Length query logic.
Replay	Independent re-evaluation of decoded solver data through the exact retained domain problem.
Role vector	Source-ordered declaration of how target arguments are modeled; roles are explicit, not inferred from names.
SAT	Boolean satisfiability, or colloquially the satisfiable result; SMT extends it with theory semantics.
Satisfiable	At least one model makes every assertion true. Z3 prints sat.
Selection seal	Generic occurrence-respecting proof that a verified batch was partitioned into selected and rejected outputs exactly once each.
Selector	A finite-domain variable naming one alternative at a sketch choice node.
Semantic signature	A vector of candidate observations on the current sample bank, used to partition behaviorally indistinguishable candidates.
Sketch	A Djex-prepared type-correct partial program with finite holes for Z3 to select.
SMT	Satisfiability modulo theories: satisfiability with arithmetic, arrays, bit-vectors, datatypes, uninterpreted functions, and other interpreted domains.
SMT-LIB	Standard language and command protocol used to express SMT problems and responses.
Sort	An SMT value category such as Bool, Int, a bit-vector width, an array sort, a datatype, or an uninterpreted sort.
Theory	A fixed semantic interpretation and axioms for operations such as integer addition, arrays, equality with uninterpreted functions, or bit-vectors.
Under-approximation	A summary containing only realizable behaviors and possibly missing others. Safe for witnesses, not for proving impossibility.
Unknown	Z3 could not return sat or unsat under the selected algorithm/resources. It is not a third truth value of the formula.
Unsatisfiable	No model satisfies all assertions. Z3 prints unsat.

Term	Meaning in this document
Unsat core	A sufficient subset of tracked assertions/assumptions whose conjunction is unsatisfiable; not guaranteed minimal.
Valuation	Assignment of concrete values to a selected set of symbols, often the source inputs requested by <code>get-value</code> .
Verified receipt	Opaque Leant value recording callback acceptance of one exact candidate occurrence.
Witness	A concrete or symbolic object demonstrating satisfiability, violation, reachability, or another existential claim.

Repository snapshots and document map

E.1 Exact revisions inspected

Repository	Commit	Role	Notes
Leant	494259dbd85ae2e6749246b08377f44b2f9936ac	Current integration	Default branch head inspected on August 16, 2026. Contains rank/filter command path and progressive selection context.
Djex submodule	7d65eba6f753727ae7b3d734fb4a7e00f1f44aa7	Code Leant is pinned to	The lib/Djex submodule target in the inspected Leant tree. Architectural claims about callable code use this revision.
Djex head	0cf82f2aa22debb1de2c7aaa13a2abbe190550d8	Additive current development	Default branch head inspected shortly after the Leant snapshot. Used only where explicitly distinguished from the pin.
Z3	bc4585e0ba5b1041f301b0c16ba652645729e796	Solver implementation snapshot	Default branch head inspected on August 16, 2026; conceptual Z3 features are also described from official guides/standards.

All source links in this document are commit-pinned. The repositories are active, so a future default branch may differ substantially.

E.2 Existing Leant LaTeX documents

The new introduction is designed to precede and complement four existing documents rather than replace them.

Document	Best use	Relationship to this introduction
docs/Leant_Overview/Leant_Overview.tex	User-facing Leant manual and REPL orientation	Explains Leant broadly. This introduction supplies the logic/solver prerequisites and behavioral-assessment internals.
docs/Djex_Leant_Codebase_Walkthrough/Djex_Leant_Codebase_Walkthrough.tex	Maintainer-level source tour of both codebases	Read after Parts IV and VI for a wider module-by-module understanding beyond Z3/Length.

Document	Best use	Relationship to this introduction
docs/Leant_Djex_Lean4_Rewrite_Analysis/Leant_Djex_Lean4_Rewrite_Analysis.tex	Architectural analysis of possible Lean rewrite/partitioning	Assumes familiarity with Leant/Djex boundaries; this introduction clarifies why solver execution and proof authority are distinct.
docs/Z3_Behavioral_Synthesis_Proposal/Z3_Behavioral_Synthesis_Proposal.tex	Detailed proposal for precise behavioral synthesis	The closest companion. It develops architecture and domains at expert level; this introduction teaches Z3 first, updates the current hard-filter status, and explains every proposal mechanism from beginner principles.

The proposal was tied to an August 15 snapshot at which Length was described as ranking-only. The Leant snapshot inspected for this document has already implemented the narrow replay-authorized filter path. Full solver-guided sketches, general CEGIS, richer domains, CHCs, and proof artifacts remain planned.

E.3 Recommended reading paths

Absolute beginner to Z3.

Parts I–III, Appendix A, then Chapters 18–21.

Leant user enabling Length assessment.

Chapters 19, 20, and 22, followed by the current Leant manual and configuration source comments.

Maintainer debugging a result.

Chapters 23, 32, and 33, then follow commit-pinned source trails.

Implementer of CEGIS/sketches.

Chapters 25, 26, 27, and 29, then the existing proposal.

Implementer of a new domain.

Chapters 28 and 30; use the current Length domain as the reference implementation for sealing, canonical encoding, decoding, replay, and evidence association.

Lean proof integration.

Chapters 30 and 34, then inspect existing theorem automation appropriate to the selected domain.

E.4 Where each key concept is introduced

The glossary defines terms; this table says where each one is first developed at length, so that a reader arriving from a later chapter can jump back.

Concept	Introduced in
Constraint, satisfiability, model	Chapter 1; formal vocabulary in Chapter 2
SAT versus SMT, theories	Chapter 3; individual theories in Chapter 8
sat/unsat/unknown and what unknown is not	Chapter 4
Z3 executable, library, options	Chapter 5
SMT-LIB syntax, declare-const, assert, check-sat	Chapter 6; reference card in Appendix A
Linear integer arithmetic, QF_LIA, ite, monus lowering	Chapter 7

Concept	Introduced in
Quantifiers, recursive definitions, incompleteness	Chapter 9
Models, get-value, response parsing	Chapter 10
Assumption literals, unsat cores, minimize/assert-soft	Chapter 11
Constrained Horn clauses, Spacer, declare-rel/rule/query	Chapter 12; planned use in Chapter 30
Bad state $P \wedge \neg Q$, over-/under-approximation, role vector	Chapter 13
Typed plan, canonical bytes, fingerprint, schema tags	Chapter 14; real bytes in Section 32.11
Process ownership, digest, empty environment, framing	Chapter 15; exact launch in Section 32.8.1
Observation versus evidence, replay, opaque receipts	Chapter 16
Incrementality, determinism, counterexample bank	Chapter 17; Leant's bank in Chapter 22
Four-layer architecture, Verified receipt, exact typed origin	Chapter 18
Spine, passive contract, provider law, case policy, sealing order	Chapter 19
Length QF-LIA translation, Euclidean witnesses, input-only models	Chapter 20
Live worker, capability probe, budgets, artifact policy	Chapter 21
Startup options, :synth request syntax, rank versus filter	Chapter 22; refusals in Section 33.12
Source map of one assessment	Chapter 23
Counterexample-guided enumeration, semantic signature	Chapter 25
Sketch, selector, materialization	Chapter 26
Nogood, sound prefix pruning	Chapter 27
Richer domains, exactness descriptor	Chapter 28
Staged plan	Chapter 31
Reading a query, troubleshooting, crosswalk	Chapters 32, 33, 34

E.5 Primary source paths

E.5.1 Leant

- [src/Main.hs](#): REPL state and progressive synthesis/assessment scheduling.
- [src/Leant/Options.hs](#): startup flags and activation source.
- [src/Leant/Synth/Engine.hs](#): typed candidates, renderers, exact origins.
- [src/Leant/Synth/Verification.hs](#): callback batches and opaque verified receipts.
- [src/Leant/Synth/PostVerification.hs](#): permutation-only post-verification boundary.
- [src/Leant/Synth/BehavioralSelection.hs](#): generic occurrence-sealed hard partition.
- [src/Leant/Synth/Length/Contract.hs](#): passive Lean-named contract source.
- [src/Leant/Synth/Length/Handoff.hs](#): exact verified-origin to Djex problem conversion.
- [src/Leant/Synth/Length/Configuration.hs](#): reusable execution/replay policy.

- `src/Leant/Synth/Length/Configuration/File.hs`: bounded startup grammar.
- `src/Leant/Synth/Length/Integration.hs`: request contexts, activation, rank/filter orchestration.
- `src/Leant/Synth/Length/Ranking.hs`: public conservative ranking report.
- `src/Leant/Synth/Length/Selection.hs`: scalar hard-filter policy.
- `src/Leant/Synth/Length/CounterexampleBank/Internal.hs`: package-private nominal bank state and context (there is no public `CounterexampleBank.hs` facade at the pinned revision).
- `src/Leant/Synth/Length/Presentation.hs`: bounded user-facing assessment notes.
- `src/Leant/Synth/Length/Command.hs`: :synth option grammar and its five syntax refusals.
- `src/Leant/Synth/Length/File/Acquire.hs`: shared bounded file acquisition (POSIX only; Windows fails closed).
- `src/Leant/Synth/Length/Adapter.hs`: checked query wrapper and detailed refusal mapping.

E.5.2 Djex

- `synthesis/src/Language/Haskell/Synthesis/Semantic/Problem.hs`: generic behavioral problem envelope.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Observation.hs`: solver status and evidence vocabulary.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Length.hs`: public `Length` aggregate.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Problem.hs`: checked contracts/problems.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Evaluate.hs`: concrete replay and finite validation.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/CounterexampleBank.hs`: replay-input banks.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib.hs`: canonical query and replay facade.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Execution.hs`: pure execution policy.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs`: scoped live worker facade.
- `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Response.hs`: bounded response vocabulary.
- `synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib.hs`: translator and model checks.
- `synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/QFLIA.hs`: typed restricted SMT-LIB AST/renderer.
- `synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Z3/Process.hs`: process ownership.
- `synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Causal/Driver.hs`: causal command/response driver.

E.5.3 Z3

- `README.md`: build/API overview and official links.
- `src/smt/`: core SMT solver implementation.
- `src/ast/`: expressions, sorts, declarations, and AST infrastructure.
- `src/muz/`: fixedpoint/CHC infrastructure including Spacer-related code.
- `src/opt/`: optimization infrastructure.
- `examples/`: API examples in several languages.

Further reading

F.1 Official Z3 material

- The [Z3 Guide](#) gives feature-oriented introductions to logics, arithmetic, arrays, bit-vectors, sequences, datatypes, quantifiers, optimization, recursive functions, and fixedpoints.
- The paper/tutorial [Programming Z3](#) develops solver APIs, incrementality, models, cores, tactics, and fixedpoints in more depth.
- The [official Z3 repository](#) contains source, build instructions, examples, release history, and documentation links.
- The [SMT-LIB initiative](#) publishes the language standard, logic declarations, theory definitions, and benchmark infrastructure. The current standard line at the time of research is SMT-LIB 2.7.

F.2 Foundational perspective

For a reader continuing beyond this document, useful topics are:

- propositional SAT, DPLL, and conflict-driven clause learning;
- Nelson–Oppen-style theory combination and congruence closure;
- decision procedures for Presburger arithmetic and bit-vectors;
- E-matching and model-based quantifier instantiation;
- abstract interpretation and Galois connections;
- CEGIS, version spaces, observational equivalence, and SyGuS;
- IC3/PDR and constrained Horn clause solving;
- proof-producing SMT and independently checkable certificates;
- mechanized semantics and refinement proofs in Lean.

These topics explain how Z3 works internally. They are not prerequisites for following the current Length integration, whose public contract is intentionally much narrower.

F.3 Citation list

Bibliography

- [1] Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, 2008. Project home: <https://github.com/Z3Prover/z3>.
- [2] Nikolaj Bjørner, Leonardo de Moura, Lev Nachmanson, and Christoph Wintersteiger. Programming Z3. <https://z3prover.github.io/papers/programmingz3.html>.
- [3] The Z3 Project. The Z3 Guide. <https://microsoft.github.io/z3guide/>.
- [4] The SMT-LIB Initiative. SMT-LIB Standard, logics, theories, and resources. <https://smt-lib.org/>.
- [5] The Leant contributors. Leant repository, inspected at commit 494259dbd85ae2e6749246b08377f44b2f9936ac. <https://github.com/VladimirReshetnikov/Leant>.
- [6] The Djex contributors, building on Lennart Augustsson’s Djinn and Lennart Spitzner’s Exference. Djex repository, Leant-pinned commit 7d65eba6f753727ae7b3d734fb4a7e00f1f44aa7; independent head 0cf82f2aa22debb1de2c7aaa13a2abbe190550d8. <https://github.com/VladimirReshetnikov/Djex>.
- [7] Leant Manual. [docs/Leant_Overview/Leant_Overview.tex](#).
- [8] Djex and Leant Codebase Walkthrough. [docs/Djex_Leant_Codebase_Walkthrough/Djex_Leant_Codebase_Walkthrough.tex](#).
- [9] Leant/Djex Lean 4 Rewrite Analysis. [docs/Leant_Djex_Lean4_Rewrite_Analysis/Leant_Djex_Lean4_Rewrite_Analysis.tex](#).
- [10] Z3 Behavioral Synthesis Proposal for Leant and Djex. [docs/Z3_Behavioral_Synthesis_Proposal/Z3_Behavioral_Synthesis_Proposal.tex](#).

Conclusion

Z3's core idea is modest and extraordinarily useful: describe a mathematical situation by constraints and ask whether any interpretation satisfies them. For behavioral verification, write the unwanted situation—precondition true and postcondition false. A satisfying model suggests a counterexample; unsatisfiability says the encoded bad state has no model. Everything else in a production synthesis system is careful bookkeeping about what was encoded, which executable answered, how the response was parsed, and what evidence the application is authorized to derive.

Leant and Djex already embody that discipline in a narrow but substantial implementation. Lean verifies exact candidate occurrences. Leant recovers exact typed origins and binds passive Lean-named contracts to retained provenance. Djex interprets a finite-spine Length fragment, constructs canonical QF-LIA bad states, fingerprints typed plans and bytes, owns a bounded capability-probed Z3 worker, and accepts model values only after independent replay. Leant can conservatively rank every verified candidate or, under explicit filter mode, reject only exact occurrences carrying replayed counterexamples.

The planned next steps do not change the division of authority. Counterexamples will guide Djex enumeration. Z3 will choose selectors in finite Djex-prepared sketches, derive scoped nogoods and optimization choices, and later reason about richer arithmetic, sequence, bit-vector, state, and recursive relational models. Lean will still elaborate every materialized term and check every claimed source-level theorem. The system becomes more precise not by declaring one component omniscient, but by making the conversions between components explicit, typed, bounded, fingerprinted, replayed, and—where a universal claim matters—proved.